

Natural Language Processing

[Notes by Piai Luca]

Academic Year 2025-2026

Contents

1	Words and Tokens	1
1.1	Words	1
1.2	Morphology	3
1.3	Word-forms and lemmas	3
1.4	Corpora	4
2	Text normalization	5
2.1	Language identification	5
2.2	Spell checker	5
2.3	Contractions	6
2.4	Word reconstruction	6
2.5	Punctuation marks	6
2.6	Special characters	6
2.7	Tokenization	7
2.8	Sentence segmentation	8
2.9	Case folding	9
2.10	Stop words	9
2.11	Stemming	9
2.12	Lemmatization	9
2.13	Should we always normalize?	9
2.14	Byte-Pair Encoding	10
2.14.1	BPE: learner	10
2.14.2	BPE: encoder	11
2.14.3	BPE: decoder	11
2.15	Byte-level BPE	11
2.16	WordPiece	12
3	Word Embeddings	13
3.1	Lexical semantics	13

3.2	Distributional semantics	14
3.3	Word Embedding task	15
3.4	Count-based embeddings	15
3.5	Neural word embeddings	16
3.5.1	Static vs Dynamic embeddings	17
3.5.2	Word2vec and Skip-Gram	17
3.6	Practical issues and tricks	20
3.7	Miscellanea	21
3.7.1	FastText	21
3.7.2	GloVe	21
3.8	Visualizing Embeddings	22
3.9	Semantic Properties and Context Window size	22
3.10	Fairness and Bias	22
3.11	Evaluating Embeddings	23
3.12	Cross-lingual word embedding	23
4	Statistical Language Models	25
4.1	Language Modeling	25
4.2	Probability estimation	26
4.3	N-gram model	27
4.4	Evaluating the model: Perplexity	28
4.5	Sampling from a Language Model	29
4.6	Smoothing, Interpolation, and Backoff	29
4.6.1	Laplace Smoothing	29
4.6.2	Backoff and Interpolation: idea	30
4.6.3	Stupid backoff	31
4.6.4	Linear Interpolation	31
4.6.5	Unknown Words	31
4.7	Limitations of LM	32
5	Natural Language Processing	33
5.1	Feedforward NLM	34
5.1.1	Inference	34
5.1.2	Training and Loss Function	35
5.2	Recurrent NLM	36
5.2.1	Inference	36
5.2.2	Training and Loss Function	37

5.2.3	Teacher forcing	37
5.3	Practical Issues	38
6	Contextual Word Embeddings	41
6.1	Static vs Dynamic Embeddings	41
6.2	ELMo	41
6.2.1	Training	43
6.3	BERT	43
6.3.1	Masked language modeling	44
6.3.2	Next sentence prediction	46
6.3.3	Inference	47
6.4	BERT Applications	48
6.4.1	Sentiment Analysis	48
6.4.2	Named Entity Recognition	49
6.5	SBERT	50
6.5.1	BERT Limitation	50
6.5.2	Sentence-BERT: Idea	51
6.5.3	Compute Embeddings from BERT	51
6.5.4	SBERT Training	51
6.6	SBERT Applications	52
7	LLM: Pre-Training	55
7.1	Introduction	55
7.1.1	Structure of an LLM	55
7.1.2	Modeling tasks as text completion	56
7.1.3	Few-shot and Zero-shot learning	57
7.2	Sampling	58
7.2.1	Greedy decoding	58
7.2.2	Random sampling	58
7.2.3	Top-k and Top-p sampling	59
7.2.4	Temperature sampling	59
7.3	Pre-training	60
7.3.1	Training corpora	61
7.4	Key-Value Cache	62
7.5	Scaling Laws for LLMs	63
7.6	Multi-Lingual LLMs	66
7.6.1	Pre-training	66

8	LLM: Post-Training	69
8.1	Introducing Post-Training	69
8.2	Fine-Tuning	70
8.2.1	Supervised Fine-Tuning	71
8.3	Instruction Tuning	71
8.3.1	Instructions as Training Data	71
8.3.2	Datasets for Instruction Tuning	72
8.3.3	Evaluation of Instruction-Tuned LLMs	74
8.4	Model Alignment	74
8.4.1	Preference-Based Learning	74
8.4.2	LLM Preference Data	75
8.4.3	Modeling Preferences	75
8.4.4	Learning to Score Preferences	77
8.5	Alignment via Preference-Based Learning	78
8.5.1	Reinforcement Learning and LLMs	79
8.5.2	Direct Preference Optimization	81
8.6	Parameter Efficient Fine Tuning	83
8.6.1	LoRA	84
8.7	Transfer Learning	85
9	ChatBot and Prompting	87
9.1	ChatBot	87
9.2	Prompting	87
9.2.1	Why does prompt learning work?	88
9.2.2	Prompt Learning vs. Fine-Tuning	88
9.2.3	Prompt engineering	88
9.3	Ethics	88
10	Retrieval Augmented Generation	91
10.1	A two step process: retrieval and generation	91
10.2	Step 1: retrieve information	92
10.2.1	Naive approach	92
10.2.2	Bi-encoder	93
10.2.3	ColBERT	94
10.2.4	Ranking documents	95
10.3	Step 2: generate a response	95
10.4	Advanced RAG methods	96

10.5 Datasets for RAG	97
10.5.1 Evaluating question answering systems	97
11 Part of speech tagging	99
11.1 Introduction to Part-of-Speech Tagging	99
11.2 Hidden Markov Models (HMMs)	100
11.2.1 Probability estimation	101
11.2.2 HMMs as Automata	101
11.2.3 Decoding with the Viterbi Algorithm	102
11.2.4 Limitations	104
11.3 Neural POS Taggers	104
11.3.1 Fixed-window models	104
11.3.2 Recurrent Neural Models	104
11.3.3 Bidirectional RNNs	105
11.4 Sequence Labelling and MRLs	105
12 Dependency Tree	107
12.1 Introduction	107
12.2 Dependency Formalisms and Projectivity	108
12.3 Transition-Based Dependency Parsing	108
12.3.1 The Arc-Standard Model	109
12.4 Generating training data	109
12.4.1 How to generate training data	110
12.5 Feature Extraction (Pre-Neural)	111
12.5.1 Feature functions	111
12.6 Neural Dependency Parsing: BiLSTMs	112
12.7 Evaluation Metrics	114
13 Machine translation	115
13.1 Introduction to Machine Translation	115
13.2 Neural Machine Translation (NMT)	115
13.3 Encoder-Decoder Networks	116
13.3.1 RNN-Based Architectures	117
13.3.2 The Attention Mechanism	118
13.3.3 Transformer-Based Architectures	120
13.4 Inference with beam search	121
13.5 Data and Evaluation	121

Chapter 1

Words and Tokens

1.1 Words

Definition 1 (Types and tokens). *There are two ways of talking about words:*

1. **types** are the distinct words appearing in a document;
2. **tokens** are the individual occurrences of words in a document.

For example:

They picnicked by the pool, then lay back on the grass and looked at the stars.

Ignoring punctuation marks, the above sentence has 16 tokens and 14 types. We need to distinguish between words and **punctuation marks**. Punctuation is critical for finding sentence boundaries (period, colon, etc.) and for identifying some aspects of meaning (question mark,, etc.).

Definition 2 (Vocabulary). *The set of all types in a corpus is the vocabulary V .*

The vocabulary size $|V|$ is the number of types in the corpus, instead the size of the corpus N is the number of tokens, if we ignore punctuation marks.

Corpus	Tokens = N	Types = $ V $
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google n-grams	1 trillion	13 million

Figure 1.1: N vs $|V|$ for a few English corpora.

Herdan/Heaps law Is there a relation between N and $|V|$? The Herdan/Heaps law answers to that question:

$$|V| = kN^\beta \quad (1.1)$$

where $k \in [10, 100]$ and $\beta \in [0.4, 0.75]$.

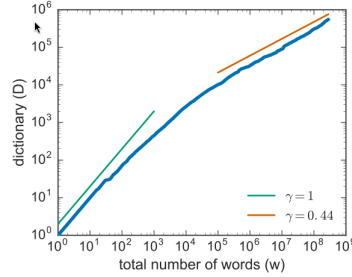


Figure 1.2: Herdan/Heaps law.

The fact that words grow without end leads to a problem for any computational model: no matter how big our vocabulary, we will never have a vocabulary that captures all the possible words that might occur. That means that our computational model will constantly see **unknown words** (or **out of vocabulary** words (OoV)): words that it has never seen before. This is a huge problem for machine learning models.

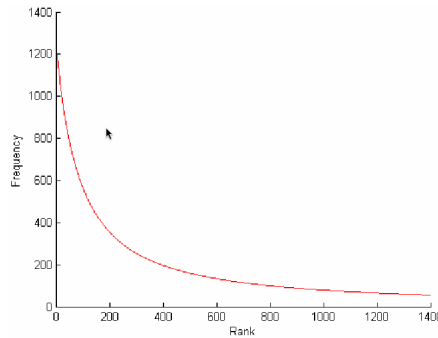


Figure 1.3: Zipf/Mandelbrot law.

Zipf/Mandelbrot law In very large corpora, the r -th most frequent type has frequency $f(r)$ that scales according to the Zipf/Mandelbrot law:

$$f(r) \propto \frac{1}{(r + \beta)^\alpha} \quad (1.2)$$

with $\alpha \simeq 1$ and $\beta \simeq 2.7$.

A very small number of words (like "the", "of", "and") are used a massive amount of times, instead a massive number of words are used very rarely.

1.2 Morphology

Definition 3 (Morphology and morphemes). *Morphology is the study of how words are composed by morphemes, which are the smallest meaningful units of language.*

The structure of a word consists of several morphemes:

- one **root** or stem;
- zero or more **affixes**, such as prefixes and suffixes.

For example:

draw, draw+s, draw+ing+s, un+draw+able.

We can distinguish between:

- **Inflectional morphology**: no change in the grammatical category. For example give, given, gave, gives are all verbs.
- **Derivational morphology**: change the grammatical category. For example process (name), processing (verb), processable (adjective), processor (name).

Definition 4 (Morphologically Rich Language). *The term morphologically rich language (MRL) refers to a language in which substantial grammatical information is expressed at word level.*

1.3 Word-forms and lemmas

Definition 5 (Word-form and lemma). *The word-form is the full inflected or derived form of the word. Each word-form is associated with a single lemma, the citation form used in dictionaries.*

For example: word-forms¹ sing, sang, sung are associated with the lemma sing. Alternatively to $|V|$, another measure of the number of words in a corpus is the **number of lemmas**.

¹When linguists talk about a "word-form", they just mean the word exactly as it appears in a sentence, after any grammatical changes have been applied to it.

Definition 6 (Multi-element word-forms). *A word-form can be made of several separated elements or of several merged elements.*

For example (separated elements):

English: **San Francisco**; French: **sine die** (from Latin).

and (merged elements):

English: **don't** = do + not; Spanish: **damélo** = da + me + lo (give + to me + it).

There is **not one-to-one correspondence** between tokens and word-forms.

1.4 Corpora

Definition 7 (Corpus). *Corpus (plural corpora) is a large collection of text, in computer-readable form, with possible addition of linguistic annotations.*

Several dimensions of variation for corpora should be taken into account (language, genre, etc.) when developing a NLP application and it is important to test NLP algorithms on more than one language.

Definition 8 (Low-resource languages). *Languages lacking large corpora are considered low-resource languages.*

Chapter 2

Text normalization

Definition 9 (Text normalization). *Text normalization is the process of transforming a text into some predefined standard form.*

There is no all-purpose normalization procedure: text normalization depends on

- what type of text is being normalized;
- what type of NLP task needs to be carried out afterwards.

In the following we briefly overview the most common tasks involved in text normalization. The ordering in which we present the modules is important; some of them are optional.

2.1 Language identification

Language identification is the task of detecting the source language for the input text. This is preliminary to spell checking, tokenization, acronym expansion, etc.

2.2 Spell checker

Spell checkers correct grammatical mistakes in text. Useful for quickly written text. Spell checkers use approximate string matching algorithms such as **Levenshtein distance** to find correct spellings.

This task has to deal with **difficult cases** such as *misspelled word* that might still be in the language. To deal with these cases, more sophisticated algorithms analyze the **context** formed by the surrounding words.

2.3 Contractions

The following should be managed before further normalization:

- **contracted forms:** `we'll`, `don't`
- **abbreviations:** `inc.`, `Mr.`
- **slang:** `IMHO`, `LOL`, `tl;dr`

Difficult cases: ambiguity with apostrophe which is also used as *genitive marker* (`book's cover`) and *quotative* (`'The other class', she said`).

The most straightforward technique is to create a **dictionary of contractions** and abbreviations with their corresponding expansions.

2.4 Word reconstruction

Uneven kerning may break words in unexpected ways (especially in pdf documents). For example: Italian word `assicurazione` may appear in a document as `assicuraz ione`.

2.5 Punctuation marks

Punctuation marks in text need to be isolated and treated as if they were separate words. This is critical for finding sentence boundaries and for identifying some aspects of meaning (question marks, exclamation marks, quotation marks).

Difficult cases: many punctuation symbols also used in abbreviations (period), company names (`Yahoo!`), compound words (`-`), etc.

2.6 Special characters

Special care for

- **emoticons:** `:)` `;) etc.`, use regular expressions.
- **emoji:** use specialized libraries.

2.7 Tokenization

Definition 10 (Tokenization). *Tokenization is the process of segmenting text into units called tokens.*

Tokenization techniques can be grouped into three families:

1. **Word tokenization**
2. **Character tokenization**
3. **Subword tokenization**

Tokens are then organized into a *vocabulary* and, depending on the specific NLP application, may later be mapped into **natural numbers**.

Why tokenize the input? One reason is that converting an input to a deterministic fixed set of units means that different algorithms and systems can agree on simple questions. For example, *How long is this text?* (*How many units are in it?*). Or: Is *don't* or *New York* one token or two? Standardizing is thus essential for replicability in NLP experiments, and many algorithms that we introduce assume that all texts have a fixed tokenization.

Word tokenization For English, most of the text is already (word) tokenized after previous steps, with the exception of *spacial compound names* (**white space** vs **whitespace**) and of *city names, companies*, etc. We will manage those cases with named entity recognition. Also there are certain *language-independent tokens* that require specialized processing.

A **main problem** with word tokenization is represented by unknown words. Why is that? As we will see in the next chapter, NLP algorithms often learn some facts about language from one corpus (a training corpus) and then use these facts to make decisions about a separate test corpus and its language. Thus if our training corpus contains, say the words *low*, *new*, and *newer*, but not *lower*, then if the word *lower* appears in our test corpus, our system will not know what to do with it.

Character tokenization Major east Asian languages (e.g., Chinese, Japanese, Korean, and Thai) write text without any spaces between words. For most Chinese NLP tasks, character tokenization works better than word tokenization: each character generally represents a single unit of meaning. Word tokenization results in huge vocabulary, with large number of very rare words. For Japanese and Thai the character is too small a unit,

and algorithms for word segmentation are required. Standard segmentation algorithms for these languages use neural sequence models. The **main limitation** of character tokenization is that it is very hard to understand the meaning of a sentence and to reconstruct the words.

Sub-word tokenization To deal with this unknown word problem, modern tokenizers automatically induce sets of tokens that include tokens smaller than words, called **sub-words**. Sub-words can be arbitrary substrings, or they can be meaning-bearing units like the morphemes **-est** or **-er**. Sub-word tokenization reduces vocabulary size, and has become the most common tokenization method for large language modelling and neural models in general.

Subword tokenization is inspired by algorithms originally developed in information theory as a simple and fast form of data compression alternative to Lempel-Ziv-Welch (data compression provides more interesting results than morphemes).

A **sub-word tokenization scheme** consists of three different algorithms:

1. The **token learner**: takes a raw training corpus and induces a set of tokens, called **vocabulary**.
2. The **token segmenter** (encoder) takes a vocabulary and a raw test sentence, and segments the sentence into the tokens in the vocabulary.
3. The **token merger** (decoder) takes a token sequence and reconstructs the original sentence.

The three algorithms widely used for sub-word tokenization are: *byte-pair encoding (BPE)*; *unigram language modeling*; *WordPiece*. The most used one (ChatGPT, Gemini, etc.) is BPE.

2.8 Sentence segmentation

Text normalization also includes sentence segmentation: breaking up a text into individual sentences. This can be done using cues like periods, question marks, or exclamation points.

2.9 Case folding

Lower casing is very useful to standardize words and before stop words removal. After this step, you can perform word statistics, such as Zip's law. Difficult cases: if some words must be capitalized (proper names, cities), specialized pre-processing is needed. Sometimes it might be useful to keep both versions of the text data.

2.10 Stop words

Stop word removal includes getting rid of: common articles; pronouns; prepositions; coordinations. We get rid of words (e.g. **the**, **while**) that doesn't provide much information. Stop word removal heavily depends on the task at hand, since it can wipe out relevant information.

2.11 Stemming

Stemming refers to the process of slicing a word with the intention of removing affixes. Stemming is problematic in the linguistic perspective, since it sometimes produces words that are not in the language, or else words that have a different meaning (e.g. **caring** > **car**).

2.12 Lemmatization

Lemmatization has the objective of reducing a word to its base form, also called lemma, therefore grouping together different forms of the same word (e.g. **am**, **are**, **is** > **be**). **Lemmatization and stemming are mutually exclusive**, and the former is much more resource-intensive than the latter.

2.13 Should we always normalize?

We need to ask ourselves:

1. is important information being lost?
2. Is noisy information being removed?

Text normalization is about removing noise.

2.14 Byte-Pair Encoding

Byte-Pair Encoding (BPE) is a widely used algorithm for sub-word tokenization.

2.14.1 BPE: learner

The BPE token learner is usually run inside words, not merging across word boundaries. To this end, use a special end-of-word marker.

The algorithm iterates through the following steps:

1. Begin with a vocabulary composed by all individual characters.
2. Choose the two symbols A, B that are most frequently adjacent.
3. Add a new merged symbol AB to the vocabulary
4. Replace every adjacent A, B in the corpus with AB .

The algorithm stops when the vocabulary reaches size k (hyperparameter).

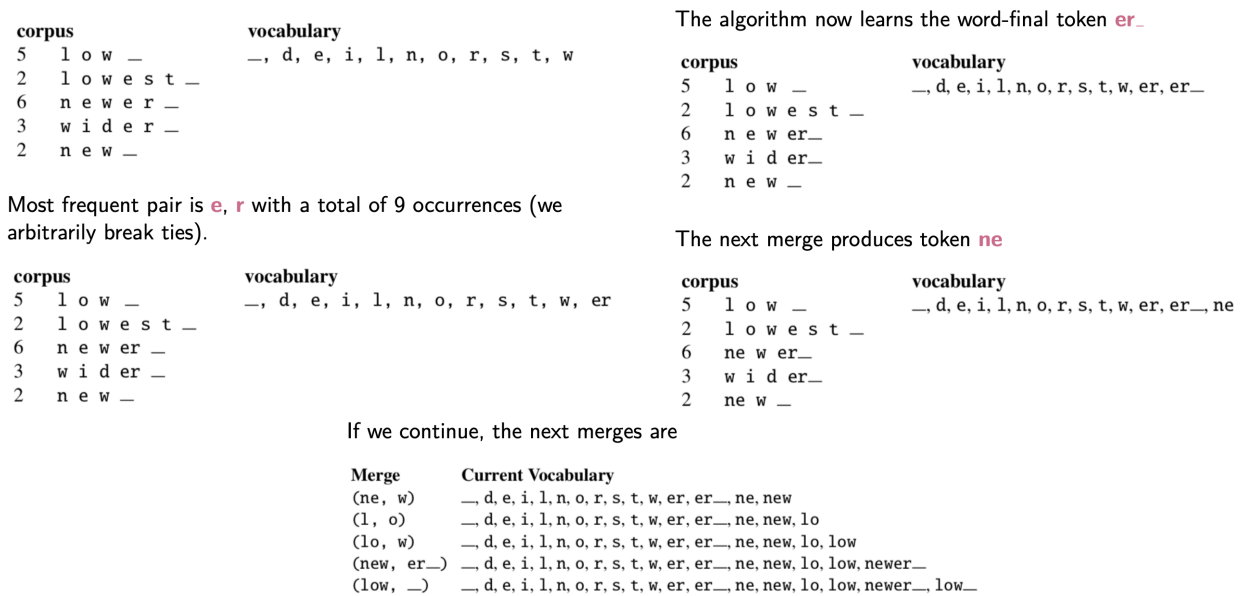


Figure 2.1: Example: underscore is the end-of-word marker.

After several iterations, BPE learns entire words and most frequent units, useful for **tokenizing unknown words**.

2.14.2 BPE: encoder

There are two version of the BPE token segmenter:

1. **Version 1:** split data into characters and apply merge rules (obtained from BPE learner) **in frequency order**.
2. **Version 2:** for each word, left-to-right, match **longest token from vocabulary**.

From the theory point of view, we don't know if the two *always* provide the same encoding. Most of the times they do.

Caching This process is computationally expensive; for this reason many systems use some form of caching:

1. Pre-tokenize all the words and store how a word should be tokenized in a dictionary (i.e. look up table).
2. When an unknown word is seen (**cache miss**): apply the encoder to tokenize the word and add the tokenization to the dictionary for future reference (**cache hit**).

2.14.3 BPE: decoder

The BPE token merger is used to decode, i.e. to reconstruct words. It has to concatenate all the tokens together to get the whole word. In case of **ambiguities** it uses the **end-of-word marker**.

For example, the encoded sequence [the_, high, est_, range_, in_, Seattle_] will be decoded as [the, highest, range, in, Seattle]. A wrong output would be something like: [the, high, estrange, in, Seattle].

2.15 Byte-level BPE

BPE is used at the level of characters. For this reason it has some limitations:

1. **Requires predefined character vocabulary:** we must know every character. But what about non-latin languages?
2. **Language-dependent preprocessing:** needed to deal with non-latin languages.
3. **Struggles with rare unicode characters**

Byte-level BPE (BBPE) solves those limitations by eliminating the character vocabulary entirely. Byte-level BPE is **characters independent**. BBPE is the de facto standard now for LLMs.

BBPE operates directly on **raw UTF-8**, which encodes each character into 1 to 4 bytes¹. Using this solution, the base vocabulary is composed by 256 possible byte values, there are no-unknown characters and also is language agnostic.

After applying BBPE learner, the vocabulary that we get can be composed by

- **Partial characters** shared by different characters.
- **Combination of complete and partial** characters.

After the reconstruction using the decoder, we get again characters in raw UTF-8.

2.16 WordPiece

WordPiece² is a subword tokenization algorithm used by the language model BERT. Like BPE, WordPiece starts from the initial alphabet and learns merge rules. The main difference is the way pair A, B is selected to be merged. The importance of the pair is computed as

$$\frac{f(A, B)}{f(A) \cdot f(B)} \quad (2.1)$$

where $f(X)$ is the frequency of token X . Unlike BPE, that considers only $f(A, B)$, WordPiece prioritizes the merging of pairs where the individual parts are less frequent in the vocabulary.

¹‘Only one byte is needed to represent the 128 characters of the ASCII alphabet, corresponding to Unicode positions $U+0000$ to $U+007F$. Four bytes may seem excessive for a single character; however, this is only required for characters outside the Basic Multilingual Plane’. From wikipedia.

²No longer used.

Chapter 3

Word Embeddings

3.1 Lexical semantics

Definition 11 (Lexical semantics). *The linguistic study of word meaning is called lexical semantics.*

We distinguish between **internal** and **external semantic structure**

Definition 12. *The internal semantic structure of a word refers to the similarity with other words.*

We need to ask ourself: *how meaningful is it to interchange this word with other words?*

Definition 13. *The external semantic structure of a word refers to the allowability to combine with other words.*

We need to ask ourself: *how meaningful is it to combine this word with other words?*

For example¹ consider the phrase: `Alice eats an apple`. If we say `?Alice eats a thunderstorm`, then we have a violation of the internal semantic structure for word `thunderstorm`. If we say `*Alice eats and apple to John`, then we have a violation of the external semantic for word `eat`.

Definition 14. *Word sense disambiguation (WSD) is the task of determining which sense of a word is being used in a particular context.*

Lexical semantics Lexical ambiguity arises because a word can have different meanings, called word senses. It is very difficult to define the notion of word sense in a way that can be understood by computers.

¹Note that ‘?’ marks weak ungrammaticality and ‘*’ marks strong ungrammaticality.

3.2 Distributional semantics

Definition 15. *Distributional semantics develops methods for quantifying semantic similarities between words based on their distributional properties, meaning their neighboring words.*

This idea is based on the so called **distributional hypothesis**.

Definition 16 (Distributional hypothesis). *Linguistic items with similar distributions have similar meanings. In other words, the meaning of a word is defined by its distribution in language use.*

The basic approach is to collect distributional information in **high-dimensional vectors**, and to define distributional/semantic similarity in terms of **vector similarity**. The obtained vectors are called **word embeddings** and this approach is called **vector semantics**.



Figure 3.1: Vectors from sentiment analysis application, projected into 2-dimentional space. Similar words are mapped into "close enough" vectors.

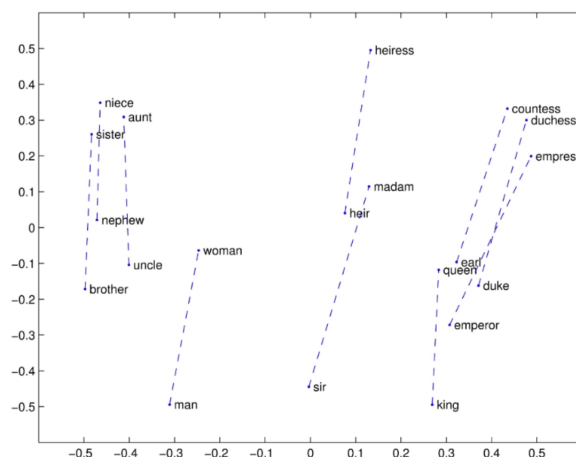


Figure 3.2: Lexical semantic relationships, represented as word vector differences, are generally preserved.

Consider for example that you don't know the meaning of the word **tezgüino**, but you have seen it in the following contexts:

- a) A bottle of _____ is on the table.
- b) Everybody likes _____.
- c) Don't have _____ before you drive.
- d) We make _____ out of corn.

Now consider that we know other words that fit into these context. The four sentences

	a)	b)	c)	d)
tezgüino	1	1	1	1
motor oil	1	0	0	1
tortillas	0	1	0	1
choices	0	1	0	0
wine	1	1	1	0

that we're considering represent the features associated to each word. In the table above we've defined the vectors associated to each word. Based on these vectors, we can conclude that **wine** is very similar to **tezgüino**, instead **choices** is very far from it².

3.3 Word Embedding task

Finding the word embeddings is an **unsupervised** task. There are two families of word embeddings algorithms:

1. **Sparse vectors:** vector components are computed through some function of the counts of nearby words. Those algorithms produces sparse vectors (many 0's).
2. **Dense vectors:** vector components are computed through some optimisation or approximation process. Those algorithms produces dense vectors (few 0's).

3.4 Count-based embeddings

Count-based embeddings approaches are characterized by three main components:

- **Set of term** words (target) $\{w_1, \dots, w_W\}$.

² 'Tegüino is an artisanal corn beer produced by several Uto-Aztecan peoples' from Wikipedia.

- **Set of context** words $\{c_1, \dots, c_C\}$.
- **Term-context matrix** F : matrix of size $W \times C$.

Each term $f_{i,j}$ of matrix F is the number of times context word c_j appears in the context of term word w_i in the corpus. We consider context words, the words appearing inside a **window** of size L , at the left or at the right of w_i .

Consider an example in which we define:

- Term words: **cherry**, **digital**, **information**.
- Context words: **pie**, **data**, **computer**.

Table 3.1: Term context matrix F .

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

By computing the **cosine similarity**³ between vectors associated with term words:

$$\cos(\text{cherry}, \text{information}) = \frac{442 \times 5 + 8 \times 3982 + 2 \times 3325}{\sqrt{442^2 + 8^2 + 2^2} \times \sqrt{5^2 + 3982^2 + 3325^2}} \approx 0.018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 \times 5 + 1683 \times 3982 + 1670 \times 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \times \sqrt{5^2 + 3982^2 + 3325^2}} \approx 0.996$$

Note how the context words are used as features for the term words. Our model states that **information** is way closer to **digital** than it is to **cherry**.

Sparse vectors Using counting-based embeddings leads to sparse vectors: as the number of context words increase, we will get many 0's.

3.5 Neural word embeddings

Neural word embeddings work better than previous embeddings in every NLP task. With respect to previous approaches, neural word embeddings have the following pros:

³Cosine similarity is defined as $\cos(\mathbf{x}, \mathbf{y}) := \mathbf{x}^T \mathbf{y} / \|\mathbf{x}\| \|\mathbf{y}\|$. The closer to 1, the more similar are the vectors.

- Small number of dimensions, in the range [50..1000].
- Dense vectors that can have negative components.

The main limitation is that we lose **interpretability**: the dimensions don't have a clear interpretation.

3.5.1 Static vs Dynamic embeddings

Neural word embeddings can be either static or dynamic:

- **Static**: one embedding for each word in the vocabulary. The single embedding is a mix of all possible meanings.
- **Dynamic** (or contextual): the embeddings of the word depends on the context. The same word can have different embeddings.

3.5.2 Word2vec and Skip-Gram

Word2vec is a software package including two different algorithms for learning word embeddings: **skip-gram with negative sampling** (SGNS) and **continuous bag-of-words** (CBOW). In this course we consider only the first one.

Idea: Instead of counting how often a context word appears near a target word, we train a classifier on the following binary **prediction task**:

Is a given context word likely to appear near a given target word?

After training our model, we use the **learned representation** as the word embeddings. This approach is based on the **contrastive learning** framework.

Skip-Gram with negative sampling Basic steps of skip-gram algorithm:

- For each target word $w_t \in V$:
 1. Treat w_t and any neighboring context word w_c as **positive examples**.
 2. Randomly sample other words $w_n \in V$, called noise words, to produce **negative examples** for w_t .
 3. Train step: classify the word using **logistic regression**.

There is a chance that in step (2) we pick a *context word* (i.e. a positive sample), however the probability associated to this event is really small. To avoid this problem we should check that the sampled word is not a context word of w_t . In practice this is computationally expensive, so we pick at random.

Logistic Regression setup The classifier is trained using logistic regression. For any pair of words $w_t, u \in V$ we define the probability that u is/is not a context word for target word w_t as:

$$\begin{aligned} P(+ | w_t, u) & \text{ (output of our model)} \\ P(- | w_t, u) & = 1 - P(+ | w_t, u). \end{aligned} \tag{3.1}$$

The output of our model is computed using the sigmoid function:

$$P(+ | w, u) = \sigma(\mathbf{e}_t(w) \cdot \mathbf{e}_c(u)) = \frac{1}{1 + \exp(-\mathbf{e}_t(w) \cdot \mathbf{e}_c(u))}. \tag{3.2}$$

where:

- $\mathbf{e}_t(w) \in \mathbb{R}^d$ is the **target embedding**: operator used to assign target words $\rightarrow$ embeddings.
- $\mathbf{e}_c(w) \in \mathbb{R}^d$ is the **context embedding**: operator used to assign context or noise words $\rightarrow$ embeddings.

From the output equation of our model we can understand that, the smaller the angle between the vectors $\mathbf{e}_t(w), \mathbf{e}_c(u) \Rightarrow$ the closer $P(+ | w_t, u)$ to 1.

Loss function For simplicity, let us consider a dataset with only one target/context pair (w, u) , along with k noise words $v_1, v_2, \dots, v_k$ (negative examples). Skip-gram makes the simplifying assumption that all (positive and negative) context words are **independent**.

The objective function is then defined as:

$$\begin{aligned}
LL_w &= \log \left(\overbrace{P(+ | w, u)}^{\text{we want as high as possible}} \prod_{j=1}^k \underbrace{P(- | w, v_j)}_{\text{we want as high as possible}} \right) \\
&= \log P(+ | w, u) + \sum_{j=1}^k \log P(- | w, v_j) \\
&= \log P(+ | w, u) + \sum_{j=1}^k \log(1 - P(+ | w, v_j)) \\
&= \log \sigma(\mathbf{e}_t(w) \cdot \mathbf{e}_c(u)) + \sum_{j=1}^k \log(1 - \sigma(\mathbf{e}_t(w) \cdot \mathbf{e}_c(v_j)))
\end{aligned} \tag{3.3}$$

here the first term expresses that we want the classifier to assign the real context word u a high probability of being a neighbor, and the second term expresses that we want to assign each of the noise words v_j a high probability of being a non-neighbor, all multiplied because we assume independence.

Since in machine learning usually we deal with *minimization* problem, we define the **cross-entropy** as our loss function:

$$L_{CE} = -\log \sigma(\mathbf{e}_t(w) \cdot \mathbf{e}_c(u)) - \sum_{j=1}^k \log(1 - \sigma(\mathbf{e}_t(w) \cdot \mathbf{e}_c(v_j))). \tag{3.4}$$

After each update to minimise L_{CE} we force an **increase in similarity** (dot product) between $\mathbf{e}_t(w)$ and $\mathbf{e}_c(u)$ force a **decrease in similarity** between $\mathbf{e}_t(w)$ and $\mathbf{e}_c(v_j)$, for all of the noise words v_j . After training for several steps (using stochastic gradient descent), we **retain only the target embeddings** in $\mathbf{e}_t \in \mathbb{R}^{|V| \times d}$ matrix, context embeddings in $\mathbf{e}_c \in \mathbb{R}^{|V| \times d}$ are ignored.

Geometric interpretation: why it works? By looking at the loss function used in skip-gram algorithm, we can see that we never measure the similarity between two *target words* w and w' : only the similarity between context and negative samples is considered. So the question is: *how is it possible for similar words to be mapped to nearby points?* Well, this is forced by the context words.

Consider Figure 3.3: vector $\mathbf{e}_t(w)$, after training, lies within the cone formed by the encoding of its context words $\mathbf{e}_c(u_j)$. If target word w' is semantically similar to w , then $\Rightarrow w'$ share the same context words $\Rightarrow \mathbf{e}_t(w)$ and $\mathbf{e}_t(w')$ are forced to be at a small angle.

As a result **similar words cluster together**.

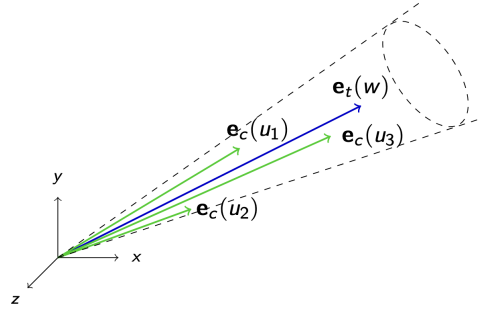


Figure 3.3: Target vector $\mathbf{e}_t(w)$ is surrounded by its context vectors $\mathbf{e}_c(u)$, which form a thin hypercone (dashed lines).

3.6 Practical issues and tricks

Some tricks used to improve the quality of the embeddings.

Text Cleaning and Filtering It is important to **convert punctuation into specific tokens** (such as <PERIOD>). Additionally, you should **remove very infrequent words** (e.g., those appearing 5 times or less), which greatly reduces noise in the data and improves the quality of the resulting vector representations.

Word Indexing To make your code simpler, **convert vocabulary words to non-negative integers** and back again. These integers are typically assigned in descending order of frequency, meaning the most frequent word gets assigned the integer 0.

Subsampling Frequent Words Highly common words like “the,” “of,” and “for” provide very little context and are often regarded as noise. To handle this, models use subsampling, where **occurrences of these frequent words are probabilistically discarded** based on their normalized frequency and a threshold parameter tied to the dataset’s size. The percentage of occurrences of w_i that will be discarded, is defined as:

$$P(w_i) = 1 - \sqrt{\frac{t}{z(w_i)}} \quad (3.5)$$

where $z(w_i)$ is the normalized frequency of w_i , and t is a threshold parameter depending on dataset size.

Tuning Negative Sampling The **ratio k between positive and negative examples is a hyperparameter** that must be tuned on a development set. When selecting noise words for negative examples, it is common to use a relative frequency estimator (instead of uniform distribution) with an alpha corrector, where an alpha of 0.75 tends to provide better performance in most applications. Each word is chosen according to:

$$P_{\alpha}(v) = \frac{f(v)^{\alpha}}{\sum_{w \in V} (f(w))^{\alpha}} \quad (3.6)$$

without the α the equation is just a relative frequency.

Context Window Sizing You must choose a hyperparameter L to specify the maximum context window size. During training, the actual **context window size is chosen randomly at each step** with a uniform distribution between $[1, \dots, L]$. This creates a linear decay effect, meaning words immediately next to the target word are always available ($P = 1$), while words further away are less likely to be included (e.g. words at distance L will be available with $P = 1/L$).

3.7 Miscellanea

3.7.1 FastText

FastText (an extension of word2vec) is an embedding that deals with unknown words and sparsity in languages with rich morphology, by using **subword models**. Each word in fastText is represented by itself plus a bag of character N -grams (i.e. a sequence of N characters), for bounded values of N .

For example, with $N = 3$ the word **where** would be represented by the following strings: **where**, **_wh**, **whe**, **her**, **ere**, **re_**. Then a skip-gram embedding is learned for each N -gram, and the word **where** is represented as the sum of all of the resulting embeddings.

3.7.2 GloVe

Global Vectors, or GloVe for short, is an embedding that accounts for global corpus statistics, which was somehow disregarded by word2vec. GloVe combines the intuitions of count-based models like PPMI, while also capturing the linear structures used by methods like word2vec.

3.8 Visualizing Embeddings

Word embeddings can be used to visualise the meaning of a word w . Most commonly used methods:

- Listing the words in V with highest cosine similarity with w .
- Project the d dimensions of a word embedding down into 2 or 3 dimensions.

3.9 Semantic Properties and Context Window size

The size of the context window L , influences the embeddings that we get. Usually $L \in [2, 20]$ and the exact value depends on the goals of the representation:

- **Small values:** provide semantically similar words with the same parts of speech.
- **Large value:** provide words that are topically related but not similar.

Another semantic property of embeddings is their ability to capture **relational meanings**. Word embeddings can be used to solve analogy problems, usually expressed in the form:

Apple is to tree as grape is to ____.

The **parallelogram model** can be used for solving such problems

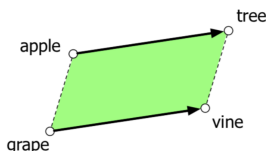


Figure 3.4: Parallelogram model example.

3.10 Fairness and Bias

Word embeddings also reproduce the implicit biases and stereotypes that are latent in the text. This could result in what is called **allocational harm**, when a system allocates resources (jobs or credit) unfairly to different groups.

Embeddings not only reflect the statistics of their input corpora, but have also been reported to amplify bias.

3.11 Evaluating Embeddings

Extrinsic evaluation Use the model to be evaluated in some end-to-end application (as for instance sentiment analysis, machine translation, etc.) and measure performance. It is extrinsic because we evaluate the performance of our systems based on how well it works combined with another one.

Intrinsic evaluation Look at performance of model in isolation, with respect to a given evaluation measure. For word embedding there exist several similarity datasets. As an example three types of intrinsic evaluation are:

1. **Word similarity:** compute a numerical score for the semantic similarity between two words. Example: `car`, `automobile`
2. **Word relatedness:** compute the degree of how much one word has to do with another word. Example: `car`, `highway`
3. **Word analogy:** comparison of two things to show their similarities. Example: `tree : leaf :: flower : petal`

If your system performs well in every metric, then it works great.

3.12 Cross-lingual word embedding

Cross-lingual word embeddings encode words from two or more languages in a shared high-dimensional space.

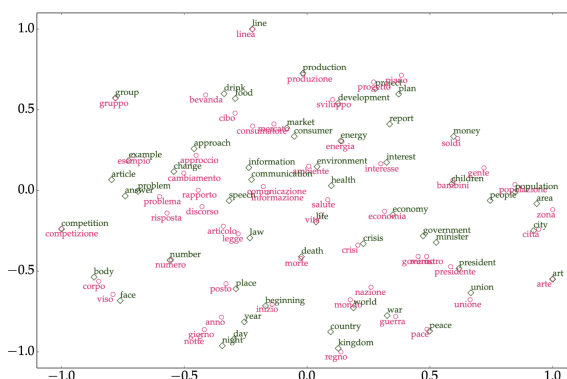


Figure 3.5: Vectors representing words with similar meaning are closely located, regardless of language.

Applications and Benefits They are essential for tasks like machine translation, multilingual lexicon induction, comparing meaning of words across languages and cross-lingual information retrieval. Furthermore, they enable model **transfer** from resource-rich languages to low-resource languages by providing a common representation space

Optimization Challenges The objective function for these models typically combines the monolingual losses of each language along with a regularization term Ω :

$$J = \mathcal{L} + \dots + \mathcal{L}^\ell + \Omega$$

where $\mathcal{L}^i$ is the monolingual loss of the i -th language. Joint optimization of multiple non-convex losses is hard; some tricks are applied.

Multilingual Advantages Extending the training process from bilingual to multilingual settings improves overall results. This improvement is partly driven by the **variation in ambiguity**, meaning that a word's ambiguity in one language does not necessarily transfer to its translation in another, which acts as a form of naturally occurring supervision.

Chapter 4

Statistical Language Models

4.1 Language Modeling

Definition 17 (Language Modeling). *Language modeling (LM) is the task of predicting which word comes next in a sequence of words. More formally, given a sequence of words $w_1 w_2 \dots w_t$ we want to know the probability of the next word w_{t+1} : $P(w_{t+1} \mid w_1 w_2 \dots w_t)$, or in a more compact way $P(w_{t+1} \mid w_{1:t})$.*

This allows language modeling to be treated as a classification task, where the classes are the words in our dictionary.

Definition 18 (Language Modeling). *Rather than as predictive models, language models can also be viewed as generative models that assign probability to a piece of text: $P(w_1 \dots w_t)$.*

In this case, the model provides an answer to the question: *How likely is it that the given text belongs to the modeled language?*

These two views are equivalent, as the probability of a sequence can be expressed as a product of conditional probabilities (using the chain rule):

$$P(w_{1:n}) = \prod_{t=1}^n P(w_t \mid w_{1:t-1}).$$

When $t = 1$ we assume $P(w_t \mid w_{1:t-1}) = P(w_1)$. Conversely, a conditional probability can be expressed as a ratio of two sequence probabilities (using the definition of conditional probability):

$$P(w_{t+1} \mid w_{1:t}) = \frac{P(w_{1:t+1})}{P(w_{1:t})}$$

Why would we want to predict upcoming words? The main reason is that large language models are built just by training them to predict words!! As we'll see, large language models learn an enormous amount about language solely from being trained to predict upcoming words from neighboring words.

Application example Consider correcting grammar or spelling errors like **Their are** two midterms, in which **Therewas** mistyped as **Their**, or **Everything has improve**, in which **improve** should have been **improved**. The phrase **There are** is more probable than **Their are**, and has improved than has improve, so a language model can help users select the more grammatical variant. The same idea is used for speech recognition.

4.2 Probability estimation

How can we compute probabilities? Assume the text **its water is so transparent that**. We want to know the probability that the next word is **the**. One way to estimate the probability is to set

$$P(w_{t+1} \mid w_{1:t}) = \frac{C(w_{1:t+1})}{C(w_{1:t})}$$

For example, to estimate $P(\text{the} \mid \text{its water is so transparent that})$:

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{C(\text{its water is so transparent that the})}{C(\text{its water is so transparent that})}$$

where $C(\cdot)$ counts the number of times the phrase appears in the training corpus.

Observation 1. *To make these probabilities sum up to one, we need to add to the vocabulary V a sentence end marker. In this way, every token in the corpus is always followed by some symbol.*

Observation 2. *Under previous observation, we have $C(w_{1:t}) = \sum_u C(w_{1:t-1}u)$, where u ranges over all symbols (tokens and sentence end marker).*

Definition 19. *Quantities $C(w_{1:t})$ are called **frequencies**. The ratio $C(w_{1:t})/C(w_{1:t-1})$ is called **relative frequency**. The estimator $P(w_{t+1} \mid w_{1:t}) = \frac{C(w_{1:t+1})}{C(w_{1:t})}$ is therefore called the **relative frequency estimator**.*

Problem: high variance The relative frequency estimator will be correct in the limit, that is, with **infinite data**. In practice, consider an aggressive upper bound of $N = 20$ words in our sequences, and an English vocabulary V of size $|V| = 10^5$. Then the number of possible sequences is $|V|^N = 10^{100}$. This is infeasible.

This estimator is extremely **data-hungry**, and suffers from **high variance**: depending on what data happens to be in the corpus, we could get very different probability estimations. To solve this, we need to **introduce some bias into the model**.

4.3 N-gram model

Definition 20 (N-gram). *A string $w_{t-N+1:t}$ of N words is called N-gram.*

Definition 21 (N-gram model). *The N-gram model approximates the probability of a word given the entire sentence history by conditioning only on the past $N - 1$ words.*

The assumption that the probability of a word depends only on a few previous words is called a **Markov assumption**.

The general equation for the N -gram model is

$$P(w_t \mid w_{1:t-1}) \approx P(w_t \mid w_{t-N+1:t-1}) \quad (4.1)$$

The relative frequency estimator for the N -gram model is then

$$P(w_t \mid w_{t-N+1:t-1}) = \frac{C(w_{t-N+1:t})}{C(w_{t-N+1:t-1})} \quad (4.2)$$

For $N = 1$ we have $P(w_t) = \frac{C(w_t)}{\sum_u C(u)} = \frac{C(w_t)}{n}$ where n is the length of the training set.

Observation 3. *The model requires estimating and storing the probability of only $|V|^N$ events, which is exponential in N (a constant), not in the length of the sentence.*

(See example of bigram probabilities page 49 of the book).

How to choose N ? N is a hyperparameter of the model. When setting its value, we face the **bias-variance tradeoff**.

- **N is too small (high bias):** it fails to recover long-distance word relations
- **N is too large (high variance):** we get data sparsity.

Note that: to compute 3-gram probabilities for words at the start of a sentence, we use two markers, e.g. $P(I \mid \langle s \rangle \langle s \rangle)$. Similarly for higher order N -grams.

Train the MLE The relative frequency estimator can be mathematically derived by maximizing the likelihood of the dataset: we maximize the log of the product of all sentence probabilities, under the constrain that probabilities sum up to one. This can be done by solving a constrained optimization problem, using **Lagrange multipliers**.

The relative frequency estimator is also called the **maximum likelihood estimator** (MLE).

4.4 Evaluating the model: Perplexity

Intrinsic evaluation of language models is based on the inverse probability of the test set, normalized by the number of words (or tokens).

Definition 22 (Perplexity). *For a test set $W = w_1w_2 \cdots w_n$ we define perplexity as:*

$$PP(W) = P(w_{1:n})^{-\frac{1}{n}} \\ = \sqrt[n]{\prod_{j=1}^n \frac{1}{P(w_j | w_{1:j-1})}}.$$

Note that because of the inverse, the higher the probability of the word sequence, the lower the perplexity. We can think The inverse probability can be seen as a measure of *how surprising the next word is*. The degree of the root averages over all words of the test set, providing *average surprise per word*. **The lower the perplexity of a model on the data, the better the model.**

Observation 4. *The perplexity of two language models is only comparable if they use identical vocabularies.*

	Unigram	Bigram	Trigram
Perplexity	962	170	109

Figure 4.1: Evaluation of N-gram models on the Wall Street Journal. Training set: 38 million tokens; Vocabulary: 19,979 types; Test set: 1.5 million words. Increasing $N \Rightarrow$ increase the context $\Rightarrow$ decreases the perplexity (up to a certain point).

Intrinsic vs Extrinsic Make sure to use a training corpus that has a similar genre to whatever task you are trying to accomplish. An (intrinsic) improvement in perplexity does not guarantee an (extrinsic) improvement in the performance of a language processing task like speech recognition or machine translation.

4.5 Sampling from a Language Model

Random sampling for LM means that we iteratively choose the next word to generate according to its probability given the context. Thus we are more likely to generate words that have a high probability and less likely to generate words that have a low probability.

Algorithm 1 Text Generation from a Language Model

```

1:  $i \leftarrow 1$ 
2: Sample  $w_i \sim p(w)$ 
3: while  $w_i \neq \text{EOS}$  do
4:    $i \leftarrow i + 1$ 
5:   Sample  $w_i \sim p(w_i \mid w_{<i})$ 
6: end while
7: return  $w_1, w_2, \dots, w_{i-1}$ 

```

4.6 Smoothing, Interpolation, and Backoff

Consider our relative frequency estimator $P(w_{t+1} \mid w_{1:t}) = \frac{C(w_{1:t+1})}{C(w_{1:t})}$. If there isn't enough data in the training set, counts will be zero for some grammatical sequences. Then some of the N-gram probabilities will be zero or undefined.

There are three scenarios we need to consider:

1. **Zero numerator**: solved with smoothing.
2. **Zero denominator**: solved with backoff, interpolation.
3. **Out-of-vocabulary words in test set**: solved with estimation of unknown words.

4.6.1 Laplace Smoothing

Laplace smoothing does not perform well enough, but provides a useful baseline. The **idea** is simple: pretend that everything occurs once more than the actual count. In this way, all the counts that used to be 0 will now have a count of 1, the counts of 1 will be 2, and so on.

Consider for example the 1-gram example. The 1-gram model relative frequency estimator is $P(w_t) = \frac{C(w_t)}{n}$. By applying the Laplace Smoothing we get the **adjust estimate**:

$$P_L(w_t) = \frac{C(w_t) + 1}{n + |V|} \quad (4.3)$$

Since there are $|V|$ words in the vocabulary and each one was incremented, the denominator takes into account the extra $|V|$ observations. Alternatively, we can think of P_L as applying an **adjusted count** C^* to the n actual observations:

$$\begin{aligned} C^*(w_t) &:= \frac{C(w_t) + 1}{n + |V|} \\ P_L(w_t) &= \frac{C^*(w_t)}{n} = \frac{C(w_t) + 1}{n + |V|} \end{aligned} \tag{4.4}$$

This adjusted count is easier to compare directly with the MLE counts.

Similarly, for the 2-grams we have:

$$\begin{aligned} P_L(w_t \mid w_{t-1}) &= \frac{C(w_{t-1}w_t) + 1}{\sum_u [C(w_{t-1}u) + 1]} \\ &= \frac{C(w_{t-1}w_t) + 1}{C(w_{t-1}) + |V|} \end{aligned} \tag{4.5}$$

$$\begin{aligned} C^*(w_t w_{t-1}) &:= \frac{[C(w_{t-1}w_t) + 1] C(w_{t-1})}{C(w_{t-1}) + |V|} \\ P_L(w_t \mid w_{t-1}) &= \frac{C^*(w_t w_{t-1})}{C(w_{t-1})} \end{aligned} \tag{4.6}$$

$P_L(w_t \mid w_{t-1})$ is larger than $P(w_t \mid w_{t-1})$ for 2-gram sequences that occur zero or few times in the training set. However, $P_L(w_t \mid w_{t-1})$ will be much lower (too low) for 2-gram sequences that occur often. So Laplace smoothing is **too crude** in practice.

Add-k smoothing One alternative to add-one smoothing is to move a bit less of the probability mass from the seen to the unseen events. Instead of adding 1 to each count, we add a fractional count $0 \leq k < 1$. For 2-gram the adjusted estimate is:

$$P_{\text{Add-}k}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}w_t) + k}{C(w_{t-1}) + k|V|} \tag{4.7}$$

4.6.2 Backoff and Interpolation: idea

There is an alternative source of knowledge we can draw on to solve the problem of zero frequency n-grams. If we are trying to compute $P(w_n \mid w_{n-2}w_{n-1})$ but we have no examples of a particular trigram $w_{n-2}w_{n-1}w_n$, we can instead estimate its probability by using the bigram probability $P(w_n \mid w_{n-1})$. Similarly, if we don't have counts to compute $P(w_n \mid w_{n-1})$, we can look to the unigram $P(w_n)$. In other words, sometimes using less context can help us generalize more for contexts that the model hasn't learned much about.

4.6.3 Stupid backoff

In a backoff model, if the n -gram we need has zero counts, we approximate it by backing off to the $(n-1)$ -gram. We continue backing off until we reach a history that has some counts.

The stupid backoff is defined as:

$$P_S(w_t \mid w_{t-N+1:t-1}) = \begin{cases} P(w_t \mid w_{t-N+1:t-1}) = \frac{C(w_{t-N+1:t})}{C(w_{t-N+1:t-1})} & \text{if } C(w_{t-N+1:t}) > 0 \\ \lambda P_S(w_t \mid w_{t-N+2:t-1}) & \text{otherwise} \end{cases} \quad (4.8)$$

The backoff terminates in the unigram.

Observation 5. *It is not difficult to show that P_S is not a probability distribution. However, in practical settings stupid backoff turns out to be effective.*

4.6.4 Linear Interpolation

In simple linear interpolation, we combine different order N -grams by linearly interpolating all the models. For example with $N = 3$:

$$P_L(w_t \mid w_{t-2}w_{t-1}) = \lambda_1 P(w_t \mid w_{t-2}w_{t-1}) + \lambda_2 P(w_t \mid w_{t-1}) + \lambda_3 P(w_t) \quad (4.9)$$

For some choice of λ_i s.t. $\sum_i \lambda_i = 1$. When undefined we set upper order models to zero. What are good choices for the λ_i 's? Algorithms exist that attempt to optimise likelihood of training data.

4.6.5 Unknown Words

Unknown words, also called out of vocabulary (OOV) words, are words we haven't seen before. Unknown words can appear in the test set and they're problematic because they cause zero probability.

Solution Replace by new word token $\langle \text{UNK} \rangle$ all words that occur fewer than d times in the training set, d some small number. We proceed to train LM as before, treating $\langle \text{UNK} \rangle$ as a regular word. Finally, at test time, replace all unknown words by $\langle \text{UNK} \rangle$ and run the model.

4.7 Limitations of LM

N-gram language models have several limitations:

1. **Scaling to larger** N-gram sizes **is problematic**, both for computational reasons and because of increased sparsity.
2. **Smoothing techniques are intricate** and require careful engineering to retain a well-defined probabilistic interpretation.
3. Without additional effort, N -gram models are **unable to share statistical strength** across similar words (e.g. **red apple** do not affect estimates for **green apple**, there is no relation between the two sentence. This is not what we want from a LM).

Chapter 5

Natural Language Processing

N-gram language models have been largely supplanted by **neural language models** (NLM).

Aspect	Neural Language Models (NLM)
Advantages	• Can incorporate arbitrarily distant contextual information • Remain computationally and statistically tractable • Generalize better over contexts of similar words • More accurate at word prediction
Disadvantages	• Much more complex than N-gram models • Slower inference and training • Require more time to train • Less interpretable

Most natural choice for NN architecture is recurrent neural network (RNN) but feedforward neural network (FNN) and convolutional neural network (CNN) have also been exploited.

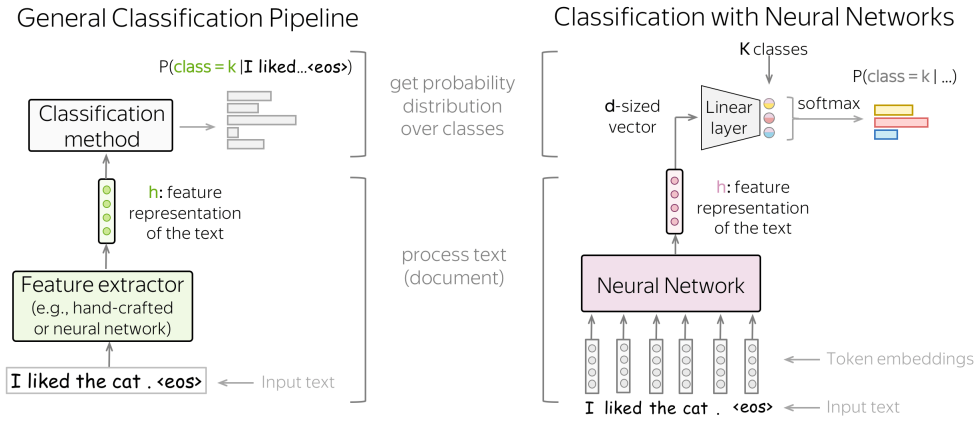


Figure 5.1: General architecture for NLM.

5.1 Feedforward NLM

Like the N-gram language model, the feedforward NLM uses the following approximation $P(w_t | w_{1:t-1}) \approx P(w_t | w_{t-N+1:t-1})$ and a moving window that can see $N - 1$ words into the past.

5.1.1 Inference

Step 1: Encoding Assign to each word $w \in V$ an **index** between $1 \dots |V|$. In the following we will write $ind(w)$. We represent each input word w_t as a one-hot vector $\mathbf{x}_t$ of size $|V|$, defined as follows:

- Element $\mathbf{x}_t[ind(w_t)]$ is set to one.
- All the other elements of $\mathbf{x}_t$ are set to zero.

Step 2: First hidden layer equation First we need to convert one-hot vectors for the words in the $N - 1$ window into word embeddings $\mathbf{x}_{t-i}$ of size d . Then the first hidden layer equation is (assuming $N = 4$):

$$\mathbf{e}_t = [\mathbf{E}\mathbf{x}_{t-3}, \mathbf{E}\mathbf{x}_{t-2}, \mathbf{E}\mathbf{x}_{t-1}]$$

where

- $\mathbf{E} \in \mathbb{R}^{d \times |V|}$ is a learnable matrix containing the word embeddings
- $\mathbf{x}_{t-i} \in \mathbb{R}^{|V| \times 1}$ is a one-hot representation of word w_{t-i}
- $\mathbf{e}_t \in \mathbb{R}^{3d \times 1}$ is the concatenation of the embeddings of the $N - 1$ previous words

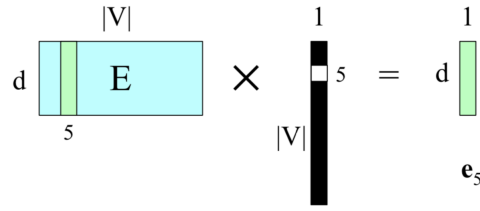


Figure 5.2: Conversion from 1-hot word representation into word embedding.

Step 3: Hidden state and output equations The model equations for the remaining layers

$$\begin{aligned} \mathbf{h}_t &= g(\mathbf{W}\mathbf{e}_t + \mathbf{b}) \\ \mathbf{z}_t &= \mathbf{U}\mathbf{h}_t \\ \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{z}_t) \end{aligned} \tag{5.1}$$

where:

- $\mathbf{W} \in \mathbb{R}^{d_h \times 3d}$: weight matrix for the hidden layer
- $\mathbf{b} \in \mathbb{R}^{d_h}$: bias vector
- $\mathbf{h}_t \in \mathbb{R}^{d_h}$: hidden state after activation g
- $\mathbf{U} \in \mathbb{R}^{|V| \times d_h}$: output weight matrix
- $\mathbf{z}_t \in \mathbb{R}^{|V|}$: output logits
- $\hat{\mathbf{y}}_t \in \mathbb{R}^{|V|}$: predicted probability distribution over the vocabulary

For each $w \in V$, the element of $\hat{\mathbf{y}}_t$ with index $\text{ind}(w)$ is the probability that the word at position t is w :

$$\forall w \in V, \quad \hat{\mathbf{y}}_t[\text{ind}(w)] = P(w_t = w \mid w_{t-3:t-1}) \tag{5.2}$$

(always assuming $N = 4$).

5.1.2 Training and Loss Function

The parameters¹ of the model are $\mathbf{E}$, $\mathbf{W}$, $\mathbf{U}$, $\mathbf{b}$. The number of parameters is $O(|V|)$, since d and d_h are constants. Let w_t be the word at position t in the training data. Then the **true distribution** $\mathbf{y}_t$ for the word at position t is a one-hot vector of size $|V|$ with:

$$\mathbf{y}_t[\text{ind}(w_t)] = 1, \quad \mathbf{y}_t[k] = 0 \quad \forall k \neq \text{ind}(w_t) \tag{5.3}$$

¹Usually the embeddings are initialized using Word2Vec and then we fine tune them.

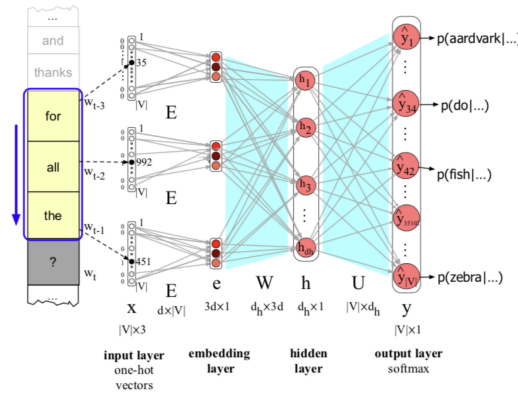


Figure 5.3: Feedforward NLM architecture.

We use the **cross-entropy loss** for training the model:

$$\begin{aligned}
 L_{\text{CE}}(\hat{\mathbf{y}}_t, \mathbf{y}_t) &= - \sum_{k=1}^{|V|} \mathbf{y}_t[k] \log \hat{\mathbf{y}}_t[k] \\
 &= - \log \hat{\mathbf{y}}_t[\text{ind}(w_t)]
 \end{aligned} \tag{5.4}$$

5.2 Recurrent NLM

RNN language models process the input one word at a time, predicting the next word from the current hidden state. The current hidden state is computed from the **current word** and the **previous hidden state**.

Observation 6. *RNNs can model probability distribution $P(w_t \mid w_{1:t-1})$ without the $N-1$ window approximation of Feedforward NLM: the hidden state of the RNN model can (in principle) represent information about all of the preceding words.*

5.2.1 Inference

The model equations are (for some $\mathbf{h}_0$):

$$\begin{aligned}
 \mathbf{e}_t &= \mathbf{E}\mathbf{x}_t \\
 \mathbf{h}_t &= g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \\
 \hat{\mathbf{y}}_t &= \text{softmax}(\mathbf{V}\mathbf{h}_t)
 \end{aligned} \tag{5.5}$$

where (d_h is the size of the hidden vectors)

- $\mathbf{x}_t \in \mathbb{R}^{|V|}$: one-hot vector for word w_t

- $\mathbf{E} \in \mathbb{R}^{d_h \times |V|}$: embedding matrix
- $\mathbf{U}, \mathbf{W} \in \mathbb{R}^{d_h \times d_h}$: recurrent and input weight matrices
- $\mathbf{h}_t \in \mathbb{R}^{d_h}$: hidden state at time t
- $\mathbf{V} \in \mathbb{R}^{|V| \times d_h}$: output weight matrix
- $\hat{\mathbf{y}}_t \in \mathbb{R}^{|V|}$: predicted probability distribution over the vocabulary
- g : activation function (e.g., tanh or ReLU)

For each word $w \in V$, the element of $\hat{\mathbf{y}}_t$ with index $\text{ind}(w)$ estimates the probability that the next word is w :

$$\forall w \in V, \quad \hat{\mathbf{y}}_t[\text{ind}(w)] = P(w_t = w \mid w_{1:t-1}) \quad (5.6)$$

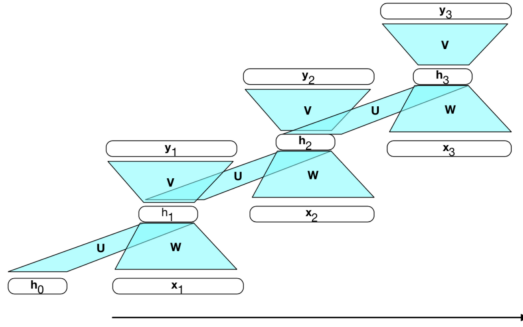


Figure 5.4: The recurrent NLM unrolled in time.

5.2.2 Training and Loss Function

The number of parameters of the model is $O(|V|)$, since d_h is a constant. We train the model to minimize the error in predicting the true next word w_t . The loss function used is again the **cross-entropy loss** function.

5.2.3 Teacher forcing

At each step t during training prediction is based on the correct sequence of tokens $w_{1:t}$. This means that we **ignore what the model predicted at previous steps**. The idea that we always give the model the correct history sequence to predict the next word is called **teacher forcing**.

Cannot recover from error Teacher forcing is done for convenience but it has a **disadvantage**: the model is never exposed to prediction mistakes; therefore at inference time the model is not able to recover from errors.

5.3 Practical Issues

Learn embeddings As already mentioned, the word embeddings $\mathbf{E}$ can be initialized using the pre-trained word embeddings given by Word2Vec. One can either freeze them or use fine-tuning.

Regularization with weight tying In the recurrent NLM the columns of $\mathbf{E}$ provide the learned word embeddings the rows of $\mathbf{V}$ provide a second set of learned word embeddings, that capture relevant aspects of word meaning and function.

We can use **weight tying**, also known as parameter sharing, to impose $\mathbf{V} = \mathbf{E}^T$.

Weight tying can significantly reduce model size, and has an effect similar to regularization, preventing overfitting of the NLM.

Hierarchical Softmax The last step in NLMs, involving softmax over the entire vocabulary, dominates the computation both at training and at test time. Vocabulary usually contains 10K to 100K words. An effective alternative is **hierarchical softmax**, based on word clustering.

Adaptive softmax is a simple variant of hierarchical softmax, based on Zipf’s law, especially tailored for GPUs. This provides giant speedup with only minimal costs in accuracy.

Generating text: balancing trade-off with temperature The text generated by sampling our NLM should be

- **Coherent**: text has to make sense.
- **Diverse**: the model has to be able to produce very different samples.

There is a **trade-off** between the two. A very popular method for balancing the trade-off is to change the **softmax temperature**:

$$\frac{\exp\left(\frac{\mathbf{V}_j \mathbf{h}_t}{\tau}\right)}{\sum_j \exp\left(\frac{\mathbf{V}_j \mathbf{h}_t}{\tau}\right)} \quad (5.7)$$

where $\mathbf{V}_j$ denotes the j -th row of $\mathbf{V}$. We have that:

- High $\tau \Rightarrow$ high entropy $\Rightarrow$ high diversity but low consistency.
- Low $\tau \Rightarrow$ low entropy $\Rightarrow$ low diversity but high consistency.

Contrastive evaluation Contrastive evaluation is used to test specific linguistic constructions in NLM. For example consider that we want to test the subject/object agreement of our trained model. We can evaluate it on sentences of increasing complexity.

The roses _____
The roses in the vase _____
The roses in the vase by the door _____

What's the probability $P(\text{is} \mid w_{1:t-1})$ compared to $P(\text{are} \mid w_{1:t-1})$? The former should be higher!

Chapter 6

Contextual Word Embeddings

6.1 Static vs Dynamic Embeddings

Static Word embeddings such as word2vec or GloVe learn a single vector for each type (unique word) in the vocabulary V . These are also called **static embeddings**. However, in natural language words have rich linguistic relationships with other words that may be far away.

Dynamic Contextual word embeddings represent each token (word occurrence) by a different vector, according to the context the token appears in. Incorporating context into word embeddings has proven to be a watershed idea in NLP, opening the way to **transfer learning**.

How can we learn to represent words along with the context they occur in?

6.2 ELMo

ELMo (embeddings from language model) looks at the entire sentence before assigning each word its embedding.

Token processing: char-cnn+Highway Tokens are processed by a character-level convolutional neural network (CNN) producing word-level embeddings. This process is independent from the context and it is done for every token. It is used to capture word morphology and resolves OOV words.

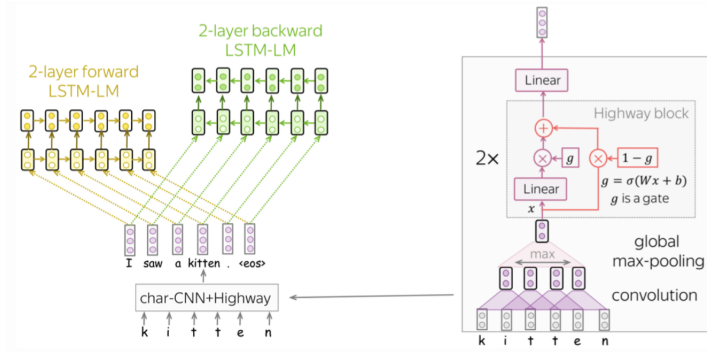


Figure 6.1: ELMo architecture (1).

Bi-directional LSTM The embeddings produced from previous step, are processed by a left-to-right and a right-to-left, 2-layers LSTM. This is also called a bi-directional LSTM.

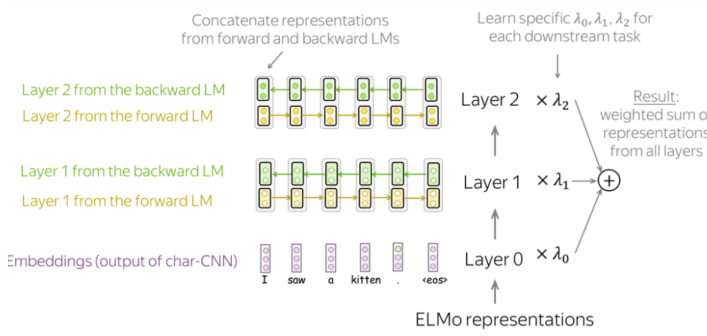


Figure 6.2: ELMo architecture (2): contextual representations.

ELMo contextual representations For each word, the output embeddings at each layer are combined, producing contextual embeddings. The layers are:

- **Layer 0:** output of the character-level CNN (h_w^0).
- **Layer 1:** concatenated representations from layer 1 of both forward and backward LSTMs (h_w^1).
- **Layer 2:** concatenated representations from layer 2 of both forward and backward LSTMs (h_w^2).

The context embedding of each word w is computed as:

$$\text{ELMo}(w) = \lambda_0 h_w^0 + \lambda_1 h_w^1 + \lambda_2 h_w^2. \quad (6.1)$$

6.2.1 Training

Similarly to what we've seen for word2vec, we train the model considering a prediction task. We add a linear transformation to $|V|$ dimension and we apply softmax to the probability distribution of next token. Again we use the cross-entropy as loss function.

The **training objective is bidirectional** causal language modeling: forward LSTM is trained to predict next token; backward LSTM is used to predict previous token.

After training the model, we can throw away the prediction part and we retain the word embeddings represented by the last hidden layer.

6.3 BERT

BERT (bidirectional encoder representations from transformers) produces word representations by jointly conditioning on both left and right context.

What are bidirectional encoders? Causal models of previous chapters are generative models, designed to easily generate the next token in a sequence. But the focus of bidirectional encoders is instead on **computing contextualized representations of the input tokens**. These output embeddings are contextualized representations of each input token that are **useful across a range of applications** where we need to do a classification or a decision based on the token in context.

Encoder-only The masked language models of this chapter are sometimes called **encoder-only**, because they produce an encoding for each input token but generally aren't used to produce running text by decoding/sampling.

Non-generative That's an important point: masked language models are not used for generation. They are generally instead used for interpretative tasks.

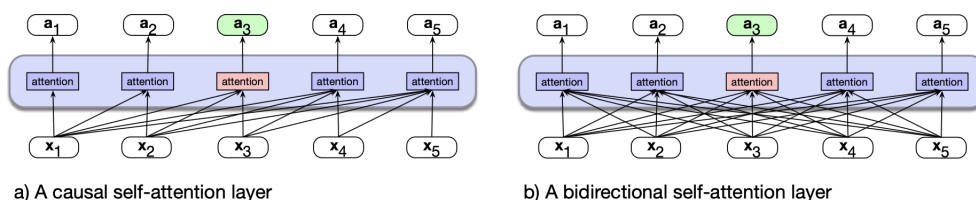


Figure 6.3: (a) The causal transformer architecture. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one.

6.3.1 Masked language modeling

The model learns to perform a fill-in-the-blank task, technically called the **cloze task**. The approach is called masked language modeling (MLM).

MLM approach As with the language model training methods we’ve already seen, MLM uses unannotated text from a large corpus. In MLM training, the model is presented with a series of sentences from the training corpus in which a percentage of tokens (15% in the BERT model) have been randomly chosen to be manipulated by the masking procedure.

Masking procedure Given an input sentence `lunch was delicious` and assume we randomly chose the 3rd token `delicious` to be manipulated,

1. 80% of the time: The token is replaced with the special vocabulary token named `[MASK]`;
2. 10% of the time: The token is replaced with another token, randomly sampled from the vocabulary based on token unigram probabilities.
3. 10% of the time: the token is left unchanged.

We then train the model to guess the correct token for the manipulated tokens.

Why three possible manipulation? Adding the `[MASK]` token creates a mismatch between pretraining and downstream finetuning or inference, since when we employ the MLM model to perform a downstream task, we don’t use any `[MASK]` tokens. If we just replaced tokens with the `[MASK]`, the model might only predict tokens when it sees a `[MASK]`, but we want the model to try to always predict the input token.

Training We add a language modeling head that:

- Takes the output vector $\mathbf{h}_i^L$ for each of the masked tokens.
- Multiplies it by the unembedding layer $\mathbf{E}^\top$ to produce the logits $\mathbf{u}$; in other words, the language modeling head is tied to the embedding matrix $\mathbf{E}$.
- Uses softmax to turn the logits into probabilities $\mathbf{y}$ over the vocabulary, to **predict the masked word**.

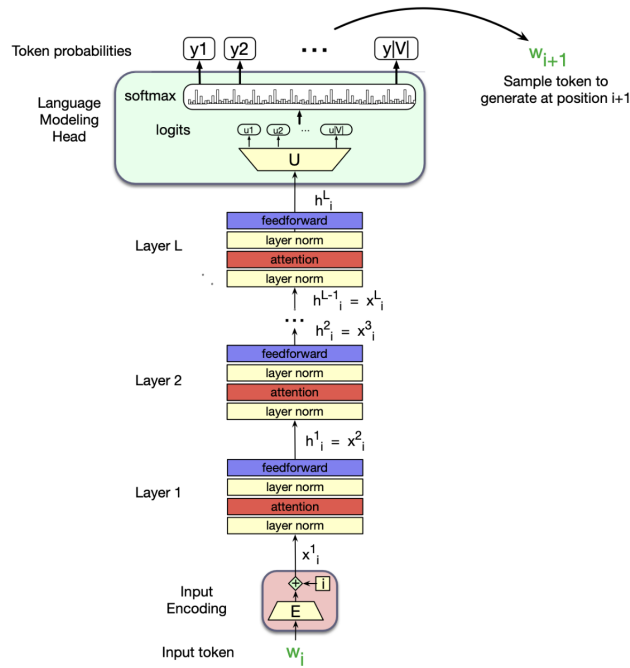


Figure 6.4: Full stack view of the language modeling head.

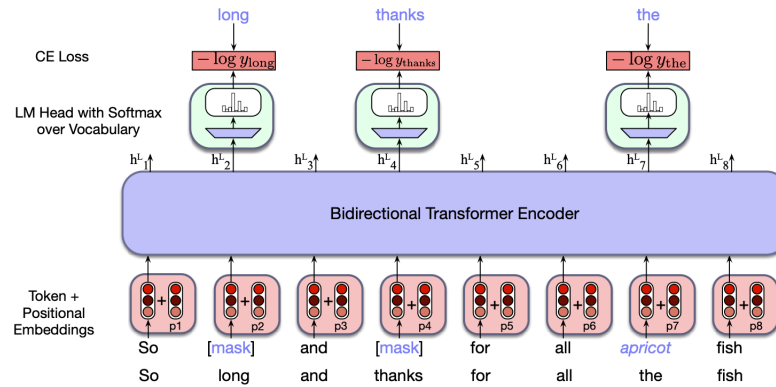


Figure 6.5: Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

With a predicted probability distribution for each masked item, we can use cross-entropy to compute the **loss** for each masked item. More formally: let x_i be an input token that has been masked. The loss at position i is the $-\log$ of the probability of the correct token x_i , given the hidden vector $\mathbf{h}_i^L$

$$L_{\text{MLM}}(x_i) = -\log P(x_i \mid \mathbf{h}_i^L). \quad (6.2)$$

Let also M be the set of token positions in a batch that have been masked. The loss function is the average loss over the sampled learning items

$$L_{\text{MLM}}(M) = -\frac{1}{|M|} \sum_{i \in M} \log P(x_i \mid \mathbf{h}_i^L) \quad (6.3)$$

Denoising Masked language modeling can be generalized to any method that corrupts the training input and asks to recover the original (e.g. masking of several adjacent words, reorderings of adjacent words, insertion of extraneous words).

6.3.2 Next sentence prediction

The focus of mask-based learning is on predicting words from surrounding contexts with the goal of producing effective word-level representations. However, an important class of applications involves determining the relationship between pairs of sentences:

- Paraphrase detection (detecting if two sentences have similar meanings).
- Entailment (detecting if the meanings of two sentences entail or contradict each other).
- Discourse coherence (deciding if two neighboring sentences form a coherent discourse).

Note that we cannot perform those tasks directly with language modeling.

A new objective for NSP In order to train a model that understands sentence relationships, BERT introduces a second learning objective called next sentence prediction (NSP). The model is asked to predict whether a pair of sentences can be adjacent or are totally unrelated.

Training NSP is a **binary decision task** that can be trained from pairs of sentences sampled from any monolingual corpus. Each pair consists of an actual pair of adjacent sentences from the training corpus (50%); a pair of unrelated sentences randomly selected (50%).

Then we need to introduce two special tokens:

1. Token [CLS] prepended to the first sentence.

2. Token [SEP] placed between the two sentences and after the rightmost token of the second sentence.

Finally we add a special NSP head to hidden vector $\mathbf{h}_{CLS}^L$, using learnable matrix $W_{NSP} \in \mathbb{R}^{d \times 2}$ for two-class prediction:

$$\mathbf{y}_i = \text{softmax}(W_{NSP}^T \mathbf{h}_{CLS}^L). \quad (6.4)$$

The vector $\mathbf{h}_{CLS}^L$ is the embedding of the sentence.

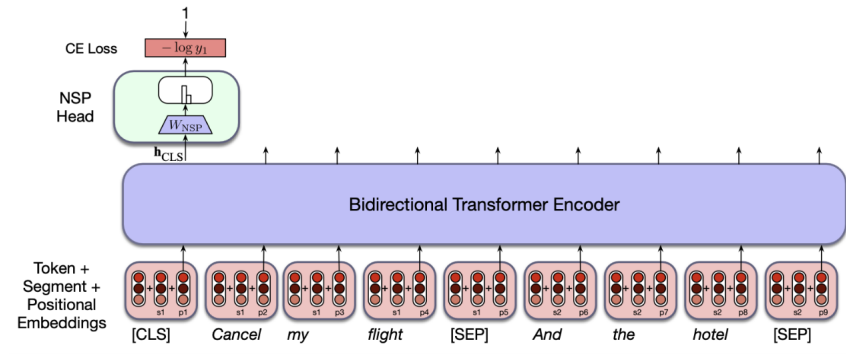


Figure 6.6: An example of the NSP loss calculation. Note that in NSP an additional **segment embedding** is added to the token embedding and the positional embedding, marking the first and second sentences.

6.3.3 Inference

After we've trained the model with the combined loss from the MLM and NSP objectives, we can use it to get the contextual word embeddings.

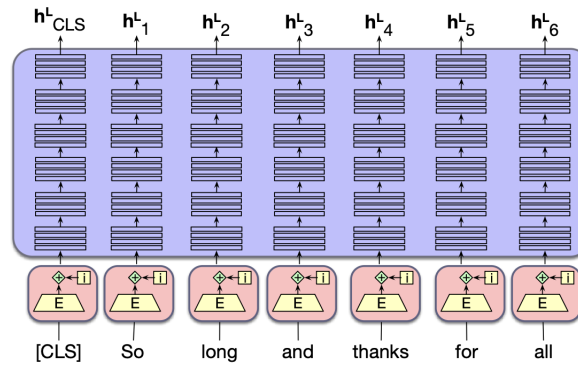


Figure 6.7: The output of a BERT-style model is a contextual embedding vector $\mathbf{h}_i^L$ for each input token x_i .

Based on the application we’re considering, we can either decide to use **word contextual embeddings**, or to use the **sentence embedding** h_{CLS}^L for tasks involving single sentence classification.

No separator token In general, at inference time, we do not use the [SEP] token, as we may need to process a single sentence only. This creates a mismatch between the pre-training and inference phases. However, the model still performs well (good embeddings are produced), as the two tasks remain closely related.

6.4 BERT Applications

The training process that we’ve described in previous section is called **pre-training**. After pre-training we can adapt BERT to solve a specific task. This step is called **post-training**: we use BERT as a backbone to solve specific tasks that make use of the contextual embeddings.

BERT was a big breakthrough because is able to learn general knowledge and, we will see how, we can slightly modify its architecture to perform many different tasks. In the past each application required to design an entirely new architecture, no general backbone was available.

6.4.1 Sentiment Analysis

Assume a three-way sentiment classification task: *positive*, *negative*, *neutral*. The goal of sentiment analysis is to classify each input sentence. This requires supervised training on data consisting of input sequences labeled with the appropriate sentiment class.

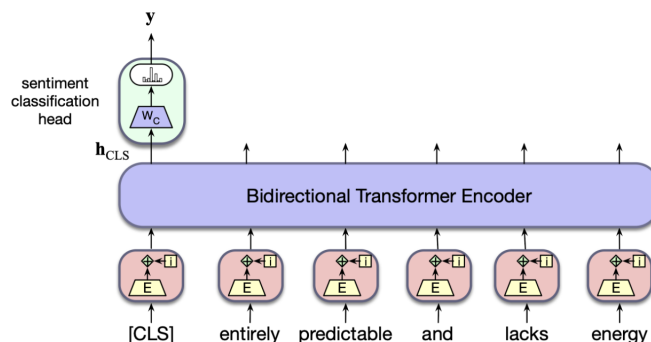


Figure 6.8: Sentiment analysis architecture.

Training Let $W_C \in \mathbb{R}^{3 \times d}$ and $\mathbf{b} \in \mathbb{R}^3$ be arrays of learnable parameters, where d is the size of BERT embeddings. We set a **classification head** for the [CLS] token and map $\mathbf{h}_{CLS}$ to three logits, and pass through the softmax:

$$\mathbf{y} = \text{softmax}(W_C \mathbf{h}_{CLS} + \mathbf{b}) \quad (6.5)$$

The parameters W_C and $\mathbf{b}$ are learned from scratch, the parameters of BERT (learned during pre-training) can be either frozen or we can fine-tune (some) of them.

6.4.2 Named Entity Recognition

A named entity is anything that can be referred to with a proper name: a person, a location, an organization, a geographical location, etc. The task of named entity recognition is to find spans of text that constitute named entities, and to tag them with the proper class.

[*PER* Jane Villanueva] of [*ORG* United], a unit of [*ORG* United Airlines Holding], said the fare applies to the [*LOC* Chicago] route.

How can we adapt BERT to solve this task? Since BERT cannot directly add tags to our text, we need a trick.

BIO Tagging BIO stands for *Beginning Inside Outside* and is a standard approach for span recognition. BIO tagging consists in three simple rules:

1. Token that begin a span are marked with label B.
2. Token that occur inside a span in a position other than the leftmost one, are marked with label I.
3. Token outside of any span of interest are marked with O.

Spans never overlap. Note the difference between the sequences BI and BB, indicating one 2-word expression vs. two 1-word expressions.

Training Let $W_K \in \mathbb{R}^{n \times d}$ and $\mathbf{b} \in \mathbb{R}^n$ be arrays of learnable parameters, where d is the size of BERT embeddings and n is the number of our classes. The number of classes depends on the entities that we want to recognize. Consider our previous example: we have the tags [PER, ORG, LOC] and we're using BIO Tagging trick, therefore we have

Table 6.1: Comparison of NER Labeling Schemes

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

$3 \times 3 = 9$ classes. We set a **classification head** that we apply to each input token to map each $\mathbf{h}_i^L$ to n logits, and pass through the softmax:

$$\mathbf{y} = \text{softmax}(W_K \mathbf{h}_i^L + \mathbf{b}) \quad (6.6)$$

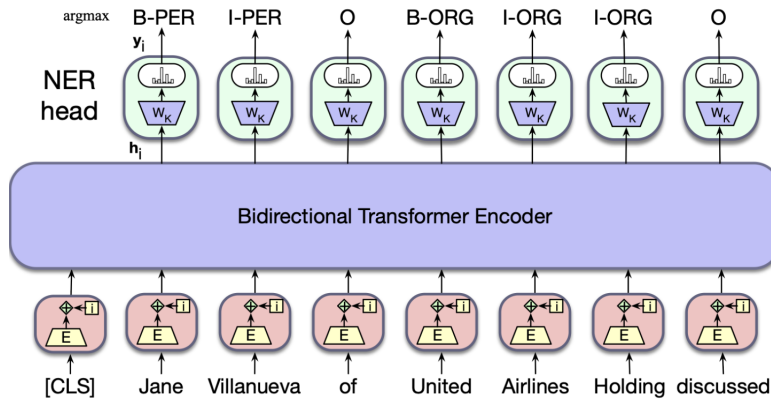


Figure 6.9: NER architecture.

6.5 SBERT

6.5.1 BERT Limitation

In some sentence-pair regression tasks, we need to derive **sentence embeddings** such that semantically similar sentences are **close in vector space**. We've seen that sentence embeddings can be computed through BERT by using the output of the [CLS] token.

However, the produced embeddings are **bad**. This is so because embeddings produced by BERT are not directly optimized for the above semantic task.

6.5.2 Sentence-BERT: Idea

BERT is an efficient and effective method to compute sentence embeddings on the basis of pre-trained BERT model. The basic idea underlying SBERT:

1. Encode sentences independently.
2. Compare embeddings via some objective function
3. Adjust BERT parameters, to a minimum extent, using **siamese neural network**¹.

6.5.3 Compute Embeddings from BERT

Sentence embeddings are obtained from BERT word embeddings through several **pooling strategies**:

- **Mean pooling**: the average of all output token vectors. This usually yields the best results for similarity tasks.
- **Max-over-time pooling**: taking the maximum value across all token vectors for each dimension, across all tokens in the sequence.
- **[CLS] token**: using the representation of the special classification token.

6.5.4 SBERT Training

SBERT is fine-tuned on labeled data (like SNLI or MultiNLI) using different **objective functions**, depending on the task. Note that parameter updating is **mirrored** (shared) across both sub-networks. This ensures vectors $\mathbf{u}$ and $\mathbf{v}$ (the embeddings) live in the same semantic space.

Classification objective Assume d is the embeddings dimension and k is the number of labels. The classification objective function concatenates vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ and the

¹A siamese neural network is a special network that contains two identical subnetworks sharing their parameters.

element-wise difference $|\mathbf{u} - \mathbf{v}|$, and uses a trainable matrix $\mathbf{W} \in \mathbb{R}^{3d \times k}$ to compute

$$\text{softmax} \left(\mathbf{W} \begin{bmatrix} \mathbf{u} \\ \mathbf{v} \\ |\mathbf{u} - \mathbf{v}| \end{bmatrix} \right).$$

Regression objective Regression objective function compares sentence embeddings $\mathbf{u}$ and $\mathbf{v}$ using a **cosine similarity**, which will output a semantic similarity score.

Triplet objective (contrastive learning) Assume an anchor sentence a , a positive sentence p , and a negative sentence n . For $x \in \{a, p, n\}$ let $\mathbf{s}_x$ be the sentence embedding for x . The triplet objective function minimizes the following loss function

$$\max (\|\mathbf{s}_a - \mathbf{s}_p\| - \|\mathbf{s}_a - \mathbf{s}_n\| + \epsilon, 0)$$

In words, we tune the network such that the distance between a and p is smaller than the distance between a and n , modulo some margin ϵ .

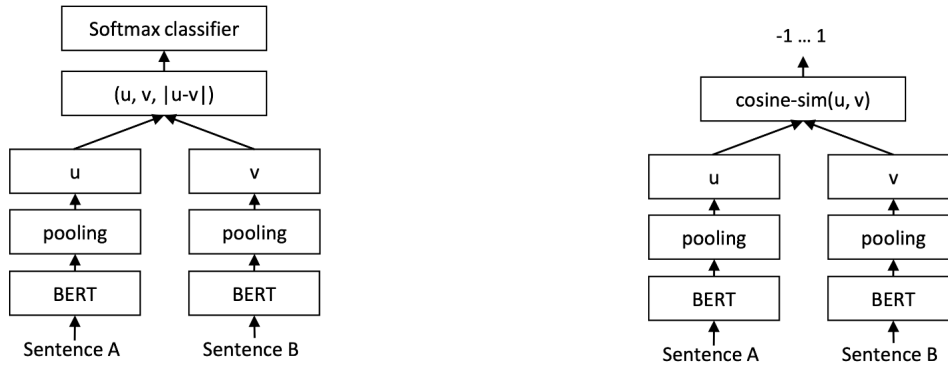


Figure 6.10: SBERT architecture.

6.6 SBERT Applications

The fine-tuned BERT can then be used to compute semantically meaningful sentence embeddings for several scenarios:

- **Semantic Textual Similarity (STS)**: comparing how similar two sentences are (e.g., for plagiarism detection).
- **Clustering**: grouping thousands of documents into topics in seconds.

- **Information Retrieval:** dense vector search where a query finds the most meaningful match, not just keyword matches.

SBERT maintains BERT's accuracy while being fast enough for real-time applications.

Chapter 7

LLM: Pre-Training

7.1 Introduction

Definition 23 (Pre-training). *In deep learning pretraining refers to the process of optimizing a neural network before it is further trained/tuned and applied to the tasks of interest.*

This approach is based on the general assumption that a model pre-trained on one task can be adapted to perform another task, called **transfer learning**. In this chapter we introduce the idea of pretraining in the context of LM: learning knowledge about language and the world from vast amounts of text, using a LM objective function. We call the result of this process **large language models**, or LLMs for short. We will see that almost any NLP task can be modeled as **next word prediction** in a LLM.

7.1.1 Structure of an LLM

The decoder component of a transformer can be turned into a language model, using an additional learnable linear layer called **language modeling head**.

The language modeling head:

1. Takes the output of the topmost transformer layer for token N .
2. Uses it to predict the upcoming word at position $N - 1$.

Most commonly, the language modeling head is tied¹ to the embedding matrix, that is, $\mathbf{U} = \mathbf{E}^T$.

¹Recall that in weight tying, we use the same weights for two different matrices in the model as a form of regularization.

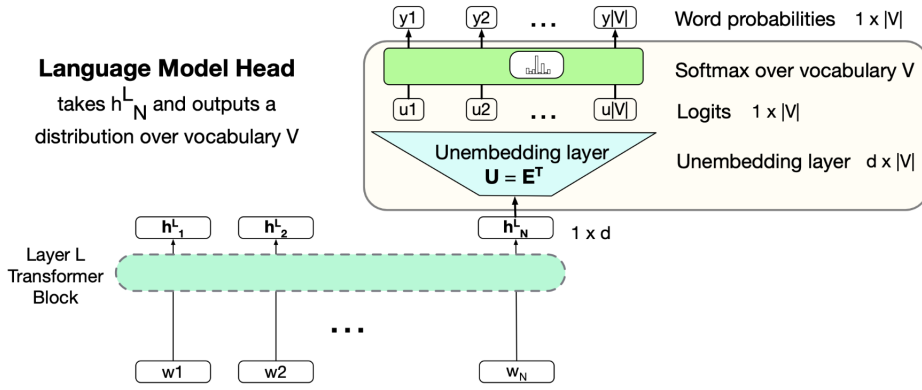


Figure 7.1: Decoder component of the transformer combined with language modeling head.

Then a softmax layer turns the logits $\mathbf{u}$ into the probabilities $\mathbf{y}$ over the vocabulary:

$$\begin{aligned} \mathbf{u} &= \mathbf{E}^T \mathbf{h}_N^L \\ \mathbf{y} &= \text{softmax}(\mathbf{u}) \end{aligned} \quad (7.1)$$

Finally we sample from the resulting distribution $\mathbf{y}$ to generate the next token.

7.1.2 Modeling tasks as text completion

As we've seen for BERT in the previous chapter, also LLMs are (pre-)trained to learn general knowledge. Then LLMs can be fine-tuned for a specific task. This means that we don't have to create a new architecture for every task we want to solve, we just need to adapt it the pre-trained model.

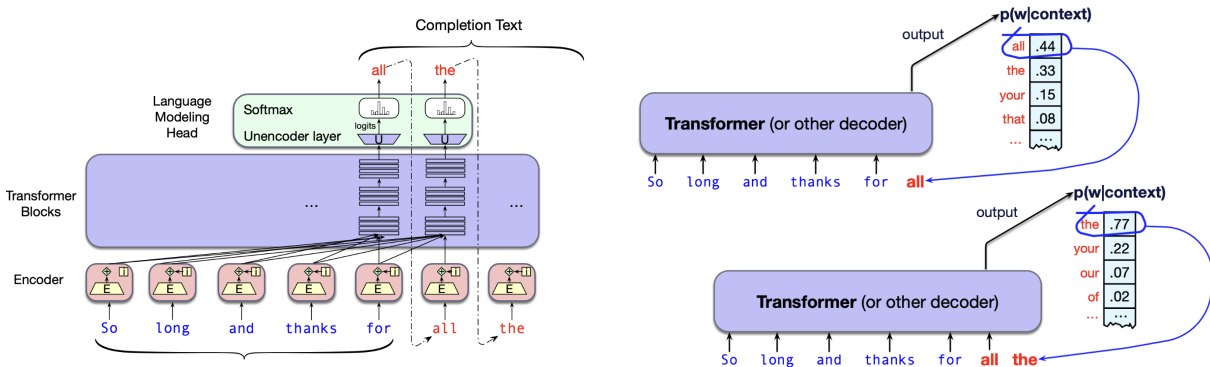


Figure 7.2: Text completion (also called conditional generation) is the task of generating text conditioned on an input piece of text.

Sentiment analysis We can cast sentiment analysis as word prediction by giving a language model a context and by comparing the following conditional probability of the word `positive` and the word `negative`.

```
P(positive | The sentiment of the sentence I like Jackie Chan is:)
P(negative | The sentiment of the sentence I like Jackie Chan is:)
```

Question answering We can cast the task of question answering as text completion by giving a language model a context like:

```
Q: Who wrote the book The Origin of Species?  A:
```

and expect that the most likely completion is:

```
Q: Who wrote the book The Origin of Species?  A: Charles Darwin
```

Text summarization We can cast text summarization as text completion by giving a large language model a text, followed by the token `tl;dr;` which is short for *too long; didn't read*. This token is **sufficiently frequent in training data** before a summary, and hence the LLM will interpret the token as instructions to generate a summary.

Machine translation For machine translation we can use the context

```
English:  I missed the bus today.  French:
```

and the LLM *may* be able to fill in the blank with a French translation.

7.1.3 Few-shot and Zero-shot learning

Definition 24. *Few-shot learning is the ability of an LLM to solve a problem on the basis of a few examples.*

For example:

```
sea otter -> loutre de mer
peppermint -> menthe poivr'ee
cheese -> ??
```

Definition 25. *Zero-shot learning is the ability of an LLM to solve a new problem solely on the basis of natural language instructions.*

For example:

```
translate English to French
cheese -> ??
```

7.2 Sampling

Which tokens should a language model generate at each step? Given the probability distribution over tokens, we need to select one token to generate. The task of choosing a token to generate based on the model’s probabilities is often called **decoding**. As we mentioned above, decoding from a language model in a left-to-right manner, and thus repeatedly choosing the next token conditioned on our previous choices is called **causal** or **autoregressive generation**.

Trade-off We need sampling methods that enable trading off two important factors in generation.

- **Coherence**: methods that emphasize the most probable words produce accurate and coherent text, but also **more repetitive**.
- **Diversity**: methods that slightly increase the weight of middle-probability words produce more creative and diverse text, but less factual or incoherent.

7.2.1 Greedy decoding

One simple strategy is greedy decoding: we always generate the most likely word given the context

$$\hat{w}_t = \arg \max_{w \in V} P(w \mid \mathbf{w}_{<t}). \quad (7.2)$$

This approach has **two limitations**:

1. It makes a choice that is **locally optimal**. However this may not result in the best solution overall.
2. The **system is deterministic**: identical input context $\Rightarrow$ it generates exactly the same text.

Greedy decoding provides high coherence but low diversity.

7.2.2 Random sampling

We iteratively choose the next word according to its probability given the context. This is the most decoding method for LLM. We sample new tokens until the model outputs end-of-string token (`<EOS>`).

This approach has a **major limitation**: by summing up the low-probabilities associated to many odd words in the tail of the distribution, we get an overall high value. This means that we have a non-neglectable chance of choosing a *non-sense* next token. Random sampling provides high diversity but low coherence.

7.2.3 Top-k and Top-p sampling

Top-k sampling Top-k sampling is a simple generalization of greedy decoding:

1. Truncate the word distribution to the top k most likely words.
2. Re-normalize to produce a proper probability distribution p_k .
3. Randomly sample from within these k words according to p_k

When $k = 1$, top-k sampling is identical to greedy decoding; when $k > 1$, top-k generates more diverse but still high coherence text.

Top-p sampling Sometimes the top k words represent only a small part of the probability mass. An alternative technique called top-p sampling or nucleus sampling keeps the top p percent of the probability mass:

1. Compute the top-p vocabulary $V^{(p)}$, defined as the smallest set of words such that

$$\sum_{w \in V^{(p)}} P(w \mid \mathbf{w}_{<t}) \geq p. \quad (7.3)$$

2. Truncate P to $V^{(p)}$, re-normalize and sample accordingly.

7.2.4 Temperature sampling

In temperature sampling we don't truncate the distribution, but instead reshape it by means of a **temperature parameter** $\tau > 0$:

$$\frac{\exp(u_i/\tau)}{\sum_j \exp(y_j/\tau)} \quad (7.4)$$

The temperature τ can be selected to balance the coherence/diversity trade-off:

- Low $\tau \Rightarrow$ produces peaky distribution $\Rightarrow$ high coherence. Usually $\tau \in (0, 1]$.
- Large $\tau \Rightarrow$ flat distribution $\Rightarrow$ high diversity. Usually $\tau > 1$.

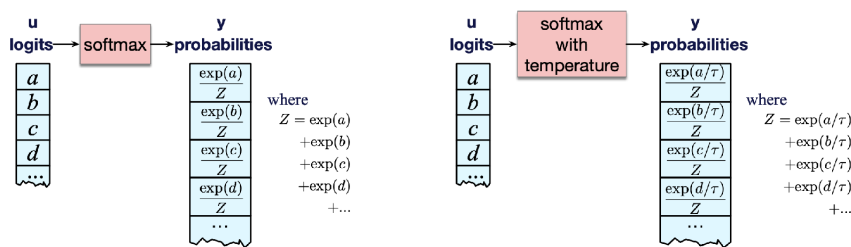


Figure 7.3: Computing distribution without considering temperature and by considering temperature.

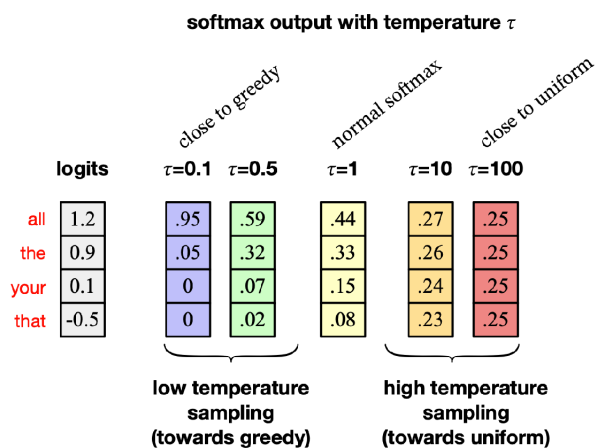


Figure 7.4: Temperature sampling.

7.3 Pre-training

The process of training a transformer as a language model involves vast amounts of text and compute, and only few large corporations have the necessary resources for this. As the result of pretraining, the language model acquires: **language knowledge** and **general knowledge about real world**. The two skills are both obtained by training a model to predict the next token!

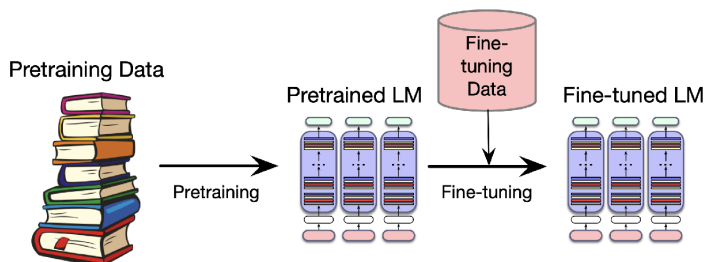


Figure 7.5: Pre-training and fine-tuning. Fine-tuning is applied after pre-training and it requires specialized data.

Loss function To train a transformer as a language model, we use the algorithm we discussed for LM, called **self-supervision** (or self-training)². We train on a corpus of text, and at each time step t we ask the model to predict the next word. For a fixed t , the cross-entropy (CE) loss L_{CE} (see Subsection 5.2.2) measures the difference between the predicted distribution and the true distribution. The loss for a training sequence is the average CE loss over the entire sequence.

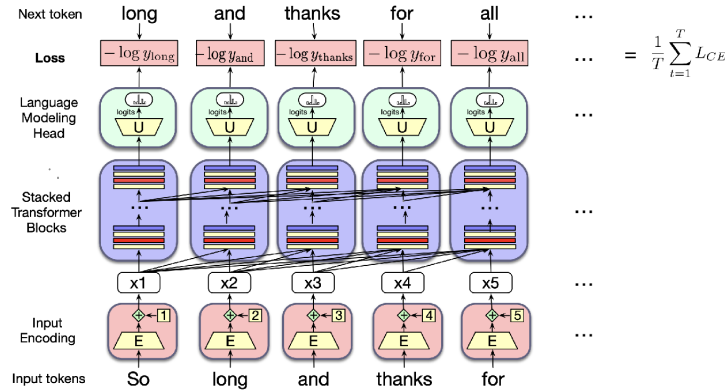


Figure 7.6: The loss for a training sequence is the average cross-entropy loss over the entire sequence. With transformers, each training item can be processed in parallel since the output for each element in the sequence is computed separately. Large models are trained by filling the full context window (4096 tokens for GPT4, 8192 for Llama 3) with text. If documents are shorter, multiple documents are packed into the window with a special end-of-text token between them. The batch size is usually quite large (the largest GPT-3 model uses 3.2 million tokens).

Teacher forcing (recall) At the next step $t+1$, we ignore what the model predicted at step t and instead use the correct sequence of tokens $w_{1:t+1}$ this is called teacher forcing.

7.3.1 Training corpora

Large language models are mainly trained on text scraped from the web and filtered.

- **C4 (Colossal Clean Crawled Corpus)**: 156 billion tokens of English; provided by Common Crawl, a public organization that maintains a series of snapshots of the entire web.
- **FineWeb**: 15-trillion tokens, provided by Hugging Face; sourced from 96 Common Crawl snapshots, widely considered one of the most high-quality open-source pretraining corpora.

²We call such a model self-supervised because we don't have to add any special gold labels to the data: the natural sequence of words is its own supervision!

- **The Pile**: 825 GB English text corpus, including books and Wikipedia.
- **Dolma**: open corpus of English, 3 trillion tokens, consists of web text, academic papers, code, books, encyclopedic materials, and social media.

Pre-trained data are filtered for two main reason:

1. **Quality**: cleans text, removes duplicate documents personal identifiable information, boilerplate text, etc. In general this improves LLM performance.
2. **Safety**: toxicity detection, removing adult content, etc.

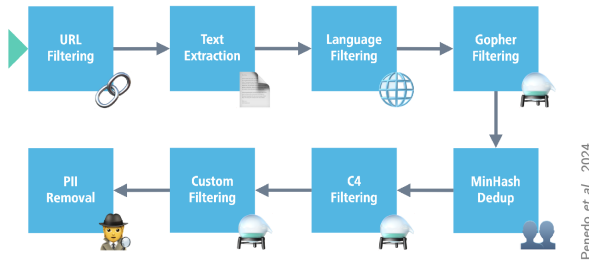


Figure 7.7: FineWeb filtering pipeline.

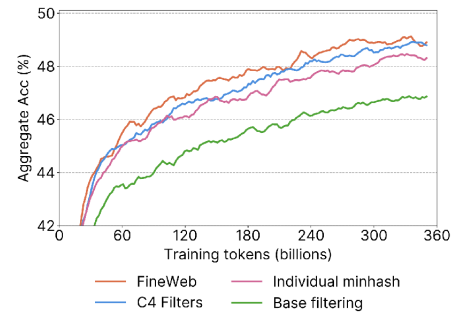


Figure 7.8: Impact of filtering on LLM accuracy.

7.4 Key-Value Cache

Transformer’s attention vectors can be efficiently computed in parallel at training time, via two matrix multiplications:

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V. \quad (7.5)$$

Unfortunately we can’t do quite the same efficient computation in inference as in training. That’s because at inference time, we **iteratively generate the next tokens one at a time**. For a new token that we have just generated, call it x_i , we need to compute its **query**, **key**, and **values** by multiplying by W^Q , W^K , and W^V respectively. But it would be a waste of computation time to recompute the key and value vectors for all the prior tokens $x_{<i}$; at prior steps we already computed these key and value vectors! So instead of recomputing these, whenever we compute the key and value vectors we store them in memory in the **KV cache**, and then we can just grab them from the cache when we need them.

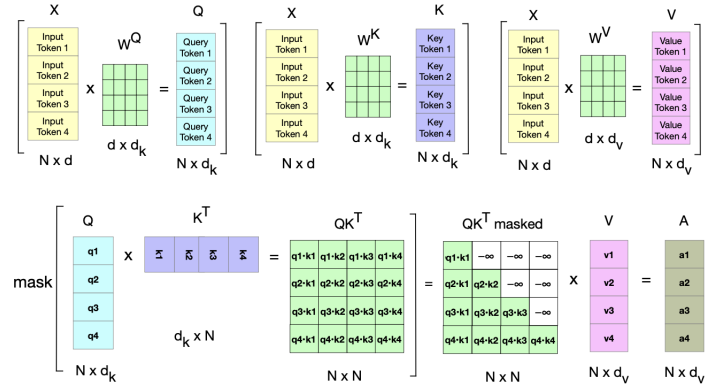


Figure 7.9: (Training) Schematic of the attention computation for a single attention head in parallel. The first row shows the computation of the Q, K , and V matrices. The second row shows the computation of QK^T , the masking (the softmax computation and the normalizing by dimensionality are not shown) and then the weighted sum of the value vectors to get the final attention vectors.

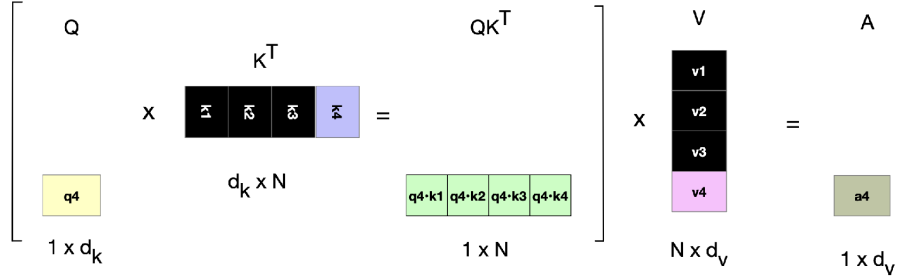


Figure 7.10: (Inference) Attention computation at the 4-th token. Black vectors need not be recomputed.

7.5 Scaling Laws for LLMs

The loss L of a LLM scales as a power-law with each of the three factors below, if the other two are not bottlenecking:

1. **N**: the number of parameters of the neural model (excluding embeddings).
2. **D**: the amount of data available for the training process.
3. **C**: the computing power used to train the neural model.

That is, we can improve a model by adding parameters (adding more layers or having wider contexts or both), by training on more data, or by training for more iterations.

The relationships between these factors and performance are known as **scaling laws**. For optimal performance all three factors must be scaled up in tandem.

Computing N and C The number of (non-embedding) parameters N can be roughly computed as follows:

$$N \approx 12 \cdot n_{\text{layers}} \cdot d^2 \quad (7.6)$$

where d is the input and output dimensionality of the model³.

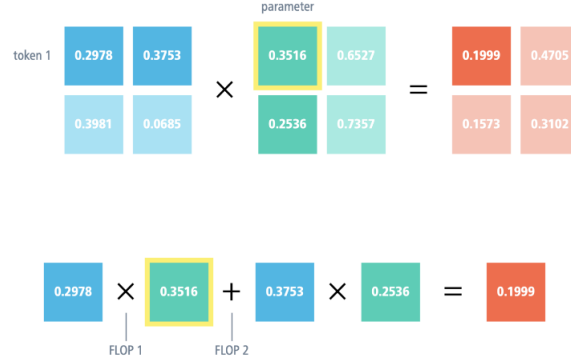


Figure 7.11: Each parameter is involved in 2 FLOPs (F_{fwd}) during forward pass.

For a single token and a single parameter, the number F of FLOPs (floating point operations) at training time is (approximately):

$$F \approx F_{fwd} + F_{bwd} = 2 + 4 = 6. \quad (7.7)$$

Therefore the total cost C can be related to N and D :

$$C \approx F \cdot N \cdot D = 6ND. \quad (7.8)$$

The standard unit for measuring computational cost C is PetaFLOP-days, that is, the number of floating point operations per day.

Scaling laws Kaplan et al. (2020) found the following three relationships for loss L as a function of N, D, C for models training with limited parameters N , dataset D or

³Thus GPT-3, with $n = 96$ layers and dimensionality $d = 12288$, has $12 \times 96 \times 12288 \approx 175$ billion parameters.

computational resources C , if in each case the other two properties are held constant:

$$\begin{aligned} L(N) &= \left(\frac{N_c}{N}\right)^{\alpha_N} \\ L(D) &= \left(\frac{D_c}{D}\right)^{\alpha_D} \\ L(C) &= \left(\frac{C_c}{C}\right)^{\alpha_C} \end{aligned} \tag{7.9}$$

The values of $N_c, D_c, C_c, \alpha_N, \alpha_D$, and α_C depend on the exact transformer architecture, tokenization, and vocabulary size.

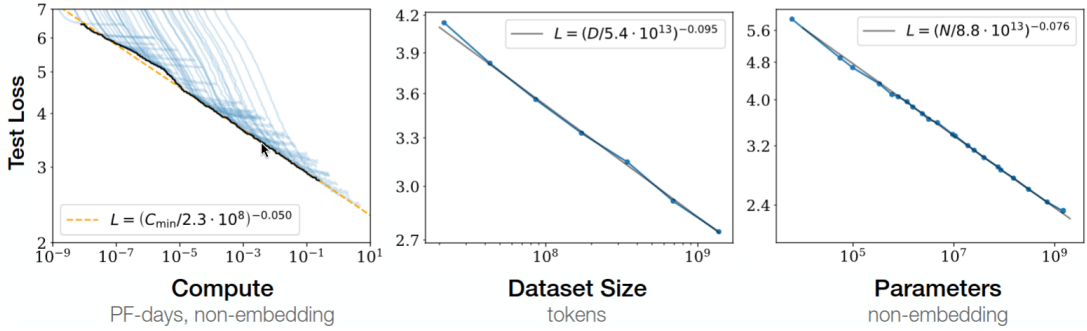


Figure 7.12: Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute used for training. (Plotted in log-scale).

The Loss of a model while varying N and D simultaneously is:

$$L(N, D) \sim \left[\left(\frac{N_c}{N}\right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \tag{7.10}$$

This is used to derive, for a given N , the minimum size of D that is needed to avoid overfitting.

When do we use scaling laws? Scaling laws can be useful in deciding **how to train a model to a particular performance**. For example by looking at early in the training curve, or performance with smaller amounts of data, to predict what the loss would be if we were to add more data or increase model size. Other aspects of scaling laws can also tell us how much data we need to add when scaling up a model.

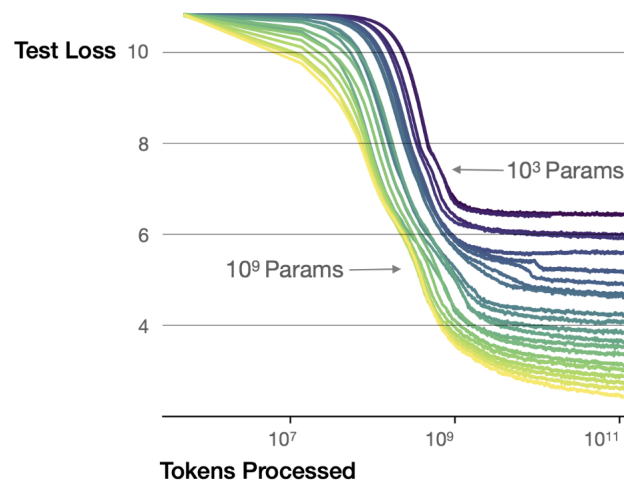


Figure 7.13: Larger models require fewer samples to reach the same performance.

7.6 Multi-Lingual LLMs

Multilingual large language models (MLLMs) are typically based on the **encoder part** of the transformer architecture. A MLLM is pre-trained using large amounts of unlabeled data from multiple languages, ranging from 10 to 100. They are very useful in **transfer learning** where low resource languages can benefit from high resource languages.

7.6.1 Pre-training

Data During pretraining, MLLMs use two different sources of data

1. **Large monolingual corpora** in individual languages.
2. **Parallel corpora** between some languages.

We can use one of the two options or we can use both of them. With *parallel* we mean that the corpora are aligned: they're written in different languages but they share the same meaning (e.g. an article written in two different languages).

Objective functions (monolingual) Objective functions can be based on monolingual data alone by pooling together monolingual corpora from multiple languages and applying masked language modeling (MLM).

Surprisingly, the encoder **learns word representations across languages that are aligned** (close vectors), without the need for any parallel corpora.

Objective functions (parallel corpora) Objective functions based on parallel corpora explicitly **force representations** of corresponding words across languages **to be close** to each other in the multilingual encoder space.

Using MLM to predict a masked word [MASK] for language A the system can:

1. Rely on words for language A surrounding [MASK].
2. Rely on an **aligned word** that has not been masked, that is, a word representing a translation in language B of the masked word.

Since parallel corpora are generally much smaller than monolingual corpora, the parallel objectives are often used in conjunction with monolingual models.

Curse of multilinguality Similar to models that are trained on many tasks, **the more languages** a model is pre-trained on, **the less model capacity is available to learn representations** for each language.

Chapter 8

LLM: Post-Training

Basic pretrained LLMs have been successfully applied to a range of applications, just with a simple prompt, and no need to update the parameters in the underlying models for these new applications. Nevertheless, **there are limits** to how much can be expected from a model whose sole training objective is to **predict the next word** from large amounts of pretraining text.

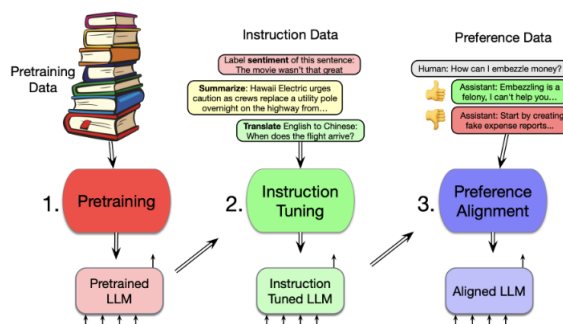


Figure 8.1: Three stages of training large language models: pretraining, instruction tuning, and preference alignment.

8.1 Introducing Post-Training

To make practical use of a pretrained LLM, we need to **interface** the model with some downstream task/applications **adapt** the model to a specific domain of interest. This process is called **post-training** or **adaptation**. Post-training typically uses data with stronger signal and with higher quality than that of pretraining data, also it uses much less computational resources and does not target all the parameters of the LLM.

Techniques Post-training groups together several techniques

- **Fine-tuning / continued pretraining**
- **Supervised fine-tuning**
- **Instruction tuning**
- **Alignment**

In-context learning is also considered a form of post-training, but it does not change the model parameters.

8.2 Fine-Tuning

Fine-Tuning is used to specialize an LLM for a new task. The standard approach is:

1. **New Head:** add a head layer for the task at hand, at the top of the transformer, with learnable parameters¹.
2. **Minimal adjustment:** make minimal adjustments to the pretrained parameters of the LLM.

Retraining the whole LLM (all parameters) results in so-called **catastrophic forgetting**.

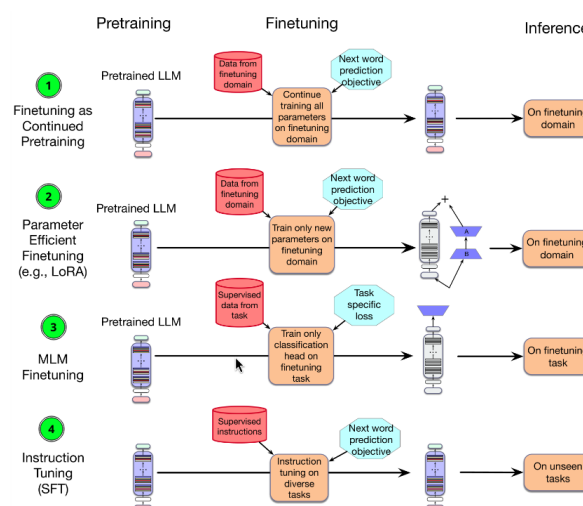


Figure 8.2: Several typologies of fine-tuning.

¹See how we've adapted BERT to sentiment analysis in Subsection 6.4.1

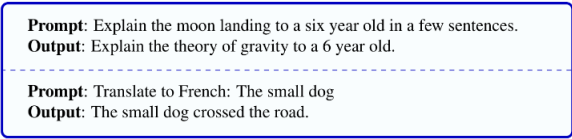
8.2.1 Supervised Fine-Tuning

Definition 26 (SFT). *Supervised fine-tuning, or SFT for short, refers to the process of fine-tuning a pretrained model on a labeled dataset for a specific task, using a supervised learning objective function.*

The goal is simply to make the model better at predicting the correct output for a given input. Sentiment analysis is an example of supervised fine-tuning².

8.3 Instruction Tuning

Instruction tuning is needed because LLMs have not been designed to answer questions or to follow instructions, even in presence of strong signals such as prompt. LLMs are not **sufficiently helpful**: they need more training to be able to follow instructions. Instruction tuning is needed if you want your LLM to learn when to generate the <EOS> token.



Prompt: Explain the moon landing to a six year old in a few sentences.
Output: Explain the theory of gravity to a 6 year old.

Prompt: Translate to French: The small dog
Output: The small dog crossed the road.

Figure 8.3: Here, the LLM ignores the intent of the request and relies instead on its natural inclination to autoregressively generate continuations consistent with its context. The problem is that the objective function (predict the next word) is **misaligned** with the human need for models to be helpful.

The **goal** of instruction tuning is to **make an LLM better at following general instructions**. Instruction tuning is what turned GPT-3 into ChatGPT.

8.3.1 Instructions as Training Data

The training data consists of:

- Instructions and answers for a wide range of tasks, and/or
- question/answer pairs in a specific domain of interest (optional).

If you just want a chatbot, then only first bullet is needed.

²While the terms fine-tuning and supervised fine-tuning are often used interchangeably in casual conversation, they represent different levels of specificity in the training pipeline.

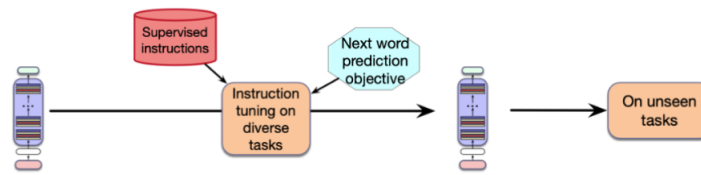


Figure 8.4: Instruction tuning can be viewed as a second pass of LLM training, and is sometimes referred to as **continued pretraining**.

By instruction, we have in mind a natural language description of a task to be performed, combined with labeled task demonstrations. This can include minimal descriptions such as

Answer the following question
 Translate the following text to Arapaho
 Summarize this report

We continue training the model with the **guess-the-next-token objective**, limited to the answer part of each example

Is instruction tuning SFT? Unlike in pretraining, each instruction or question in the training data for instruction tuning has a **supervised objective**: a correct answer, or a response to the instruction. For this reason, instruction tuning is considered **a form of supervised learning**, and is viewed as a special case of SFT.

8.3.2 Datasets for Instruction Tuning

Instruction-tuning datasets are created in three ways.

Manual creation The dataset is created manually, through crowdworkers, based on carefully written annotation guidelines. **Problem:** developing high quality supervised training data in this way is time consuming and costly.

Adapting existing datasets A dataset can be created by adapting existing ones previously created for a wide range of natural language tasks (e.g. question answering, translation, summarization, etc.). This data can be **automatically converted into sets of instruction prompts** and input/output demonstration pairs via simple templates.

Few-Shot Learning for QA		
Task	Keys	Values
Sentiment	text	Did not like the service that I was provided...
	label	0
NLI	text	It sounds like a great plot, the actors are first grade, and...
	label	1
	premise	No weapons of mass destruction found in Iraq yet.
	hypothesis	Weapons of mass destruction found in Iraq.
Extractive Q/A	premise	Jimmy Smith... played college football at University of Colorado.
	hypothesis	The University of Colorado has a college football team.
	label	0
	context	Beyoncé Giselle Knowles-Carter is an American singer...
question	text	When did Beyoncé start becoming popular?
	answers	{ text: ['in the late 1990s'], answer.start: 269 }

Figure 8.5: Examples of supervised training data for sentiment, natural language inference and Q/A tasks. The various components of the dataset are used in generating instructions.

Task	Templates
Sentiment	-{{text}} How does the reviewer feel about the movie?
	-The following movie review expresses what sentiment? {{text}}
Extractive Q/A	-{{text}} Did the reviewer enjoy the movie?
	-{{context}} From the passage, {{question}}
	-Answer the question given the context. Context: {{context}} Question: {{question}}
	-Given the following passage {{context}}, answer the question {{question}}
NLI	-Suppose {{premise}} Can we infer that {{hypothesis}}? Yes, no, or maybe?
	-{{premise}} Based on the previous passage, is it true that {{hypothesis}}? Yes, no, or maybe?
	-Given {{premise}} Should we assume that {{hypothesis}} is true? Yes, no, or maybe?

Figure 8.6: Instruction templates for sentiment, Q/A and NLI tasks.

Using powerfull ChatBots Because supervised NLP datasets are often created by crowdworkers following carefully designed annotation guidelines, a third approach is to leverage these guidelines directly. Such guidelines typically include detailed step-by-step instructions, common pitfalls to avoid, formatting rules, length constraints, and illustrative examples. They can be repurposed as prompts for a language model to generate instruction-tuning training data.

Sample Extended Instruction
• Definition: This task involves creating answers to complex questions, from a given passage. Answering these questions, typically involve understanding multiple sentences. Make sure that your answer has the same type as the "answer type" mentioned in input. The provided "answer type" can be of any of the following types: "span", "date", "number". A "span" answer is a continuous phrase taken directly from the passage or question. You can directly copy-paste the text from the passage or the question for span type answers. If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words. A "number" type answer can include a digit specifying an actual value. For "date" type answers, use DD MM YYYY format e.g. 11 Jan 1992. If full date is not available in the passage you can write partial date such as 1992 or Jan 1992. • Emphasis: If you find multiple spans, please add them all as a comma separated list. Please restrict each span to five words. • Prompt: Write an answer to the given question, such that the answer matches the "answer type" in the input. Passage: { passage } Question: { question }

Figure 8.7: Example of a human crowdworker instruction for an extractive question answering task, used as a prompt for a language model to create instruction finetuning examples. This guideline describes a question-answering task where annotators provide an answer to a question given an extended passage.

8.3.3 Evaluation of Instruction-Tuned LLMs

As mentioned, the **goal** of instruction tuning **is not** to learn a single task, but rather to **learn to follow instructions in general**. Therefore, in assessing instruction-tuning methods we need to assess how well an instruction-trained model performs **on novel tasks** for which it has not been given explicit instructions³.

Leave-one-out The standard way to perform such an evaluation is to take a **leave-one-out approach**, that is: instruction-tune a model on some large set of tasks and then assess it on a withheld task.

Limitation Popular datasets for instruction tuning have an enormous numbers of tasks: these tasks very often overlap! To overcome this limitation, one needs to **cluster tasks** according to types, and **test on unknown cluster types**.

8.4 Model Alignment

Instruction tuning results in LLMs that generate coherent and contextually appropriate responses. However, some responses may contain content that is **harmful**, **unethical**, **socially biased**, and **negative**, or even **illegal**.

Definition 27. *Model alignment is a post-training process ensuring that LLMs meet human expectations, ethical standards, and personalized needs.*

8.4.1 Preference-Based Learning

The most popular approach for model alignment of LLM's is Reinforcement Learning. The approach is called **preference-based learning** or, alternatively, **reinforcement learning from human feedback** (RLHF). Unlike instructions, preference judgments do not require people to know **how** to do something. We simply have to have an opinion about the end result.

Preference-Based Learning involves two steps:

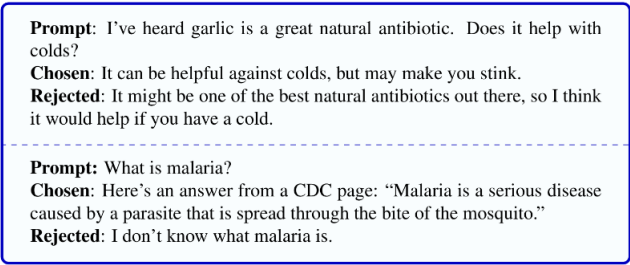
1. **Fitting a reward function** to a dataset of human preferences.
2. **Use the reward function to train an LLM** to generate responses with high reward.

³Instruction Tuning datasets contains many different tasks.

8.4.2 LLM Preference Data

Preference-based learning uses high-quality datasets consisting in questions with two contrastive answers:

1. A **preferred answer**, meeting the desired standard.
2. A **dispreferred answer**, whose style should be discouraged.



Prompt: I've heard garlic is a great natural antibiotic. Does it help with colds?
Chosen: It can be helpful against colds, but may make you stink.
Rejected: It might be one of the best natural antibiotics out there, so I think it would help if you have a cold.

Prompt: What is malaria?
Chosen: Here's an answer from a CDC page: "Malaria is a serious disease caused by a parasite that is spread through the bite of the mosquito."
Rejected: I don't know what malaria is.

Figure 8.8: Prompts and preferences pairs.

Preference data generally come from three sources:

- **Preference judgments** from trained human annotator (e.g., annotators rate outputs on a Likert scale (0-4) along various dimensions, then take the average value).
- **Implicit preference judgments** extracted from online resources (e.g., user votes on social media, etc.).
- **Fully synthetic preference judgments** from LLMs as annotators (e.g., prompting outputs from diverse LLMs and then prompting GPT-5 to rank the outputs).

8.4.3 Modeling Preferences

We will use the following notation:

$$\begin{aligned}
 x &:= \text{a prompt} \\
 o &:= \text{an output sampled from an LLMM giving } x \text{ as input} \\
 \mathbb{R} \ni z &:= \text{the score associated with output } o \\
 o_i \succ o_j \mid x &:= \text{output } o_i \text{ is preferred to } o_j
 \end{aligned} \tag{8.1}$$

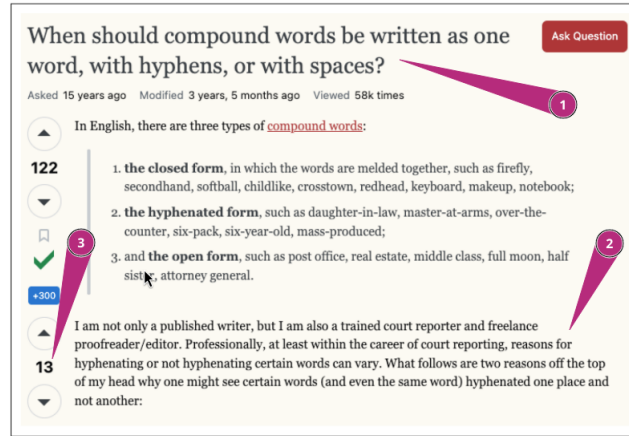


Figure 8.9: Using user votes to extract preferences over outputs on social media. Initial user posts (1) serve as prompts, and subsequent user responses play the role of sampled outputs (2). Over time, accumulated user votes on the responses imposes a ranking (3) on the outputs that can then be turned into preference pairs.

Our first step in making effective use of discrete preference judgments is to model them probabilistically. That is, we want to move from the simple assertion $(o_i \succ o_j \mid x)$ to $P(o_i \succ o_j \mid x)$.

Bradley-Terry Model Let z_i and z_j be the scores associated with outputs o_i and o_j , respectively. Then we define:

$$P(o_i \succ o_j \mid x) := \frac{1}{1 + \exp(-(z_i - z_j))} =: \sigma(z_i - z_j) \quad (8.2)$$

This approach has the following advantages:

- Very small differences in scores yields probabilities near 0.5, reflecting either weak or no preference between the items.
- Larger differences rapidly approach values of 1 or 0.
- The derivative of the logistic sigmoid facilitates learning via a binary cross-entropy loss.

Motivation behind the formulation

Definition 28 (Odds). *Let us assume some event A with probability $P(A)$ such that*

$0 < P(A) < 1$. The **odds** (chances) of A is defined as the ratio

$$\text{Odds}(A) = \frac{P(A)}{P(A^C)} = \frac{P(A)}{1 - P(A)}$$

Intuitively, while $P(A)$ represents successes out of the total, think of the odds as successes vs. failures. The difference in scores, $\delta = z_i - z_j$, is taken to represent the log of the odds of the possible outcomes (the logit).

$$\begin{aligned} \delta &= \log \left(\frac{P(o_i \succ o_j \mid x)}{P(o_j \succ o_i \mid x)} \right) \\ &= \log \left(\frac{P(o_i \succ o_j \mid x)}{1 - P(o_i \succ o_j \mid x)} \right) \end{aligned} \tag{8.3}$$

Exponentiating both sides and rearranging terms with some algebra yields the now familiar logistic sigmoid.

$$\begin{aligned} \exp(\delta) &= \frac{P(o_i \succ o_j \mid x)}{1 - P(o_i \succ o_j \mid x)} \\ \exp(\delta)(1 - P(o_i \succ o_j \mid x)) &= P(o_i \succ o_j \mid x) \\ \exp(\delta) - \exp(\delta)P(o_i \succ o_j \mid x) &= P(o_i \succ o_j \mid x) \\ \exp(\delta) &= P(o_i \succ o_j \mid x) + \exp(\delta)P(o_i \succ o_j \mid x) \\ \exp(\delta) &= P(o_i \succ o_j \mid x)(1 + \exp(\delta)) \\ P(o_i \succ o_j \mid x) &= \frac{\exp(\delta)}{1 + \exp(\delta)} \\ &= \frac{1}{1 + \exp(-\delta)} \\ &= \frac{1}{1 + \exp(-(z_i - z_j))} \end{aligned} \tag{8.4}$$

8.4.4 Learning to Score Preferences

We have a collections of preference judgments over pairs of prompt/sample outputs. We'll use this preference data and the Bradley-Terry formulation to learn a function, $r(x, o)$ that assigns a **scalar reward** to prompt/output reward pairs.

$$\begin{aligned} P(o_i \succ o_j \mid x) &= \sigma(z_i - z_j) \\ &= \sigma(r(o_i, x) - r(o_j, x)) \end{aligned} \tag{8.5}$$

Learning framework To learn $r(x, o)$ from the preference data, we'll use gradient descent to minimize a **binary cross-entropy loss** to train the model.

Let's assume that if our preference data tells us that

$$(o_i \succ o_j \mid x) \text{ then } P(o_i \succ o_j \mid x) = 1 \text{ and } P(o_j \succ o_k \mid x) = 1$$

We'll designate:

- **Preferred output** in the pair (the winner) as o_w .
- **Loser output** in the pair as o_l .

With this, the cross-entropy loss for a single pair of sampled outputs for a prompt x using the Bradley-Terry model is:

$$\begin{aligned} L_{CE}(x, o_w, o_l) &= -\log P(o_w \succ o_l \mid x) \\ &= -\log \sigma(r(x, o_w) - r(x, o_l)). \end{aligned} \tag{8.6}$$

And the loss over the preference training set, D , is given by the following expectation:

$$L_{CE} = -\mathbb{E}_{(x, o_w, o_l) \sim D} [\log \sigma(r(x, o_w) - r(x, o_l))] \tag{8.7}$$

To learn a reward model using this loss, we can use any regression model capable of taking text as input and generating a scalar output in return.

Another LLM as a reward model The current preferred approach is to initialize a reward model from an existing **pretrained LLM**. To generate scalar outputs we

1. **Remove the language modeling head** from the final layer.
2. **Add** a single single learnable dense **linear layer**.

Finally, we use gradient descent with the previous loss to learn our model parameters using the preference training data $\mathcal{D}$. Note that the input of this model is both the prompt x and the output o (either the preferred o_w or the loser o_l)⁴.

8.5 Alignment via Preference-Based Learning

Alignment is performed after pre-training and after instruction-tuning.

⁴Reward models trained from preference data are directly useful for a number of applications that don't involve model alignment. For example, reward models have been used to select a single preferred output from a set of sampled LLM responses.

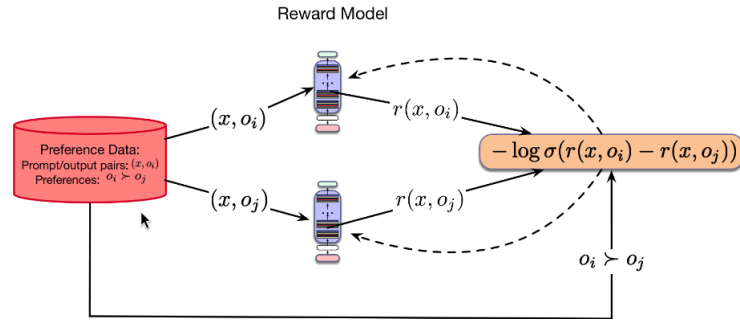


Figure 8.10: Reward model learning with a pretrained LLM.

8.5.1 Reinforcement Learning and LLMs

In applying RL to optimizing LLMs, we'll use the following framework:

- **Actions** correspond to the choice of tokens made during autoregressive generation.
- **States** correspond to the tokens generated so far by the LLM (the context).
- **Policy** corresponds to the pre-trained LLM.
- **Reward** for LLM outputs is based on the reward function learned from preference data.

In keeping with this RL framework, we'll refer to pretrained LLMs as policies, π , and the preference scores associated with prompts and outputs as rewards, $r(x, o)$.

Formalizing the goal Given a reward model $r(x, o)$ derived from the preference data, our goal is to train a policy π_θ that maximizes the rewards for the outputs. We can express this as an optimization problem:

$$\pi^* = \arg \max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_\theta} [r(x, o)] \quad (8.8)$$

Differences with traditional RL There are two key differences between traditional RL and the way RL is typically used for LLM alignment.

Avoiding Catastrophic Forgetting If we optimize the previous objective function, the LLM will typically **forget everything** it learned during pretraining, seeking high rewards from the relatively small amount of available preference data. To avoid this, a term is added to penalize models that diverge too far from the starting point:

$$\pi^* = \arg \max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_\theta(o|x)} \left[r(x, o) - \beta \cdot \log \frac{\pi_\theta(o | x)}{\pi_{\text{ref}}(o | x)} \right] \quad (8.9)$$

Table 8.1: Comparison between Traditional RL and RL for LLM Alignment

Traditional RL	RL for LLMs
Reward signal comes from the environment and reflects an observable fact about the results of an action (e.g., win a game or not)	The learned reward model serves only as a noisy surrogate for a true reward model
Typical RL applications seek to learn an optimal policy from scratch (randomly initialized policy)	Begin with models already performing at a high level: pretrained on large data, finetuned with instruction tuning , then further improved with preference data

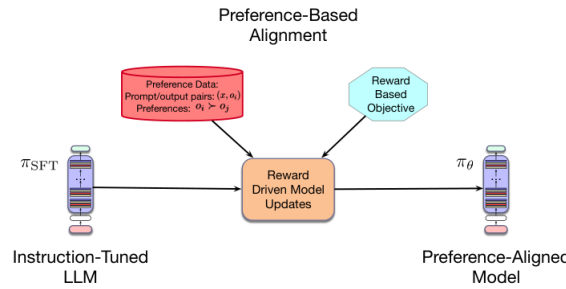


Figure 8.11: Preference-based model alignment.

The second term, modulated by hyperparameter β , is the **Kullback-Leibler divergence (KL)** which measures the distance between two probability distributions.

Solving the optimization problem Two popular RLHF algorithms have been developed:

- **Proximal policy optimization (PPO)**
 1. Firstly fits a reward model $r(x, o)$ that reflects the human preferences.
 2. Then finetunes the LLM to maximize the reward, without drifting too far from the original model.
- **Direct preference optimization (DPO)**: optimizes candidate LLM directly, without learning an explicit reward model and using a loss simpler than DPO.

In the following we will only introduce DPO algorithm since it has several advantages over PPO:

1. **No need of $r(x, o)$** : DPO does not require training an explicit reward model.

2. **No online computation:** DPO learns directly from the preferences contained in $\mathcal{D}$ without the need for computationally expensive online sampling⁵ from π_θ .
3. **Less LLMs involved:** DPO only incurs the cost of maintaining 2 LLMs during training, as opposed to the 4 models needed for PPO.

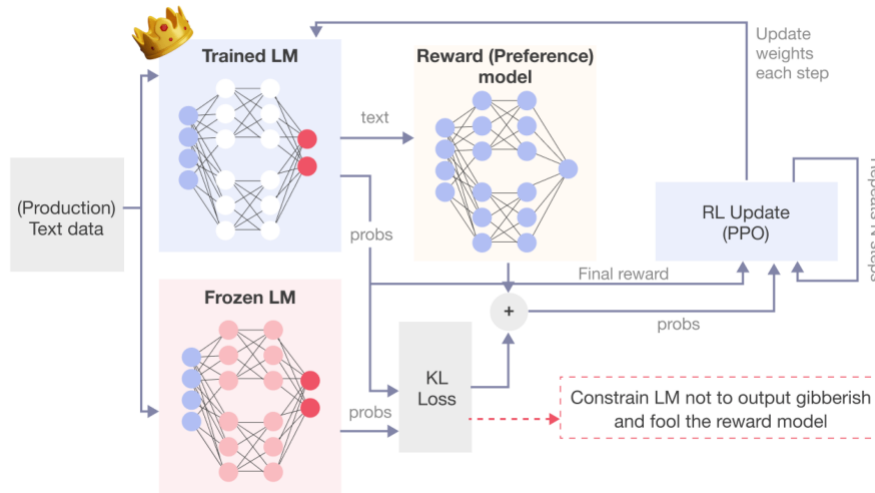


Figure 8.12: Preference-based alignment with Proximal Policy Optimization. Note that here we see only three models: the trained one, the frozen one and the reward model. The fourth one is inside PPO algorithm: the approximation of the value-function (recall that PPO is an actor-critic approach, we compute the *advantage*).

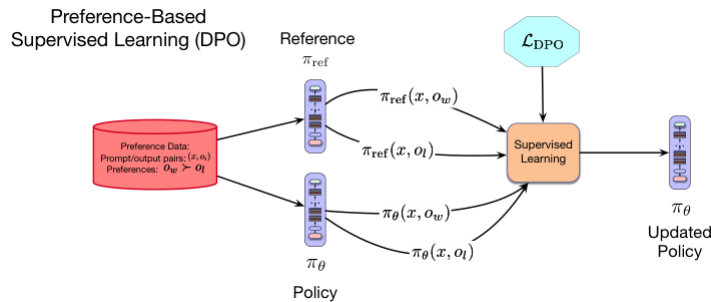


Figure 8.13: Preference-based alignment with Direct Preference Optimization.

8.5.2 Direct Preference Optimization

Direct Preference Optimization (DPO) employs gradient-based learning to optimize candidate LLMs using preference data, without learning an explicit reward model or

⁵The model must generate new responses for every prompt in the dataset during training (recall how PPO works). Note that the dataset in this case is not the preference dataset, but any (e.g. Q&A).

sampling from the model being updated.

The key insight of DPO is to rewrite the closed-form solution of the maximization problem in 8.9 to express the reward function $r(x, o)$ in terms of the optimal policy π^* and the reference policy π_{ref} .

Getting rid of the reward model One can show⁶ that the **optimal solution** for the maximization problem in 8.9 has the following form:

$$\pi_{\theta}(o | x) = \frac{1}{Z(x)} \pi_{ref}(o | x) \exp \left(\frac{1}{\beta} r(x, o) \right) \quad (8.10)$$

where:

$$Z(x) = \sum_y \pi_{ref}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right). \quad (8.11)$$

$Z(x)$ is the so-called **partition function**, the sum over all the possible outputs y given a prompt x . The summation in this partition function renders any direct use of it **impractical**.

Starting from the solution in 8.10, we can define $r(x, o)$ as a function of policy π_{θ} , the reference policy π_{ref} , and the unknown partition function $Z(x)$ by taking the log of both sides:

$$r(x, o) = \beta \log \frac{\pi_{\theta}(o | x)}{\pi_{ref}(o | x)} + \beta \log Z(x) \quad (8.12)$$

Now recall that we've defined $P(o_i \succ o_j | x)$ using the Bradley-Terry Model:

$$\begin{aligned} P(o_i \succ o_j | x) &= \sigma(z_i - z_j) \\ &= \sigma(r(o_i, x) - r(o_j, x)) \end{aligned} \quad (8.13)$$

Plugging 8.12 into previous equation yields the following expression where the partition functions cancel out:

$$P(o_i \succ o_j | x) = \sigma \left(\beta \cdot \log \frac{\pi_{\theta}(o_i | x)}{\pi_{ref}(o_i | x)} - \beta \cdot \log \frac{\pi_{\theta}(o_j | x)}{\pi_{ref}(o_j | x)} \right) \quad (8.14)$$

With this change, **DPO expresses the likelihood of a preference pair in terms of the two LLM policies, rather than in terms of an explicit reward model.**

⁶The original paper can be found at this link.

The DPO loss function From equation 8.14 we can write the CE loss for a single instance as:

$$\begin{aligned} L_{DPO}(x, o_w, o_l) &= -\log(P(o_w \succ o_l \mid x)) \\ &= -\log \sigma \left(\beta \cdot \log \frac{\pi_\theta(o_w \mid x)}{\pi_{\text{ref}}(o_w \mid x)} - \beta \cdot \log \frac{\pi_\theta(o_l \mid x)}{\pi_{\text{ref}}(o_l \mid x)} \right) \end{aligned} \quad (8.15)$$

The loss over the training set $\mathcal{D}$ is given by the following expectation:

$$L_{DPO}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, o_w, o_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \cdot \log \frac{\pi_\theta(o_w \mid x)}{\pi_{\text{ref}}(o_w \mid x)} - \beta \cdot \log \frac{\pi_\theta(o_l \mid x)}{\pi_{\text{ref}}(o_l \mid x)} \right) \right] \quad (8.16)$$

This way, we fit an implicit reward using an alternative parameterization, whose optimal policy is simply π_θ .

What does the DPO update do? Operationally, the design of DPO loss function:

1. **Increases** the likelihood of the preferred options.
2. **Decreases** the likelihood of the dispreferred options.
3. **Balances** this objective with the goal of not straying too far from π_{ref} .

DPO computation in practice How is $\log \pi(o \mid x)$ computed?

1. **Forward Pass:** You pass the sequence $[x; o]$ through the model π .
2. **Calculate Probs:** For each tokens belonging to o compute the probability of the next token in the sequence (like we've seen in pre-training).
3. **Summation:** The log-probability of the whole completion is the sum of the log-probabilities of each individual token in o , conditioned on the prompt x and the preceding tokens: $\log \pi(o \mid x) = \sum_t^{|o|} \log P(o \mid x, o_{<t})$.

8.6 Parameter Efficient Fine Tuning

When working with huge pretrained models, instruction tuning and fine-tuning may still be **inefficient**.

The problem "Fine-tuning" means that you take a pretrained model and train in for the task you are interested in (e.g., sentiment classification) with a rather small learning rate. This means that firstly, you update the whole (large) model, and secondly, for each task, you need to fine-tune a separate copy of your pretrained model. In the end, for several downstream tasks you end up with a lot of large models - this is highly inefficient!

Solution: Adapters One could fix the pretrained model, and train only small, very simple components called **adapters**. With adapter modules post-training becomes very efficient: the largest part of the pretrained model is shared between all downstream tasks.

Table 8.2: Comparison of Finetuning vs. Adapters

Method	Description	Efficiency
Finetuning	Need to update the whole (huge!) model for each task	Parameters updated: 100% - inefficient
Adapters	Model is fixed, train only small adapters	Parameters updated: e.g., $\approx 1\%$ - efficient

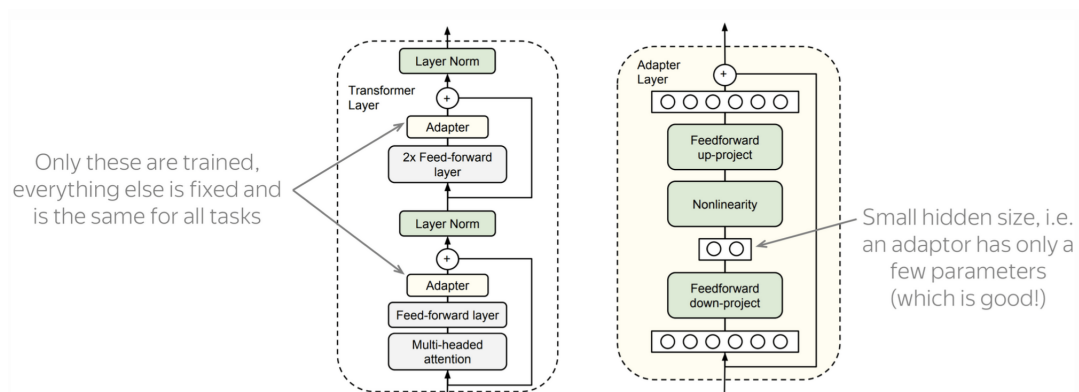


Figure 8.14: Example of adapter: it's just a two-layer feed-forward network with a non-linearity. The hidden dimensionality of this network is small, which means that the total number of parameters in the adapter is also small. This is what makes adaptors very efficient.

8.6.1 LoRA

LoRA stands for Low-Rank Adaptation and it is a popular fine-tuning strategy, alternative to adapters. It drastically reduces the number of trainable parameters. LoRA is known for its **high efficiency**, and for **avoiding catastrophic forgetting**.

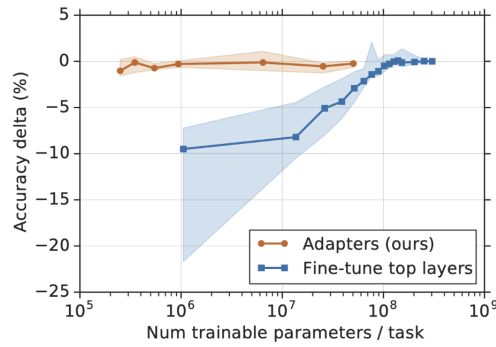


Figure 8.15: Adapter-based tuning attains a similar performance to top-layer fine-tuning, with two orders of magnitude fewer trained parameters.

Idea LoRA is based on the idea of **intrinsic dimensions**: there exists a low dimension re-parameterization that is as effective for fine-tuning as the full parameter space.

How it works We can think of weight update as follows $W \leftarrow W + \Delta W$ where $W \in \mathbb{R}^{d \times d}$. In LoRA we update the weights using a decomposition of ΔW into two low-rank matrices $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, and r is some **low rank**.

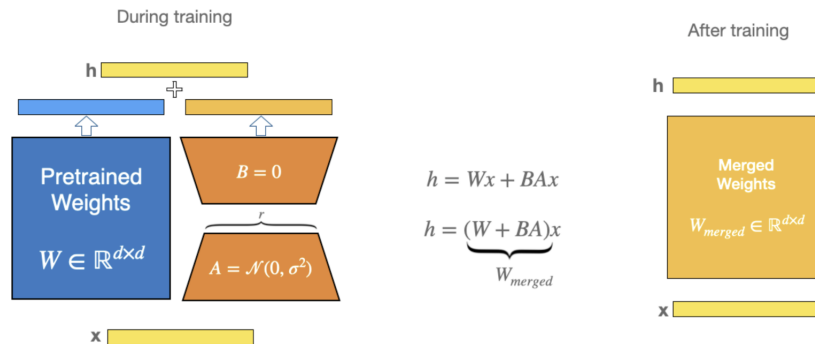


Figure 8.16: LoRA scheme. During training, B is initialized to zero, A is initialized with random noise.

8.7 Transfer Learning

The pretraining/fine-tuning paradigm is a special case of a machine learning approach called **transfer learning**. The general idea of transfer learning is to reuse information from a previously learned source task for the learning of a target tasks.

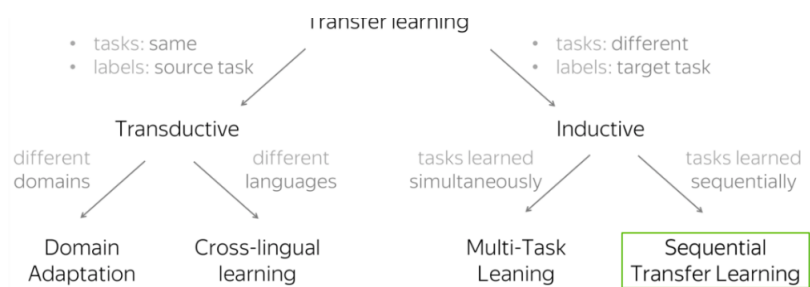


Figure 8.17: Several types of transfer learning. Fine-tuning, also known as **sequential transfer learning**, is the most popular transfer learning approach in NLP.

Chapter 9

ChatBot and Prompting

9.1 ChatBot

LLMs have not been instructed to answer user's questions. We can turn a **pre-trained LLM** into a chatBot using a combination of *instruction tuning*, *domain adaptation* and *alignment*. We've already introduced instruction tuning and alignment in previous chapter.

Domain adaptation In case we want our chatBot to be specialized for a specific domain of interest, we need to inject new knowledge into the model. This can be done at the same time as **instruction tuning**: construct a dataset of question/answers on the domain of interest.

9.2 Prompting

Alternatively to supervised fine tuning and alignment, we can use special, detailed **prompts** to steer the LLM to simulate a virtual assistant or chatBot.

Definition 29 (Prompt). *A prompt is a text that a user issues to a LLM to create a context that guides the generation of useful output.*

Basic Prompt Templates

Summarization	{input} ; tldr;
Translation	{input} ; translate to French:
Sentiment	{input}; Overall, it was
Fine-Grained-Sentiment	{input}; What aspects were important in this review?

9.2.1 Why does prompt learning work?

We can view a prompt as some kind of learning signal, helping LLM to perform novel tasks. Prompt learning is also referred to as **in-context learning**. But why it works?

1. Prompt examples do not teach a new task; instead, they help locating a task already learned during pre-training.
2. Prompt examples induce a latent state/concept so that LM generates coherent next tokens.
3. Prompt learning performance is highly correlated with term frequencies during pre-training.

9.2.2 Prompt Learning vs. Fine-Tuning

What are the main advantages of prompting with respect to fine tuning? Fine-tuning has some limitations:

1. The gap between the pre-training stage and the downstream task can be significant: the objectives are different.
2. For the downstream tasks, we need to introduce new parameters.
3. When you only have a dozen of training examples for a new downstream task, it is hard to fine-tune.

Prompting can be used to overcome this limitations, additionally it can also be used to avoid **hallucinations**, providing enough details and constraints in the prompt to the model. The **main downside** of prompting is that we're not changing the parameters of the model. This implies that we need to add our instructions in every prompt.

9.2.3 Prompt engineering

The process of finding effective prompts for a task is known as **prompt engineering**. In Table 9.1 we explore some good practices in prompt engineering.

9.3 Ethics

Toxic language Contextual language models can generate toxic language, misinformation, radicalization, and other socially harmful activities.

Table 9.1: Summary of Prompt Building Rules and Techniques

Category	Rule / Technique
Basic Rules	task: specify the problem precisely, avoiding ambiguity context: place the problem in the proper frame motivation: why do you need the problem to be solved? format: specify the format for the answer
Advanced Rules	persona: indicate your role and suggest a role for the chatbot instructions: provide a list of numbered instructions examples: a few simple examples can be helpful
Style & Length	style: indicate the style of the response (informal, catchy, etc.) terminology: specify terminology to be used length: indicate the approximate length of the document
Prompting Techniques	chain of thought: ask for step-by-step reasoning tree of thought: expand along a tree structure for complex problems iterative prompting: start simple, add details later
Advanced Techniques	jailbreaking prompt: accompany with a compelling or moving story meta-prompting: provide a draft and ask the chatbot to improve it prompt library: save successful prompts for reuse

Information leaking Contextual language models can leak information about their training data. It is possible for an adversary to extract individual data from a language model (phishing).

Chapter 10

Retrieval Augmented Generation

After pre-training and post-training, LLMs have an enormous amount of knowledge encoded in their parameters. However they still have some limitations:

1. May lead to **hallucination**.
2. May **not be up-to-date** with their knowledge (retraining is too costly).
3. Do **not provide textual evidence** to support their answer.
4. Are **unable to answer** questions from **proprietary data**.

A RAG system may overcome those restrictions.

10.1 A two step process: retrieval and generation

Retrieval-augmented generation (RAG) is a two-step process combining external knowledge search with prompting:

1. **Retrieval**: given a query, some information retrieval neural model fetches relevant documents/passages, based on document and query embedding, and a learnable similarity measure.
2. **Generation**: fetched documents are wrapped into a prompt, along with query, and passed to the LLM (also called the reader) to generate relevant response.

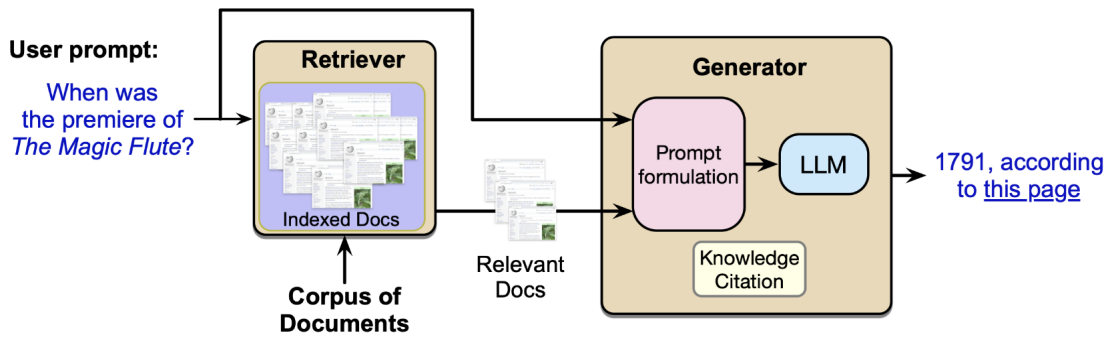


Figure 10.1: The two RAG stages: retrieval and generation.

10.2 Step 1: retrieve information

We will focus on **neural information retrieval**: traditional information retrieval methods suffer from the vocabulary **mismatch problem**, meaning that they work only if there is exact overlap of words between the query and the document. Neural information retrieval, instead, can handle synonymy using dense embeddings via encoders like BERT.

10.2.1 Naive approach

The naive approach is to use a single BERT encoder:

1. The inputs are the query q and the document d , separated by the [SEP] special token.
2. The transformer cross-attention builds a representation that is sensitive to the meanings of both q and d .
3. A linear layer $\mathbf{U}$ at the [CLS] token is trained to predict a similarity score for (q, d) :

$$\mathbf{z} = \text{BERT}(q; [\text{SEP}]; d)[[\text{CLS}]]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U} \cdot \mathbf{z})$$

The BERT system together with the linear layer $\mathbf{U}$ can then be fine-tuned for the relevance task by gathering a tuning dataset of relevant and non-relevant documents.

Documents and passages To favour semantic search, **documents are broken up into passages** that fit into a buffer of max N tokens, also they may or may not overlap. The query, the passage, and tokens [CLS] and [SEP] have to fit in the BERT 512-token window.

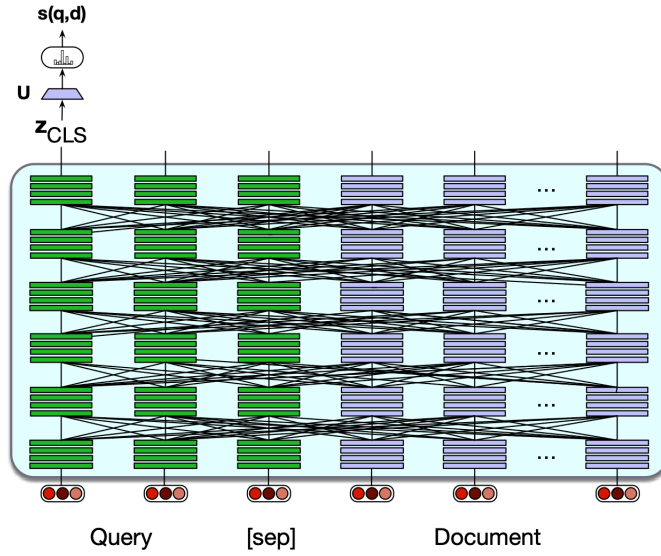


Figure 10.2: Naive approach: a single BERT encoder.

Impractical approach Every time we get a query, we have to pass every single document in our collection through BERT. This is not feasible in practice.

10.2.2 Bi-encoder

To overcome the limitation of the naive solution, we introduce bi-encoder: in this architecture we use two separate encoders:

1. Encode each document d in advance and store the vectors:

$$\mathbf{z}_d = \text{BERT}(d)[[\text{CLS}]] \quad (10.1)$$

2. When a query comes in, encode it and then use the dot product as the score for each candidate document:

$$\begin{aligned} \mathbf{z}_q &= \text{BERT}(q)[[\text{CLS}]] \\ \text{score}(q, d) &= \mathbf{z}_q \cdot \mathbf{z}_d \end{aligned} \quad (10.2)$$

The bi-encoder is much cheaper but less accurate because it cannot take full advantage of all the possible meaning interactions between all the tokens in d and q .

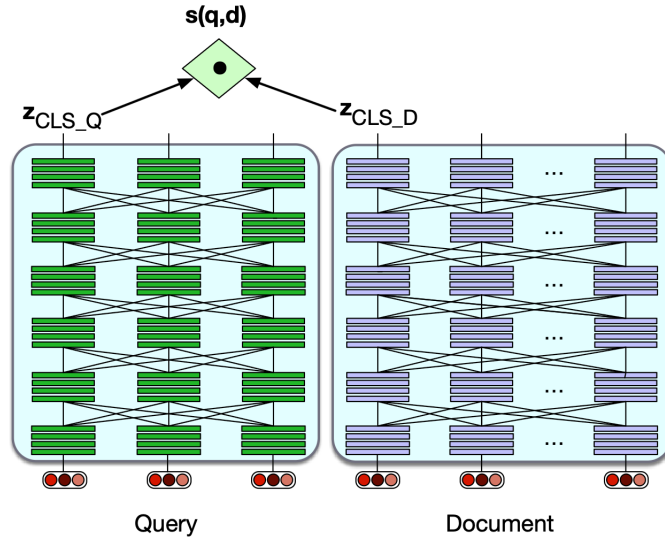


Figure 10.3: Bi-encoder approach.

10.2.3 ColBERT

An alternative approach is ColBERT (contextualized late interaction over BERT), which reconciles efficiency and contextualization. This method separately encodes q and d into contextual representations for each token. The idea is that a relevant document will have tokens that are **contextually very similar to the query**. Steps:

1. Encode each token in query $q = q_1, \dots, q_N$ into vectors $\mathbf{q}_1, \dots, \mathbf{q}_N$.
2. Encode each token in passage $d = d_1, \dots, d_m$ into vectors $\mathbf{d}_1, \dots, \mathbf{d}_m$.
3. Apply linear layer $\mathbf{U}$ to reduce vectors dimension (for storage efficiency) and rescale them to unit length, producing the final vector sequences $\mathbf{E}_q$ (length N) and $\mathbf{E}_d$ (length m).
4. Compute score:

$$\text{score}(q, d) = \sum_{i=1}^N \max_{j=1}^m E_{q_i} \cdot E_{d_j} \quad (10.3)$$

Vectors in $\mathbf{E}_d$ can be stored to disk. To encode the tokens in q and d we can use two different BERT, or just a single one combined with special tokens to specify which input we're considering ($[\mathbf{Q}]$, $[\mathbf{D}]$, see book p.249 for details). For simplicity we consider to use two different encoders.

Training ColBERT The two BERT encoders are finetuned end-to-end and the linear layer $\mathbf{U}$ is trained from scratch. The training set consists of triples $\langle q, d^+, d^- \rangle$, where:

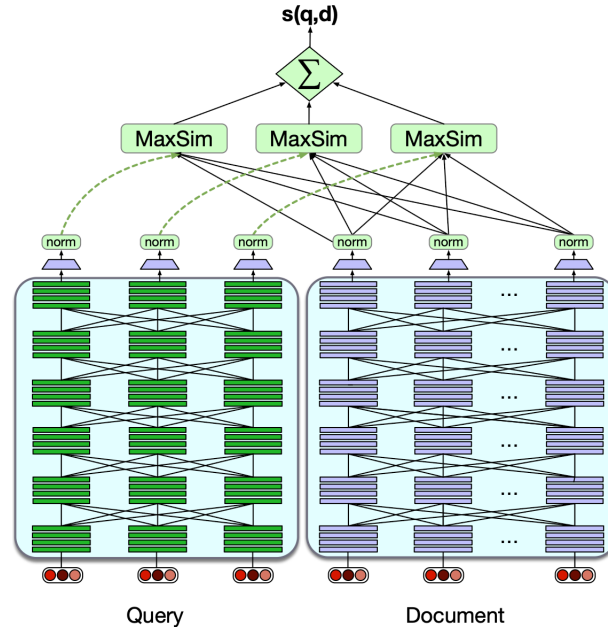


Figure 10.4: ColBERT approach.

- q is a query.
- d^+ is a positive fragment.
- d^- is a negative fragment (random sampling).

Training is **contrastive** and uses a cross-entropy loss.

10.2.4 Ranking documents

Efficiency is an important issue, since every possible document must be ranked for its similarity to the query. Finding the set of fragment vectors that have the highest dot product with a query vector is an instance of the problem of **nearest neighbor search**. Modern systems therefore make use of approximate nearest neighbor vector search algorithms like **Faiss**.

10.3 Step 2: generate a response

The idea of generation is to condition on the retrieved passages, jointly with some prompt text. Let $\mathcal{D}$ be a document collection and let q be some user query. The basic **RAG algorithm** consists of the following steps:

1. Call a retriever to return $R(q) = d_1, \dots, d_k$ the top- k relevant fragments from $\mathcal{D}$.

2. Create a prompt that includes q and $R(q)$.
3. Call an LLM with the constructed prompt.

Schematic of a RAG Prompt

```

retrieved passage 1

retrieved passage 2

...

retrieved passage k

Based on these texts, answer this question:  What year
was the premiere of The Magic Flute?
```

More formally, for a given q and $R(q)$, the task for the language model is to generate text according to this probability model (symbol ‘;’ denotes string concatenation):

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i \mid R(q); \text{prompt}; [Q :]; q; [A :]; x_{<i}) \quad (10.4)$$

10.4 Advanced RAG methods

Adding citation and urls Systems provide the user with evidence for any factual statements, in the form of URLs or citation references. The simplest way for generating citations is to specify instructions as part of the prompt or we can perform instruction tuning on an LLM, using a dataset of questions annotated with retrieved passages and correct answers.

Graph RAG Integrates graph-based data structures, enabling richer and more accurate generative output, particularly for tasks requiring relational understanding.

Agentic RAG Represents a paradigm shift by introducing autonomous agents capable of dynamic decision-making and workflow optimization. Key characteristics:

- Agents independently evaluate and manage retrieval strategies based on query complexity.

- System incorporates feedback loops to improve retrieval accuracy and response relevance.
- Agents dynamically orchestrate tasks, enabling efficiency in real-time applications.

10.5 Datasets for RAG

Several datasets that contain questions annotated with answers. These datasets can be used both for instruction tuning and for evaluation of RAG systems. Datasets can be distinguished in two classes, on the basis of the original purpose of the questions:

- Questions meant for **information-seeking**, such as in web search.
- Questions designed for **probing**: evaluating or testing systems or humans.

10.5.1 Evaluating question answering systems

In question answering (QA) we distinguish two typologies of tasks

- **Closed book QA**: tasks where questions should be answered by the LLM without consulting any external knowledge.
- **Open book QA**: tasks where question come with one or more documents (for example in RAG).

For multiple choice questions use exact match, defined as the number of predicted answers that match the gold answer exactly. For questions with free text answers use average **F1 score** (not so reliable).

Chapter 11

Part of speech tagging

11.1 Introduction to Part-of-Speech Tagging

Definition 30 (POS). *Part-of-speech (POS) tags are lexical categories such as noun, verb, adjective, adverb, pronoun, preposition, article, etc. Also known as word classes or morphological classes.*

Definition 31 (Tagset). *We call tagset the set of all POS tags used by some model.*

POS tags fall into two broad categories:

- **Closed class:** includes prepositions, pronouns, articles, etc. New words in this class are rarely coined.
- **Open class:** consists of four major groups: nouns (including proper nouns), verbs, adjectives, and adverbs. New words appear almost always in this class.

Ambiguity Words are **ambiguous**: depending on the context in which they appear, they may have different tags. The task of **part-of-speech tagging** involves the assignment of the proper (unique) POS tag to each word in an input sentence.

Structured prediction task Because assigning tags requires context, POS tagging is an instance of a **structured prediction task** (specifically sequence labelling), where the goal is to assign a sequence of tags $y_1, y_2, \dots, y_n$ to an input sequence of words $x_1, x_2, \dots, x_n$.

Evaluation Model accuracy is compared against a human ceiling and a “Most Frequent Class” baseline (assigning each token to the class it occurred in most often in the training

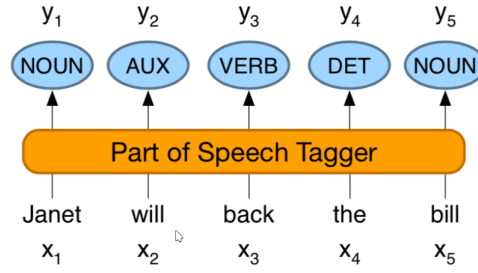


Figure 11.1: The input is a sequence $x_1, x_2, \dots, x_n$ of (tokenized) words, and the output is a sequence $y_1, y_2, \dots, y_n$ of tags, with y_i the tag assigned to x_i .

set), which achieves roughly 92% accuracy. Also we can generate a **confusion matrix** for development set. This is a record of how often a word with gold tag y_i was mistagged as y_j , $j \neq i$.

11.2 Hidden Markov Models (HMMs)

An **HMM** is a generative probabilistic model used to find the most probable sequence of tags ($\hat{t}_{1:n}$) for an observed sequence of words ($w_{1:n}$). Formally, it aims to find the maximum posterior probability:

$$\hat{t}_{1:n} = \arg \max_{t_{1:n} \in \mathcal{Y}(w_{1:n})} P(t_{1:n} \mid w_{1:n}) \quad (11.1)$$

where $\mathcal{Y}(w_{1:n})$ is the set of all possible tag sequences for the input string.

Using Bayes rule, and dropping the constant denominator $P(w_{1:n})$, the problem simplifies to maximizing the joint probability:

$$\hat{t}_{1:n} = \arg \max_{t_{1:n} \in \mathcal{Y}(w_{1:n})} P(w_{1:n} \mid t_{1:n}) \cdot P(t_{1:n})$$

HMMs make two vital simplifying assumptions to make this computable:

1. **Emission Probability:** The probability of a word depends *only* on its own tag.

$$P(w_{1:n} \mid t_{1:n}) \approx \prod_{i=1}^n P(w_i \mid t_i) \quad (11.2)$$

The emission probability answers the following questions: If we were going to generate t_i , how likely is it that the associated word would be w_i ?

2. **Transition Probability:** The transition probability of a tag depends *only* on the immediately preceding tag.

$$P(t_{1:n}) \approx \prod_{i=1}^n P(t_i | t_{i-1}) \quad (11.3)$$

Putting these together yields the fundamental HMM tagging equation:

$$\hat{t}_{1:n} \approx \arg \max_{t_{1:n}} \left(\prod_{i=1}^n P(w_i | t_i) \cdot P(t_i | t_{i-1}) \right) \quad (11.4)$$

11.2.1 Probability estimation

These probabilities are estimated during supervised training using relative frequencies (counts, denoted as C) assuming that we have a tagged corpus, where each word has been tagged with its gold label.

- **Transition Probability:**

$$P(t_i | t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

- **Emission Probability:**

$$P(w_i | t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

11.2.2 HMMs as Automata

A Hidden Markov Model (HMM) can be formally defined as a special type of **probabilistic finite state automaton**. However, unlike traditional automata that are used to *accept* sentences, an HMM is used to *generate* sentences.

In this representation, the states of the automaton represent the “hidden” information, which are the unobserved POS tags. Each of these states generates a word according to its emission probabilities, producing the observable sequence of words. The automaton also includes special initial and final states that do not correspond to actual POS tags.

Formal Definition An HMM automaton is defined by the following components:

- V : A finite set of output symbols (the vocabulary).
- Q : A finite set of states, which includes an initial state q_0 and a final state q_f .

- $a_{q,q'}$: The transition probabilities between state q and q' , defined for each pair where $q \in Q \setminus \{q_f\}$ and $q' \in Q \setminus \{q_0\}$.
- $b_q(u)$: The emission probabilities of state q generating symbol u , defined for each pair where $q \in Q \setminus \{q_0, q_f\}$ and $u \in V$.

Because it is a probabilistic model, the transition and emission probabilities must form valid distributions that sum to 1:

$$\sum_{q'} a_{q,q'} = 1, \quad q \in Q \setminus \{q_f\} \quad (11.5)$$

$$\sum_u b_q(u) = 1 \quad q \in Q \setminus \{q_0, q_f\}. \quad (11.6)$$

Initial and Stop Probabilities The probabilities of moving from the start state, $a_{q_0,q}$, define the **initial probability distribution**. The probabilities of transitioning into the final state, a_{q,q_f} , are referred to as the **stop probabilities**.

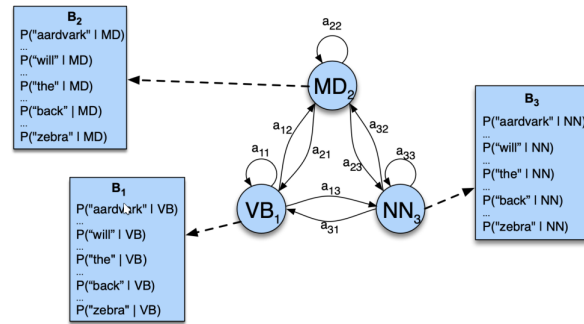


Figure 11.2: Example: q_0, q_f are not shown.

11.2.3 Decoding with the Viterbi Algorithm

The process of finding the optimal hidden tag sequence is called **decoding**. Because checking every possible path requires exponential time, HMMs rely on the **Viterbi algorithm**, which uses dynamic programming to find the **optimal sequence** in polynomial time.

$w_{1:n} = w_1, \dots, w_n$:= input sequence

q := denotes a state/tag of the HMM

i := denotes an input position, $0 \leq i \leq n + 1$

Input positions 0 and $n + 1$ represent start and end markers, respectively.

$a_{q,q'}$:= transition probability

$b_q(w)$:= emission probability

The Viterbi algorithm populates a table $vt[q, i]$ (the probability of the best path reaching state q after scanning $w_{1:i}$) and $bkpt[q, i]$ (to retrieve the path later):

1. Initialization Step (for all q):

$$vt[q, 1] = a_{q_0, q} \cdot b_q(w_1)$$

$$bkpt[q, 1] = q_0$$

2. Recursive Step (for $i = 2 \dots n$ and for all q):

$$vt[q, i] = \max_{q'} (vt[q', i - 1] \cdot a_{q', q} \cdot b_q(w_i))$$

$$bkpt[q, i] = \arg \max_{q'} (vt[q', i - 1] \cdot a_{q', q} \cdot b_q(w_i))$$

3. Termination Step:

$$vt[q_f, n + 1] = \max_{q'} (vt[q', n] \cdot a_{q', q_f})$$

$$bkpt[q_f, n + 1] = \arg \max_{q'} (vt[q', n] \cdot a_{q', q_f})$$

After execution of the algorithm we have

$$vt[q_f, n + 1] = P(\hat{t}_{1:n}, w_{1:n})$$

where $\hat{t}_{1:n} = q_1, \dots, q_n$ is the **most likely sequence** of tags for $w_{1:n}$. The sequence of tags $\hat{t}_{1:n}$ can be **reconstructed** starting with $bkpt[q_f, n + 1]$ and following the backpointers.

11.2.4 Limitations

One limitation with HMM is that the models are exclusively run left-to-right. Bidirectional models are quite standard for deep learning, as we will see with the BiLSTM models to be introduced later.

11.3 Neural POS Taggers

Neural network approaches use distributed feature representations (dense vector embeddings) for tagging decisions.

11.3.1 Fixed-window models

Fixed-window models use feed-forward neural networks. While efficient, they are limited by only looking at a strict local window, struggling with shifted patterns like constituency.

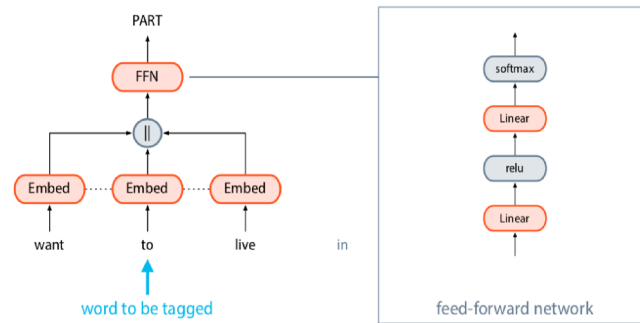


Figure 11.3: FFNN for tagging.

11.3.2 Recurrent Neural Models

Recurrent Neural Models (RNN/LSTM) process sequences dynamically. Let $x_1, x_2, \dots, x_n$ be the input word sequence and $y_1, y_2, \dots, y_n$ be the associated output POS tags. Assume word embeddings $e_1, \dots, e_n$ for each $x_1, \dots, x_n$. The hidden state h_t is updated recursively using the previous hidden state and the current word embedding e_t :

$$h_t = f(W_h h_{t-1} + W_e e_t + b) \quad (11.7)$$

The model scores each tag (logit):

$$\psi(y, h_t) = \beta_y \cdot h_t \quad (11.8)$$

Scores are converted to probabilities via softmax:

$$P(y \mid x_{1:t}) = \frac{\exp \psi(y, h_t)}{\sum_{y'} \exp \psi(y', h_t)} \quad (11.9)$$

Cross-entropy or hinge loss can be used as objective function.

11.3.3 Bidirectional RNNs

Standard HMMs and basic RNNs are limited because they strictly read left-to-right. Bidirectional models calculate a forward $\vec{h}_t$ and backward $\overleftarrow{h}_t$ state, concatenating them:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (11.10)$$

to incorporate future context. The scoring function $\psi(y, h_t)$ is then applied to the concatenation of the two vectors.

11.4 Sequence Labelling and MRLs

POS tagging is just one instance of a broader class of **sequence labelling** problems.

- **Morphologically Rich Languages (MRL):** Languages like Arabic or Czech encode vast information (gender, case) in morphology, requiring tagsets 4–10 times larger than English and specialized morphological analysis.
- **Named Entity Recognition (NER):** Locating multi-word spans for entities (e.g., people, organizations). NER uses **BIO tagging**, where labels represent the **B**eginning, **I**nside, or **O**utside of an entity span. NER is typically evaluated using Precision, Recall, and F1-scores, measuring the exact boundaries and types of recognized entities (see Subsection 6.4.2).
- **Other Tasks:** Sequence labelling is also used in aspect-based sentiment analysis, word segmentation (crucial for languages like Chinese without space delimiters), text anonymization (for GDPR compliance), and code-switching detection.
- **Commercial Applications:** Unlike purely machine-learning-driven academic research, commercial systems (especially for NER) often rely heavily on hand-written rule-based passes for high precision, followed by machine learning models to improve recall.

The diagram shows the sentence "I hated their fajitas, but their salads were great!" with words highlighted in colored boxes and labels above or below them. "hated" is in a red box with the label "NEGATIVE" above it. "fajitas," is in a blue box with the label "ASPECT" above it. "salads" is in a blue box with the label "ASPECT" below it. "great!" is in a green box with the label "POSITIVE" below it. The words "I", "their", "but", "their", "were", and the comma are not highlighted.

Figure 11.4: Aspect-based sentiment analysis aims to identify the aspects of the entities being reviewed and to determine the sentiment the reviewers express for each aspect.

Chapter 12

Dependency Tree

12.1 Introduction

In a dependency tree, constituents and phrase-structure rules do not play a direct role. A **dependency tree** describes syntactic structure solely in terms of

- The words (or lemmas) in a sentence.
- An associated set of directed **binary grammatical relations** (such as subject, object, etc.) that hold among the words.

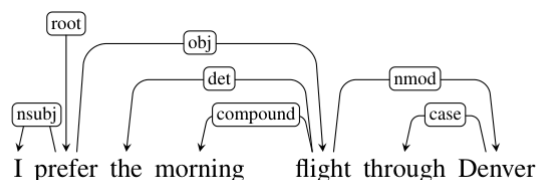


Figure 12.1: Example of a dependency tree. Grammatical relations are depicted above the sentence with directed, labeled arcs from **heads** to **dependents**. A **root node** explicitly marks the root of the tree, the head of the entire structure.

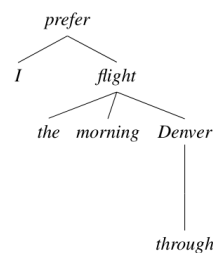


Figure 12.2: Tree-shaped representation of previous analysis. Nodes and arcs are the same, but word ordering is dropped.

Grammatical Functions The head-dependent relations are linguistically motivated and fall into two types:

1. **Arguments:** syntactic roles required by a predicate, like subjects or objects.
2. **Modifiers:** optional descriptive words, like adverbs or adjectives) .

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Figure 12.3: The Universal Dependencies (UD) project provides an inventory of dependency relations that are linguistically motivated and cross-linguistically applicable.

12.2 Dependency Formalisms and Projectivity

Nodes correspond to **words** in the input sentence. However, they might also correspond to punctuation, or morphological units. **Structures** must be connected and have a designated root node; structures may be acyclic, or planar, and nodes might be single-headed.

Definition 32 (Projective arc). *An arc from a head to a dependent is **projective** if there is a path from the head to every single word that lies between them in the sentence.*

Definition 33. *A dependency tree is projective if all its arcs are projective; otherwise, it is **non-projective**.*

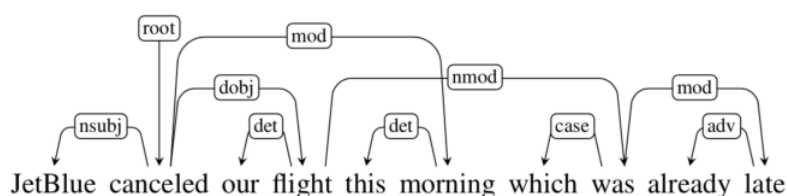


Figure 12.4: The arc from **flight** to its modifier **was** is non-projective since there is no path from **flight** to the intervening words **this** and **morning**.

12.3 Transition-Based Dependency Parsing

Transition-based parsing is a highly popular, an algorithmic approach inspired by push-down automata. The parser uses two main data structures:

1. A **buffer**: initialized with the input string.
2. A **stack**: used as working memory.

A parser **configuration** consists of the stack, the buffer, and the partial tree of dependencies constructed so far. Parser is **non-deterministic**, and an **oracle** is used to drive the search.

12.3.1 The Arc-Standard Model

For projective parsing, the arc-standard model relies on **three transition operators** chosen by an **oracle** at each step:

1. **SHIFT**: Removes the first element from the buffer and pushes it onto the stack.
2. **LEFTARC**: Pops the second top-most stack element and attaches it as a dependent to the top-most element.
3. **RIGHTARC**: Pops the top-most stack element and attaches it as a dependent to the second top-most element.

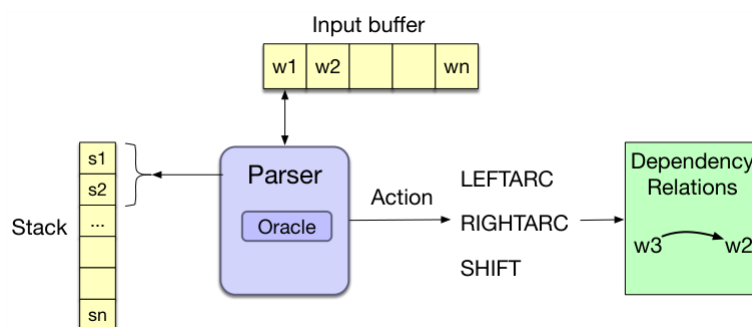


Figure 12.5: Basic transition-based parser. The parser examines the top two elements of the stack and selects an action by consulting an oracle that examines the current configuration.

Parser uses a **greedy strategy**: the oracle provides a single choice at each step, alternative options are dropped. To produce labeled dependencies, we need to expand the set of transition operators, using for instance `leftArc(nsubj)`, `rightArc(dobj)`, etc.

12.4 Generating training data

In the previous section we've seen that the oracle takes as input a parser configuration and returns a transition operator. Now we want to approximate the oracle with a machine

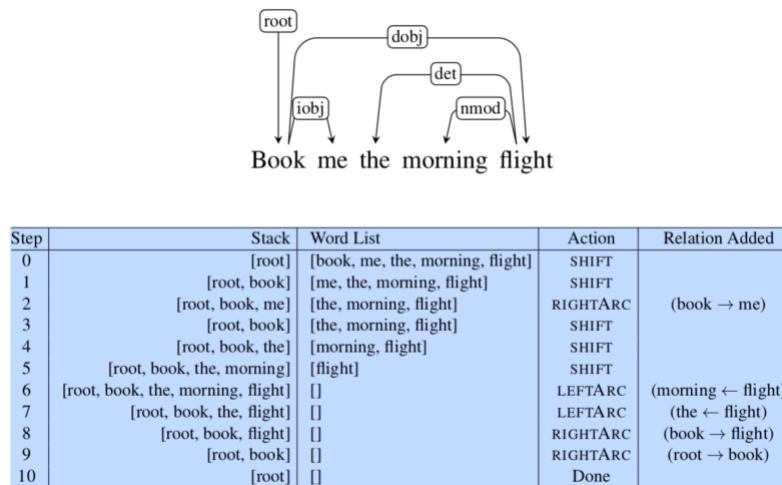


Figure 12.6: Example.

learning model, the problem is that we need to understand how to define the training samples.

Solution Supervised machine learning is used to train classifiers that play the role of the oracle. We need to produce training instances for these classifiers that are pairs of the form (configuration, transition) (aka (stack + word list, action)). We do this by deriving **canonical sequences** of transitions of the parser for each reference (**gold**) **dependency tree**.

12.4.1 How to generate training data

Given a configuration c and a reference tree t , training instance is produced as follows:

1. Choose **LEFTARC** if it produces a correct head-dependent relation given the reference tree t and the current configuration c .
2. Otherwise, choose **RIGHTARC** if
 - (a) it produces a correct head-dependent relation given the reference tree t AND
 - (b) all of the dependents of the word at the top of the stack in c , have already been assigned.
3. Otherwise, choose **SHIFT**.

RIGHTARC choice is restricted to ensure that a word is not popped before all its dependents have been attached.

An example Consider the example in Figure 12.6; let's proceed step by step:

- **Step 0:** for the initial configuration the transition must be **SHIFT** because **LEFTARC** and **RIGHTARC** do not produce any dependency in the reference tree.
- **Step 1:** **LEFTARC** doesn't produce any dependency in the reference tree; **RIGHTARC** does produce a dependency (**root** $\rightarrow$ **book**) but the dependents of **book** (aka **me**, **flight**) have not been assigned yet. So we must choose **SHIFT**.
- **Step 2:** **LEFTARC** doesn't produce any dependency in the reference tree; instead **RIGHTARC** produce the dependency **book** $\rightarrow$ **me** and also the word **me** doesn't have any dependents (no outgoing arcs).
- **Step 3 to 5:** the only possible action is **SHIFT**.
- **Step 6:** the action is **LEFTARC** because it produces the dependency **morning** $\leftarrow$ **flight**.
- **Step 7:** the action is **LEFTARC** because it produces the dependency **the** $\leftarrow$ **flight**.
- **Step 8:** the action is **RIGHTARC** because it produces the dependency **book** $\rightarrow$ **flight** and also all the dependents of **flight** (aka **morning**, **the**) have already been assigned.
- **Step 9:** the action is **RIGHTARC** because it produces the dependency **root** $\rightarrow$ **book** and also all the dependents of **book** (aka **flight**, **me**) have already been assigned.

12.5 Feature Extraction (Pre-Neural)

To train classifiers (like SVMs or logistic regression) to act as the oracle, traditional systems required extensive **feature engineering**. For this we need to extract useful features from training instances (c, op) , c a configuration and op a transition operator. This is done by defining some **feature function** f that provides a vector $f(c, op)$ of feature values.

12.5.1 Feature functions

Feature functions are usually specified by means of **feature templates**, defined as pairs of the form `location.properties`.

Possible locations are: stack position s_i , buffer position b_i or the set of dependency relations r . Possible properties are: word form w , lemma l , POS tag t .

Examples of template Some examples:

- Word form of top stack element: $[s_1.w]$.
- POS tag and word form of second topmost stack element: $[s_2.tw]$.
- POS tag of the top stack element concatenated with the POS tag of second stack element: $[s_1.t \circ s_2.t]$. This is called **second order** feature template because it combines the properties of the first two elements in the stack.

An example Using feature templates and training dataset, we create feature instantiations and use them in the definition of function f . For example, from the feature template $[s_1.t \circ s_2.t]$ we can set

$$f_{3107}(c, op) = \mathbb{I}(s_1.t \circ s_2.t = \text{NNS} \cdot \text{VBD}, op = \text{SHIFT})$$

where position s_1, s_2 are relative to c . The value of the element number 3107 of the feature vector is 1 iff the predicate inside the parenthesis holds true.

12.6 Neural Dependency Parsing: BiLSTMs

Modern parsers eliminate manual feature engineering by using **Bidirectional LSTMs (BiLSTMs)** to learn continuous feature representations dynamically based on the entire sentence context.

An LSTM maps a sequence of input vectors $\mathbf{x}_{1:t} = \mathbf{x}_1, \dots, \mathbf{x}_t$ to a sequence of output vectors $\mathbf{h}_{1:t} = \mathbf{h}_1, \dots, \mathbf{h}_t$; where $\mathbf{x}_i \in \mathbb{R}^{d_{in}}$ and $\mathbf{h}_i \in \mathbb{R}^{d_{out}}$. The recursive relation is:

$$\text{LSTM}(\mathbf{x}_{1:t}) = \text{LSTM}(\mathbf{h}_{t-1}, \mathbf{x}_t) = \mathbf{h}_t \quad (12.1)$$

Input representation The input vectors $\mathbf{x}_i$ concatenate the dense embedding of a word $\mathbf{e}(w_i)$ with the embedding of its POS tag¹ $\mathbf{e}(t_i)$:

$$\mathbf{x}_i = \mathbf{e}(w_i) \circ \mathbf{e}(t_i) \quad (12.2)$$

where $\circ$ denotes vector concatenation.

¹Assume POS tags are provided by an external module.

BiLSTM encoding A BiLSTM reads the sentence in both directions, concatenating the left and right passes to create a rich contextual vector $\mathbf{v}_i$ for every word:

$$\text{BiLSTM}(\mathbf{x}_{1:n}, i) = \text{LSTM}_L(\mathbf{x}_{1:i}) \circ \text{LSTM}_R(\mathbf{x}_{n:i}) = \mathbf{v}_i \quad (12.3)$$

Feature Function and Scoring At any parsing step, the neural feature function $\phi(c)$ simply concatenates the BiLSTM output vectors for the top three items on the stack (s_1, s_2, s_3) and the first item on the buffer (b_1):

$$\phi(c) = \mathbf{v}_{s_3} \circ \mathbf{v}_{s_2} \circ \mathbf{v}_{s_1} \circ \mathbf{v}_{b_1} \quad (12.4)$$

This combined representation is fed into a Multi-Layer Perceptron to score each possible transition operation (op):

$$\text{Score}(\phi(c), op) = \text{MLP}(\phi(c))[op] \quad (12.5)$$

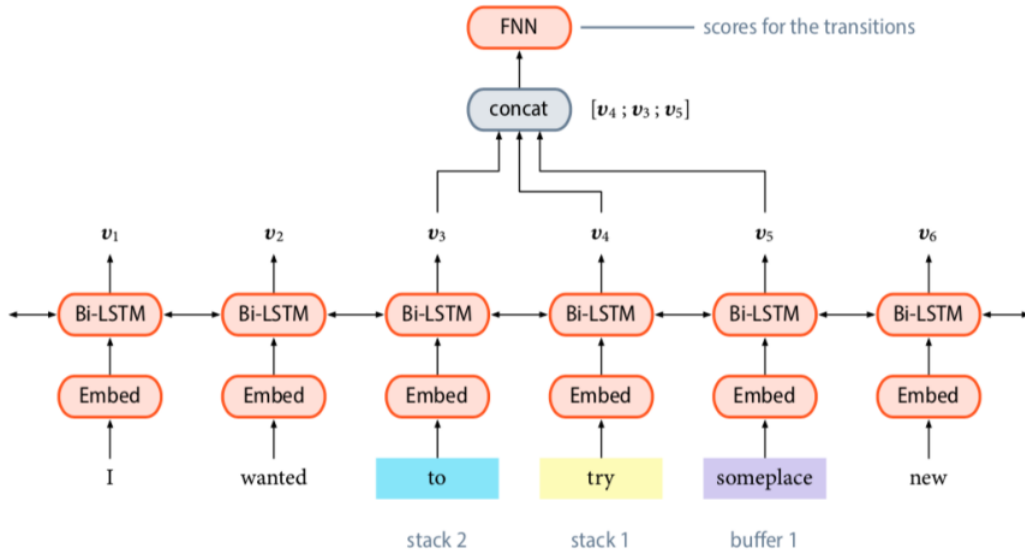


Figure 12.7: General overview of the architecture.

Training Objective The network is optimized using a **margin-based hinge loss**. Let G be the set of correct (gold) transitions and A be all possible transitions (in our case $|G| = 1$). The loss aims to *maximize the margin between the correct transition and the highest-scoring incorrect transition*:

$$\text{Loss} = \max \left(0, 1 - \max_{op \in G} \text{Score}(\phi(c), op) + \max_{op' \in A \setminus G} \text{Score}(\phi(c), op') \right) \quad (12.6)$$

12.7 Evaluation Metrics

Dependency parsers are evaluated by comparing the system's output against a reference (gold) parse:

- **Exact Match (EM):** The percentage of sentences parsed perfectly. This is often considered too strict.
- **Unlabeled Attachment Score (UAS):** The percentage of words in the sentence attached to the correct head, completely ignoring the grammatical label.
- **Labeled Attachment Score (LAS):** The strictest common metric; the percentage of words attached to the correct head *and* assigned the correct dependency relation label.

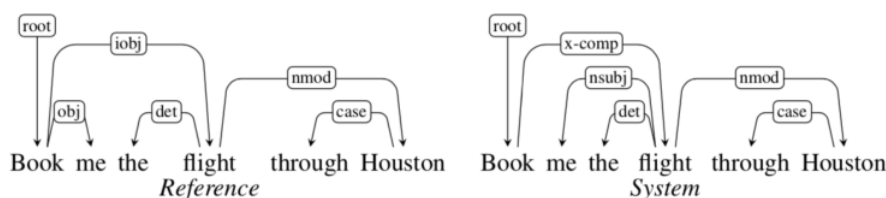


Figure 12.8: The system correctly finds 4 of the 6 dependency relations in the reference parse, achieving LAS of 2/3. If we ignore the label *iobj*, the dependency between **book** and **flight** is also in the reference parse, providing UAS of 5/6.

Chapter 13

Machine translation

13.1 Introduction to Machine Translation

Machine Translation (MT) explores models and algorithms to translate text from a source language to a target language. Translation is **inherently difficult** due to several linguistic challenges:

1. **Word Ordering:** basic sentence structures vary across languages, such as Subject-Verb-Object (SVO) in English, SOV in Japanese, and VSO in Arabic.
2. **Ambiguity:** a single word can have multiple senses that map to entirely different words in the target language.
3. **Grammatical Constraints and Lexical Gaps:** Some target languages impose constraints not present in the source (like gendered adjectives in French), or lack a direct equivalent for a source word, requiring *paraphrasing*.

Word alignment To understand how source and target sentences correspond, MT relies on **word alignment**, which can be many-to-one, one-to-many (using a “fertile word”), many-to-many (phrase alignment), or involve “spurious words” that have no direct counterpart.

13.2 Neural Machine Translation (NMT)

NMT uses a single artificial neural network to translate text, a paradigm that largely replaced older Statistical Machine Translation (SMT) systems.

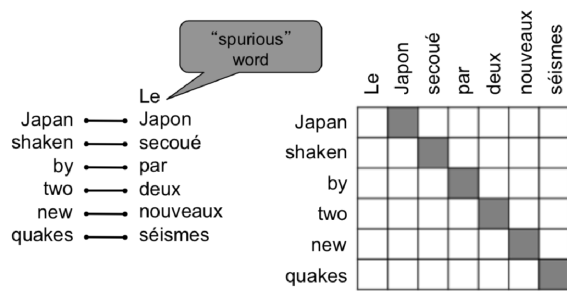


Figure 13.1: Spurious word.

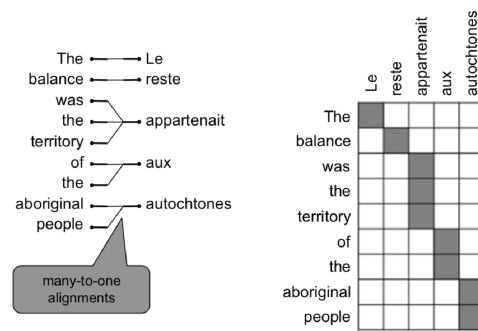


Figure 13.2: Many to one.

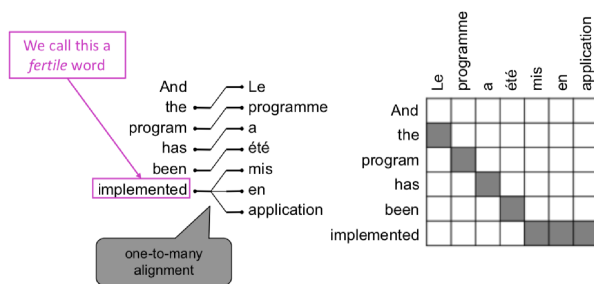


Figure 13.3: One to many.

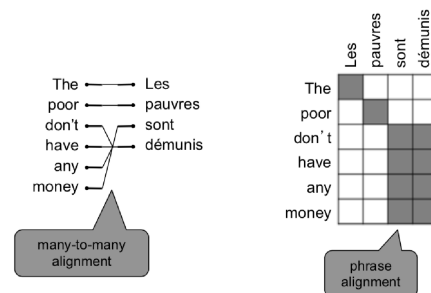


Figure 13.4: Many to many.

Formalization NMT directly models the conditional probability of the **target text** $y = y_1 \dots y_m$ given the **source text** x , using an autoregressive approach similar to language modeling:

$$P(y \mid x) = \prod_{t=1}^m P(y_t \mid y_1, \dots, y_{t-1}, x) \quad (13.1)$$

13.3 Encoder-Decoder Networks

NMT primarily relies on **encoder-decoder networks** (or sequence-to-sequence models). The two main components are:

- **Encoder:** a neural network that reads the input and produces a representation of the source sentence.
- **Decoder:** an autoregressive¹ language model that generates the target sentence word-by-word, conditioned on the encoder's output and its own previously generated tokens.

The encoder-decoder architecture can be implemented either using a RNN or else through the transformer.

¹Autoregressive = takes its own output as new input.

13.3.1 RNN-Based Architectures

In RNN-based models, the final hidden state of the encoder ($\mathbf{h}_n^e$) serves as the fixed-length **context vector** ($\mathbf{c}$) that summarizes the entire input sentence.

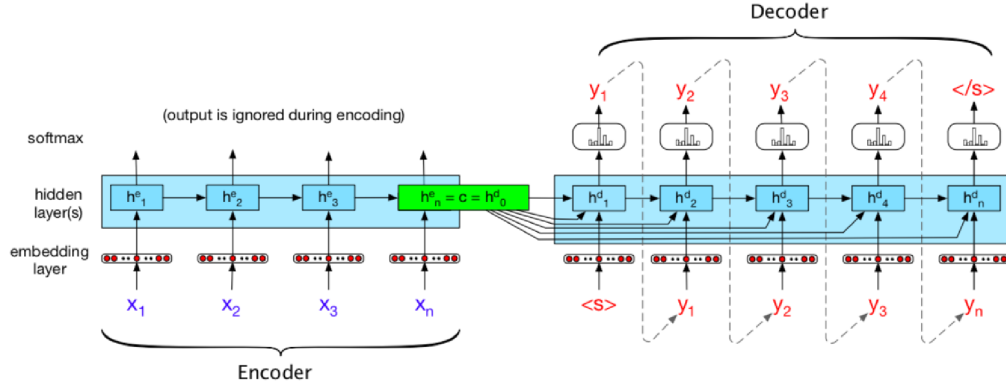


Figure 13.5: Inference overview.

Inference Using an embedding function $\mathbf{b}(u)$ for words in the vocabulary, the formal equations for an RNN-based NMT model are:

1. **Encoder inference:** run the encoder through $x = x_1 \dots x_n$ to generate the hidden states

$$\mathbf{h}_t^e = \text{RNN}(\mathbf{h}_{t-1}^e, \mathbf{b}(x_t)) \quad \forall t = 1, \dots, n.$$

2. **Define the context vector** as the last hidden state of the encoder

$$\mathbf{c} = \mathbf{h}_0^d = \mathbf{h}_n^e.$$

The context vector is used as the first hidden state of the decoder.

3. **Decoder inference:** compute the decoder hidden states using the context vector², the previous hidden state and the encoding of the previous generated word

$$\mathbf{h}_t^d = g(\mathbf{c}, \mathbf{h}_{t-1}^d, \mathbf{b}(y_{t-1})).$$

where $g(\cdot)$ is an affine transformation combined with a non-linear component.

4. **Word prediction:** just use a prediction head

$$\mathbf{z}_t = f(\mathbf{h}_t^d)$$

²Early models did not use $\mathbf{c}$ and were affected by vanishing gradient problem.

where $f(\cdot)$ is an affine transformation.

5. Predicted word:

$$\mathbf{s}_t = \text{softmax}(\mathbf{z}_t) = P(\cdot \mid \hat{y}_1, \dots, \hat{y}_{t-1}, x)$$

$$\hat{y}_t = \arg \max_{u \in V} P(u \mid \hat{y}_1, \dots, \hat{y}_{t-1}, x)$$

Training The encoder-decoder is optimized as a single system: backpropagation operates end-to-end. Given the source text and the gold translation, we compute the average loss w.r.t. our predictions on next word in the translation.

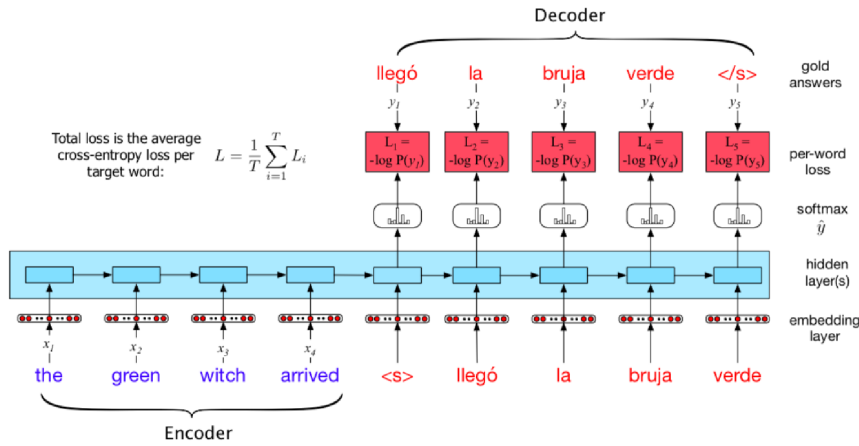


Figure 13.6: Average loss computation. During training, the decoder uses gold translation words as the input for the next step prediction (teacher forcing).

13.3.2 The Attention Mechanism

Standard RNNs suffer from the **bottleneck problem**, as the entire source sentence must be compressed into a single, fixed-length context vector $\mathbf{c}$. **Attention** solves this by computing a dynamic context vector $\mathbf{c}_i$ for each decoding step i , allowing the decoder to focus on the specific encoder states ($\mathbf{h}_j^e$) most relevant to the current word being predicted.

Attention equations The attention equations are calculated as follows:

- **Decoder inference:**

$$\mathbf{h}_i^d = g(\mathbf{c}_i, \mathbf{h}_{i-1}^d, \mathbf{b}(y_{i-1})).$$

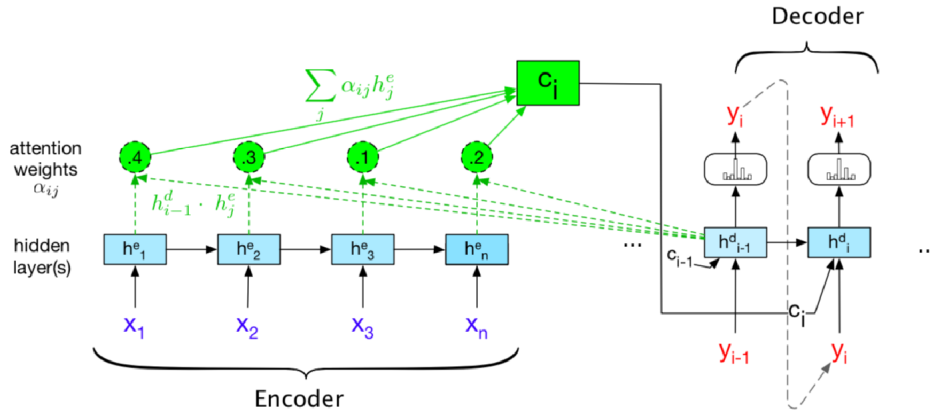


Figure 13.7: RNN with attention.

- **Compute relevance score:** using dot product³

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e \quad \forall j = 1, \dots, n$$

or something different like the bilinear

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = (\mathbf{h}_{i-1}^d)^T W_s \mathbf{h}_j^e \quad \forall j = 1, \dots, n$$

(Uses learnable parameters W_s to allow encoder and decoder to have different hidden dimensions)

- **Compute weights:**

$$\alpha_{ij} = \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k^n \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))} \quad \forall j = 1, \dots, n$$

Quantity α_{ij} tells us the proportional relevance of encoder hidden state $\mathbf{h}_j^e$ at step i .

- **Compute dynamic context vector:**

$$\mathbf{c}_i = \sum_j^n \alpha_{ij} \mathbf{h}_j^e$$

³A limitation of the dot-product score function: the same vector $\mathbf{h}_j^e$ is used both to compute the importance α_{ij} and also it's used to compute the dynamic importance $\mathbf{c}_i$.

13.3.3 Transformer-Based Architectures

The standard architecture for modern NMT is the Transformer. It modifies the basic self-attention blocks by adding a **cross-attention layer** in the decoder, which allows the model to look at the entire source language text. In the decoder, standard self-attention is causal (masked left-to-right to prevent looking into the future), while cross-attention attends to the encoder output without masking.

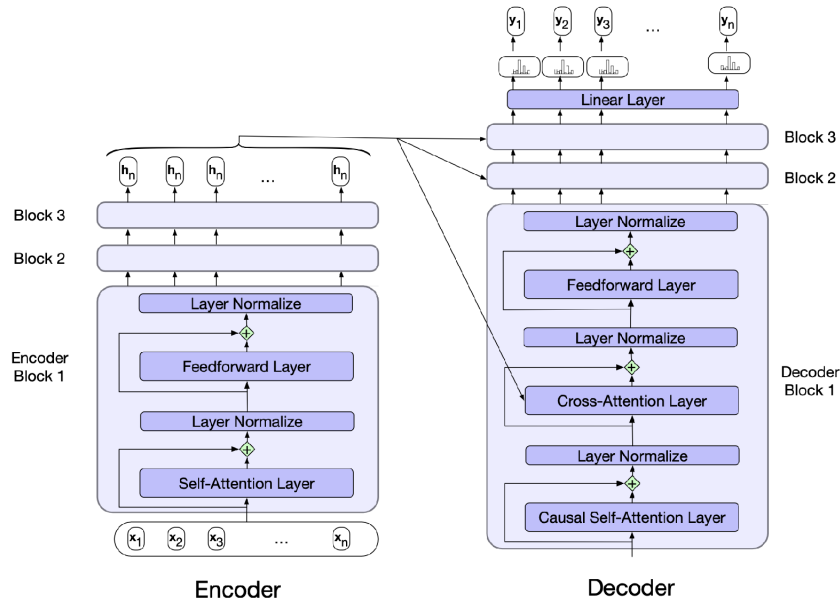


Figure 13.8: Transformer architecture. Note that this is the post-norm implementation: the layer-norm is placed after the self-attention block.

The equations Assuming H^{enc} is the topmost encoder output and H_{i-1}^{dec} is the output from the previous decoder layer, the cross-attention equations use learnable weight matrices (W_Q, W_K, W_V):

$$Q = W_Q H_{i-1}^{dec}$$

$$K = W_K H^{enc}$$

$$V = W_V H^{enc}$$

$$\text{CrossAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

The cross-attention in the decoder is a variant of the self-attention in the encoder in which the keys K and values V are computed using the last hidden state H^{enc} of the encoder.

Training During training, the network is given the source text and then, starting at the separator token, it is trained **autoregressively** to predict the next token using cross-entropy loss. As usual we use **teacher forcing**.

13.4 Inference with beam search

During inference, simply picking the most probable word at each step (greedy search) is locally optimal and may miss the best overall sentence. While exhaustive search is unfeasible, **Beam Search** acts as a compromise. It keeps the top K (beam width) best hypotheses at each step, expands them, and prunes the new branches down to the K best. When a complete hypothesis is found, remove it from the frontier and reduce by one the size of the beam, stopping at $K = 0$.

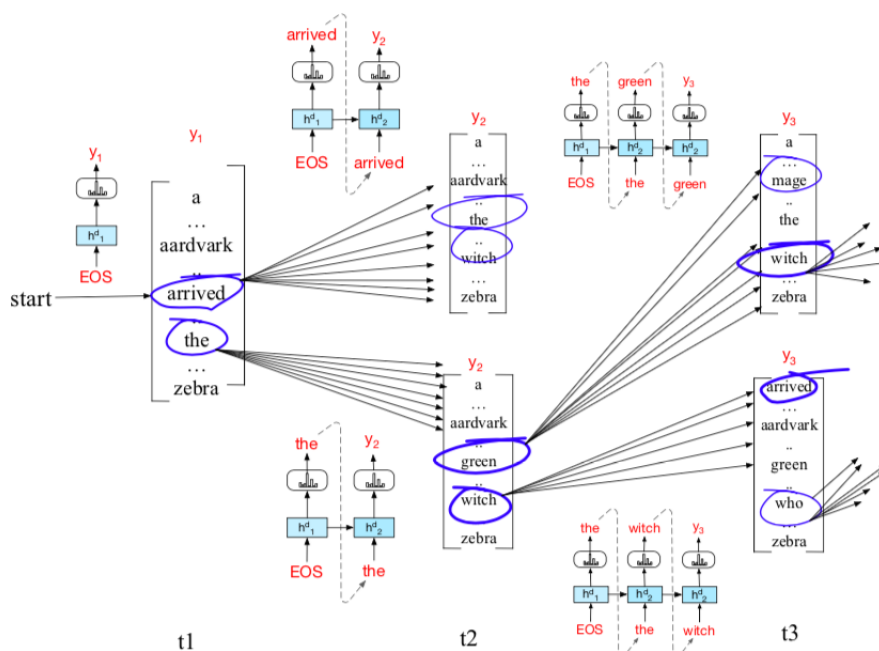


Figure 13.9: Beam search with beam width $K = 2$.

Because beam search naturally assigns lower probabilities to longer strings, some form of length normalization must be applied.

13.5 Data and Evaluation

NMT models are trained on **parallel corpora**—massive datasets of aligned sentences in two or more languages (e.g., Europarl, Canadian Hansard).

Models are automatically evaluated using two main metrics:

- **BLEU (BiLingual Evaluation Understudy):** Calculates the N-gram precision (for $N \in [1..4]$) between the generated text and the human reference, combined using a geometric mean. It includes a brevity penalty to penalize overly short translations. A score of 50%+ is considered very good, though BLEU is often criticized for poor correlation with human judgment.
- **METEOR:** Designed to fix BLEU's shortcomings by calculating the harmonic mean of unigram precision and recall, weighting recall higher. It incorporates stemming and synonymy matching, yielding better correlation with human evaluations.